



A K T U

B.Tech II-Year



CS IT & CS Allied

DATA STRUCTURE

Unit-2 : Stacks and Queues

in ONE SHOT *(Part 1 + Part 2)*



By Dr. Nidhi Parashar Ma'am

- Ph.D (CSE)
- M.Tech Gold Medalist
- Net Qualified



Pdf Notes



AKTU PYQs



Imp. Questions



CS IT & CS Allied III-Sem

Subject : DATA STRUCTURE



Unit-2 : Syllabus

Part-1

Stacks: Abstract Data Type, Primitive Stack operations: Push & Pop, Array and Linked Implementation of Stack in C, Application of stack: Prefix and Postfix Expressions, Evaluation of postfix expression, Iteration and Recursion- Principles of recursion, Tail recursion, Removal of recursion Problem solving using iteration and recursion with examples such as binary search, Fibonacci numbers, and Hanoi towers. Tradeoffs between iteration and recursion.

Part-2

Queues: Operations on Queue: Create, Add, Delete, Full and Empty, Circular queues, Array and linked implementation of queues in C, Dequeue and Priority Queue.



AKTU

B.Tech III-Sem



GATEWAY CLASSES

CS IT & CS Allied

DATA STRUCTURE

Unit-2 Lecture-1

Today's Target

- Introduction to stack
- Stack operations
- Array implementation of stack
- AKTU PYQs
- DPP



By Dr. Nidhi Parashar Ma'am

Ph.D (CSE)

- **M.Tech Gold Medalist**
- **Net Qualified**

Stack

- A stack is a linear data structure.
- Elements in the stack can be inserted and deleted from one side only.
- It follows a particular order in which the operations are performed. The order may be **LIFO**(**Last In First Out**) or **FILO**(**First In Last Out**). **LIFO** implies that the element that is inserted last, comes out first and **FILO** implies that the element that is inserted first, comes out last.

a, b, c, d



a stack of dishes

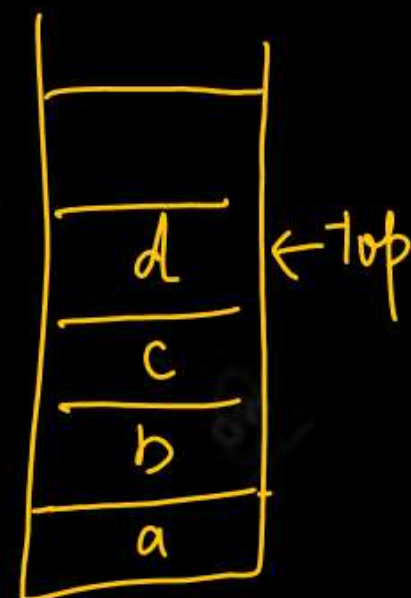
LIFO



a stack of coins



a stack of folded towels



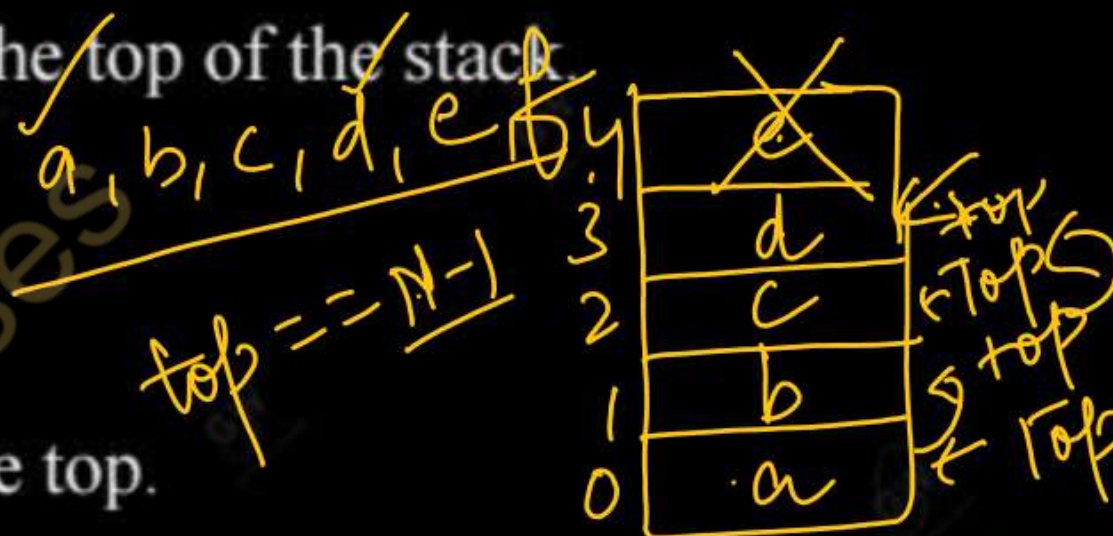
The Stack ADT is a linear data structure that follows the LIFO (Last In, First Out) principle. It allows elements to be added and removed only from one end, called the top of the stack.

Operations:

Stack ADT should also support the following operations:

1. **push():** Insert an element at one end of the stack called the top.
2. **pop():** Remove and return the element at the top of the stack, if it is not empty.
3. **peek():** Return the element at the top of the stack without removing it, if the stack is not empty.
4. **size():** Return the number of elements in the stack.
5. **isEmpty():** Return true if the stack is empty; otherwise, return false.
6. **isFull():** Return true if the stack is full; otherwise, return false. Only relevant for fixed-capacity stacks (e.g., array-based).

$$\frac{n=5}{0-4}$$



$$\text{top} = -1$$



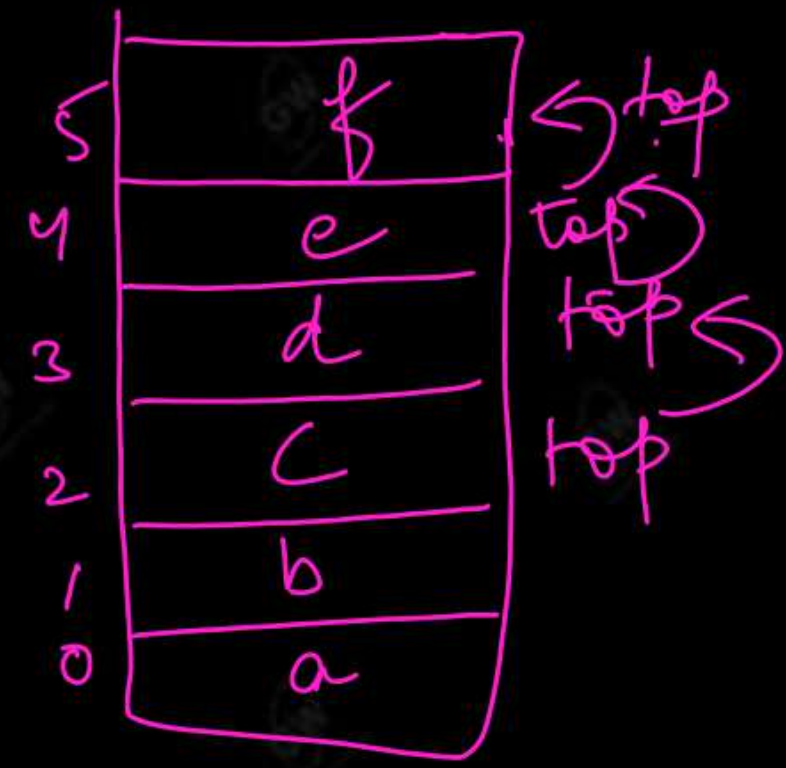
Working of Stack

N=6

a, b, c, d, e, f, g

can't be inserted

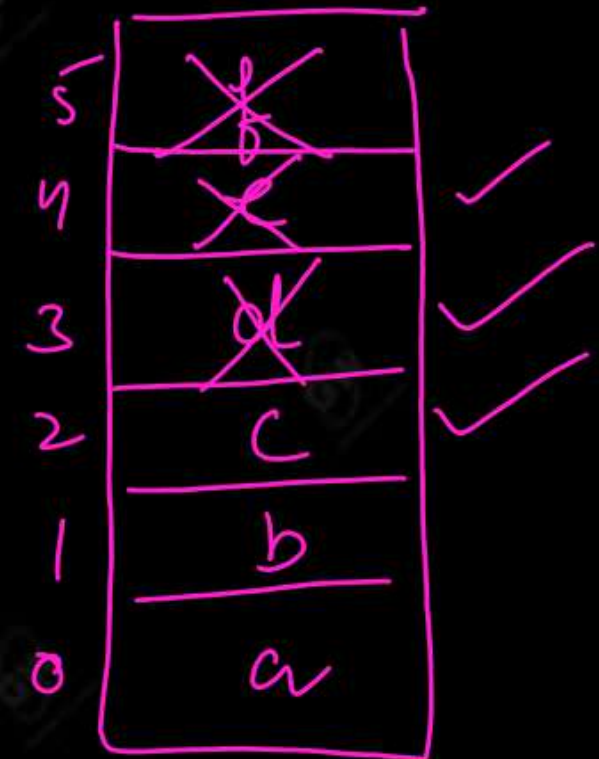
$$\text{Top} = \frac{N-1}{\text{Top} = -1}$$



top = -1

top == 5

→ overflow



Representation of Stack

1. Array Representation
2. Linked list representation

top = -1 ∅ 2

- *TOP*: a variable contains the location of the top element of stack.
- *N*: maximum number of elements that can be stored in stack or size of the linear array.
- *TOP* is *NULL*, indicate stack is empty
- *TOP* is *N*, indicate stack is full.

$$N = 7$$

given if index starts from 1.



- The *TOP* is pointing to 3 which says that stack has three items and as the $N = 6$, there is still space for accommodating four items.

Q: Write algorithm for Push and Pop operations in stack. (AKTU2019-20)

Function **PUSH** (STACK, TOP, N, ITEM)

//This function adds or pushes an ITEM in stack

If ($TOP == N - 1$)

Printf ("stack is overflow");
exit;

Else

{ $TOP = TOP + 1$;
STACK [TOP] = $ITEM$;
}

RETURN

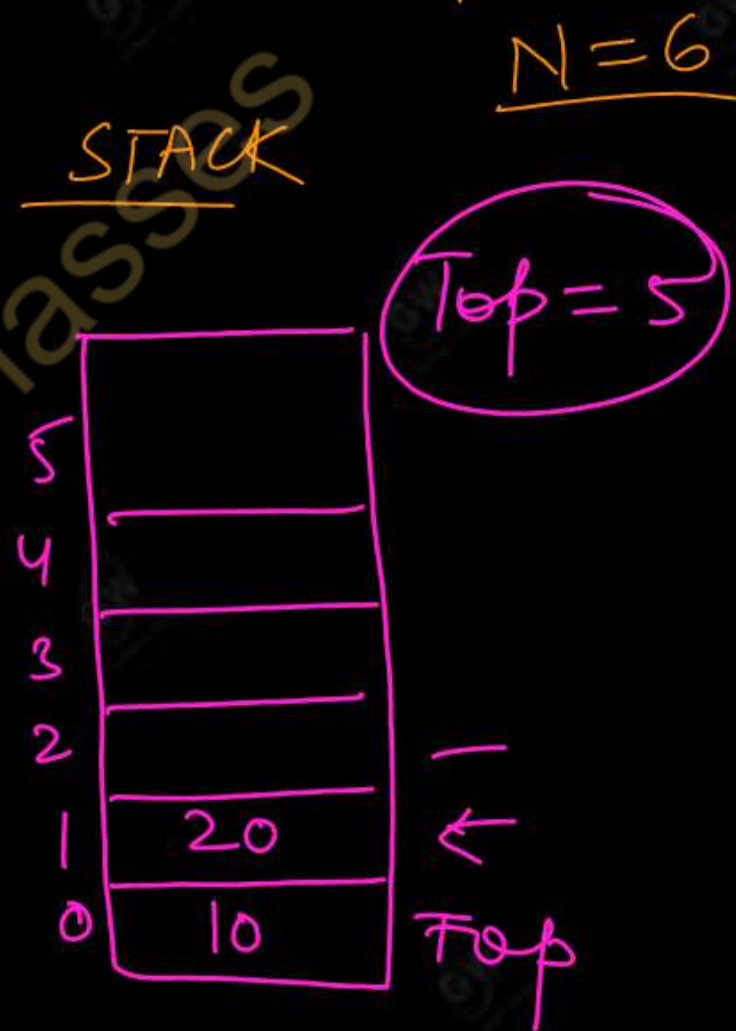
Name of stack

size of stack

element to be pushed in the stack

variable that holds the index of topmost element

element



$Top == 5$
↓
overflow

top = -1

Contd...

Function *POP (STACK, TOP, ITEM)*

[This function deletes/pops the *TOP* element of *STACK* and assigns it to variable *ITEM*]

If ($TOP == -1$)

Printf ("stack is underflow");

exit;

Else

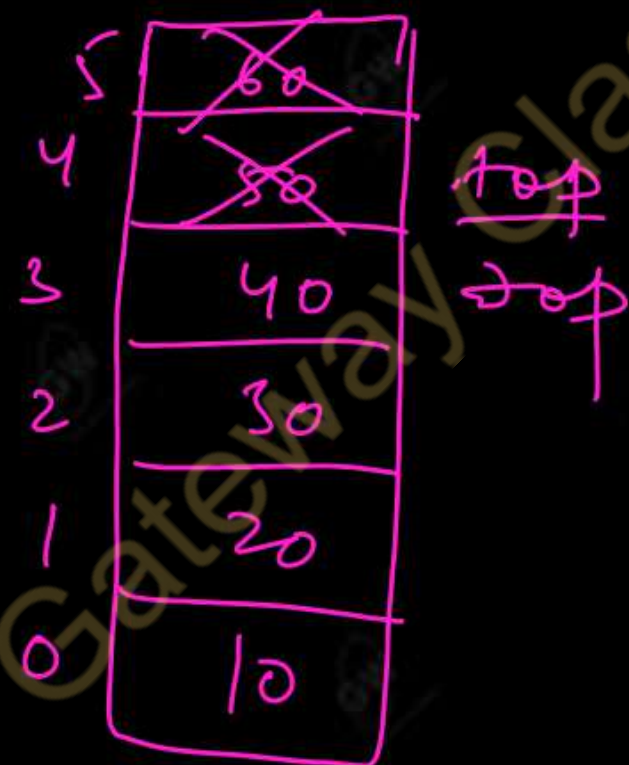
{

$ITEM = STACK[TOP];$

$TOP = TOP - 1;$

}

RETURN



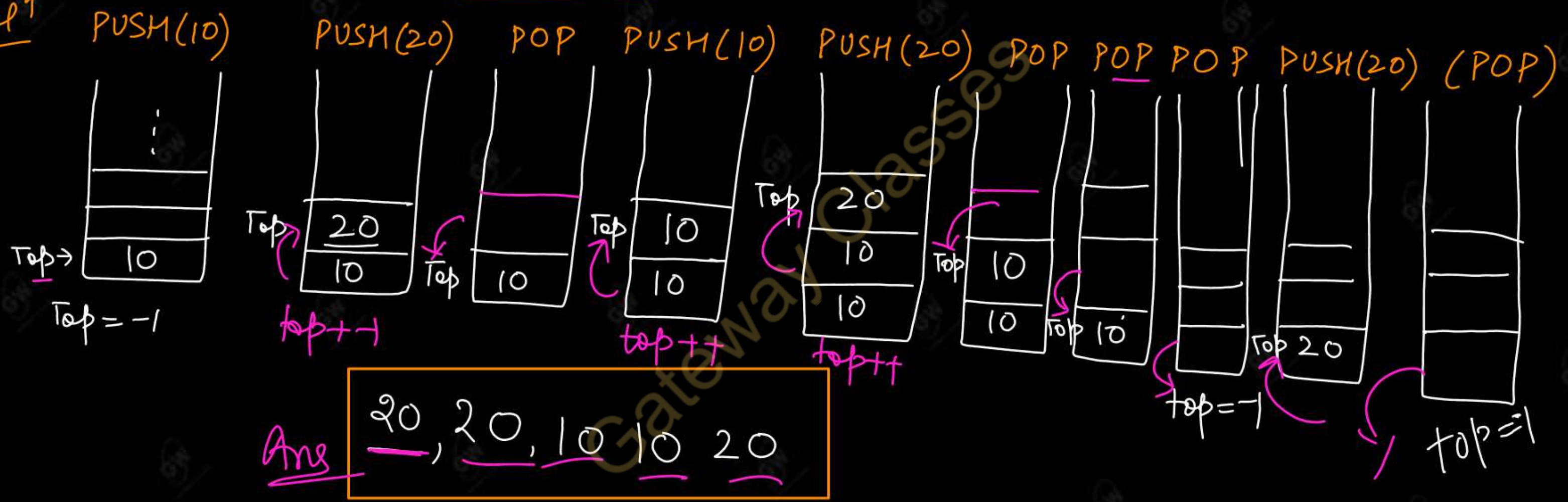
= $STACK[5]$

60

→ $top = -1$

Q: The following sequence of operations is performed on stack: PUSH (10), PUSH (20), POP, PUSH (10), PUSH (20), POP, POP, POP, PUSH (20), POP. What is the sequence of values popped out? (AKTU2024-25)

Solⁿ



Ans

20, 20, 10, 10, 20

```
#include <stdio.h>
```

```
#define N 5
```

```
int stack[N];
```

```
int top = -1;
```

```
void Push()
```

```
{ int x;
```

```
    printf("Enter data");
```

```
    scanf("%d", &x);
```

```
    if (top == N-1)
```

```
    { printf("Overflow"); }
```

```
    else { top++;
```

```
          stack[top] = x;
```

```
    } }
```

Array Implementation of Stack

```
void Pop()
```

```
{ int item;
```

```
    if (top == -1) // Stack is Empty
```

```
    { printf("Underflow"); }
```

```
    else { item = stack[top];
```

```
           top--;
```

```
           printf("%d", item);
```

```
    }
```

```
void Peek()
```

```
{ if (top == -1)
```

```
    { printf("Empty stack\n"); }
```

```
    else { printf("Top element: %d\n", stack[top]); } }
```

```
void Display()
```

```
{ int i;
  if (top == -1)
  { printf("Stack is empty\n"); }
  else
  { printf("Stack elements: "); }
  for (i = top; i >= 0; i--)
  { printf("%d ", stack[i]);
    printf("\n");
  }
}
```



50
40
30
20
10

```
void main()
```

```
{ int ch;
  do
  { printf("\nEnter choice: 1: Push 2: Pop 3: Peek 4: Display
    0: Exit\n");
    scanf("%d", &ch);
    switch (ch)
    { case 1: Push();
      break;
      case 2: Pop();
      break;
      case 3: Peek();
      break;
      case 4: Display();
      break;
      default: printf("Invalid choice\n");
    }
  } while (ch != 0); }
```

```
int isEmpty()
{ if (top == -1)
  return 1;
  else return 0;
}
```

```
int isfull()
{ if (top == N-1)
  return 1;
  else return 0;
}
```

Time Complexity of Stack Operations

Operation	Time Complexity
Push()	$O(1)$ ✓
Pop()	$O(1)$ ✓
Peek()	$O(1)$ ✓
Traversal/Display	$O(n)$ ✓
IsEmpty()	$O(1)$ ✓
IsFull()	$O(1)$ ✓

Q: Write a C program to reverse a string using stack. (AKTU2021-22)

```
#include<stdio.h>
#include<string.h>
#include<stdlib.h>
#define MAX 20
int top = -1;
char stack[MAX];
void push(char item)
```

```
{ if(top == (MAX-1))
```

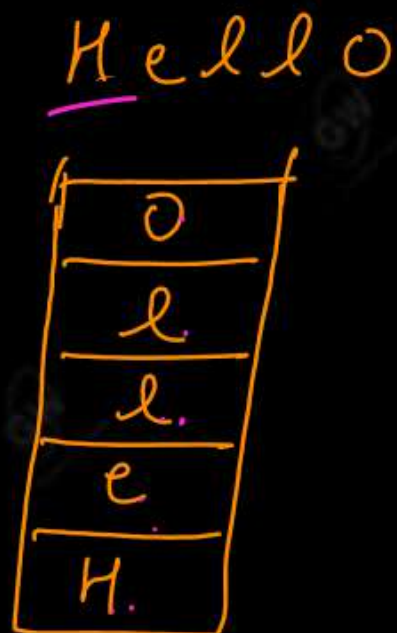
```
{ printf("\nStack Overflow\n");
```

```
return;
```

```
}
```

```
stack[++top] = item;
```

```
}
```



Handwritten string "olleH" representing the reversed string.

Handwritten annotations for the push operation:

- $top = top + 1$ (with an arrow pointing to the $++$ in `stack[++top]`)
- $stack[top] = item$ (with an arrow pointing to the assignment)

```
char pop()
```

```
{
```

```
if(top == -1)
```

```
{ printf("\nStack Underflow\n");
```

```
exit(1);
```

```
}
```

```
return stack[top--];
```

```
}
```

```
int main()
```

```
{ char str[20];
```

```
  unsigned int i;
```

```
  printf("Enter the string : " );
```

```
  gets(str);    //Push characters of the string str on the stack
```

```
  for(i=0; i<strlen(str); i++) // i = 0 --- 4
```

```
    push(str[i]); //Pop characters from stack and store in string
```

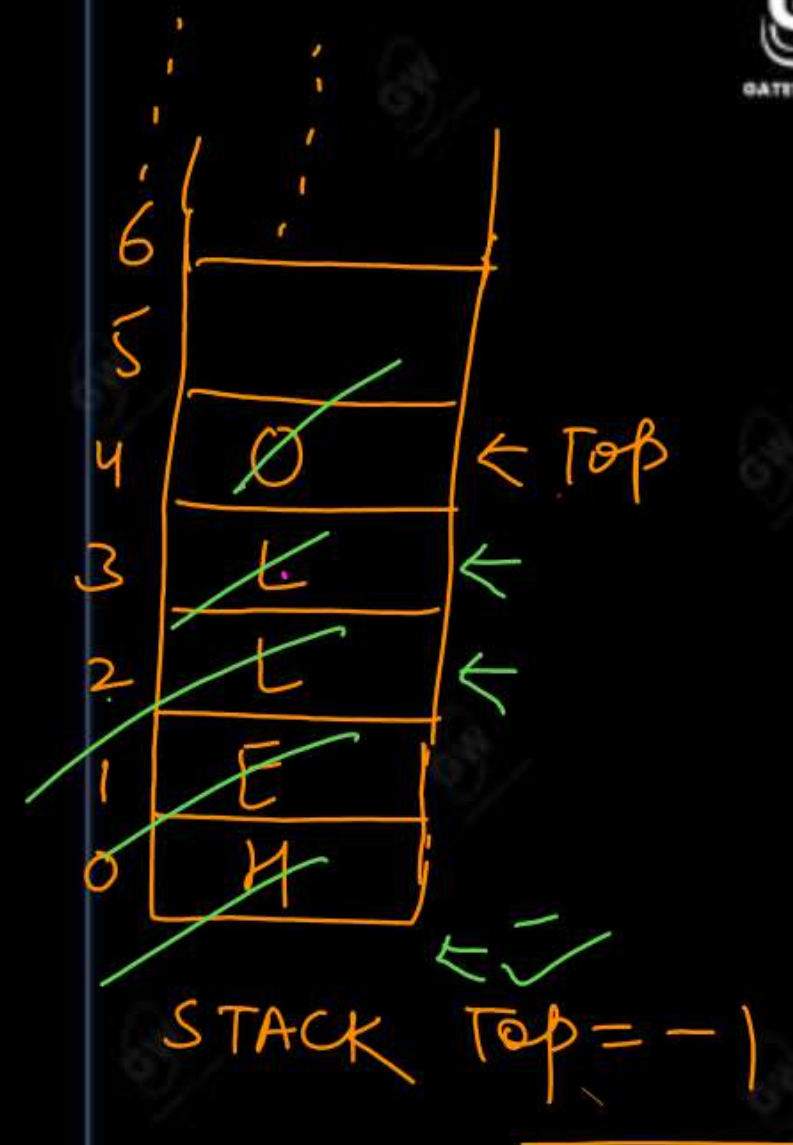
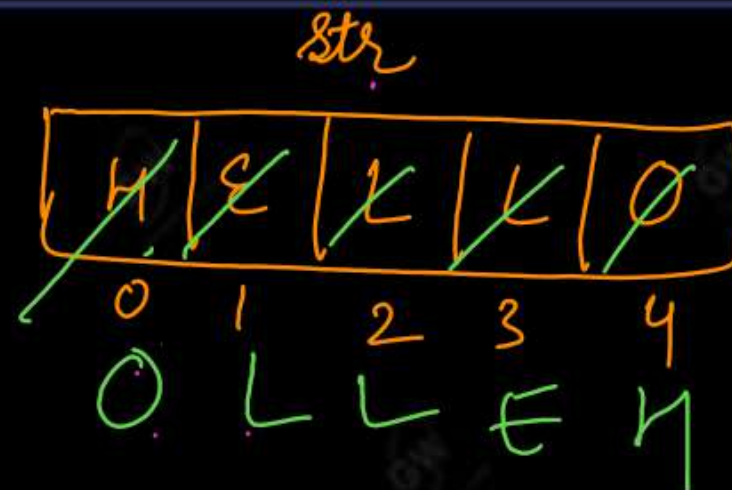
```
  for(i=0; i<strlen(str); i++)
```

```
    str[i]=pop();
```

```
  printf("\nReversed string is : ");
```

```
  puts(str);
```

```
  return 0; }
```



Data Structure

Unit-2 DPP-1

1. Write algorithm for Push and Pop operations in stack. **(AKTU2019-20)**
2. The following sequence of operations is performed on stack: PUSH (10), PUSH (20), POP, PUSH (10), PUSH (20), POP, POP, POP, PUSH (20), POP. What is the sequence of values popped out?
(AKTU2024-25)
3. Write a Program in C to implement all operations of a Stack using array. **(AKTU2019-20)**
4. Write a C program to reverse a string using stack. **(AKTU2021-22)**



AKTU

B.Tech III-Sem



GATEWAY CLASSES

CS IT & CS Allied

DATA STRUCTURE

Unit-2 Lecture-2

Today's Target

- Linked list implementation of stack
- AKTU PYQs
- DPP



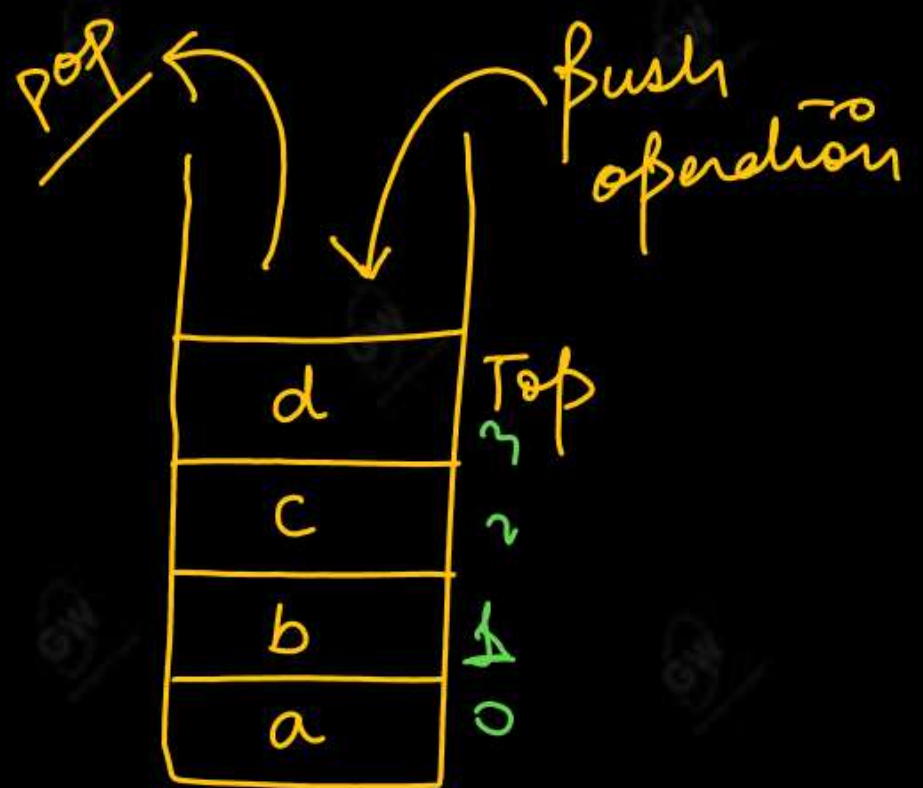
By Dr. Nidhi Parashar Ma'am

Ph.D (CSE)

- M.Tech Gold Medalist
- Net Qualified

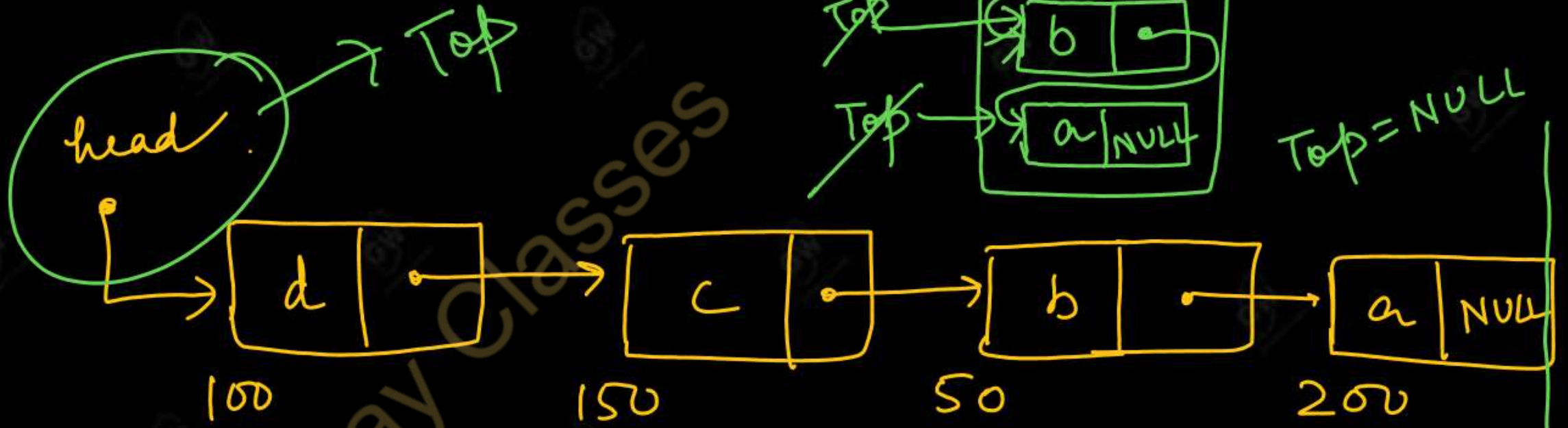
Linked list Representation of Stack

Stack



top = -1 1) LIFO

linked list



2) Top $\Rightarrow$ Add each new element at beginning of linked list.

Linked list Representation of Stack

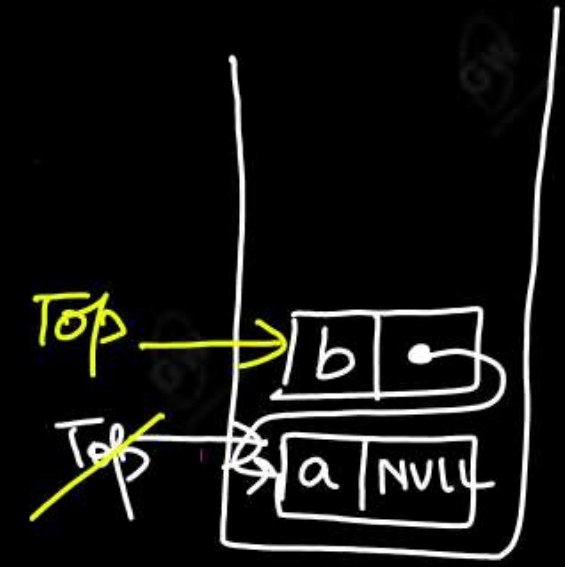
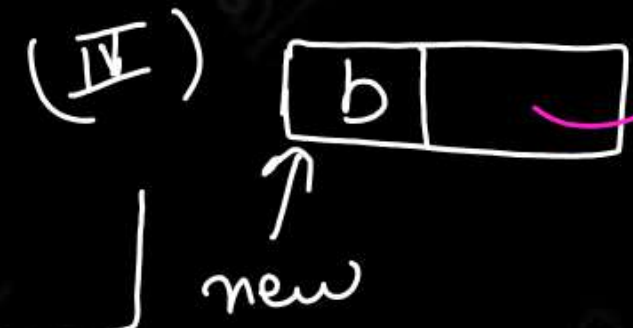
Sequence of values pushed into the stack :- a, b, c, d

(I) top = NULL

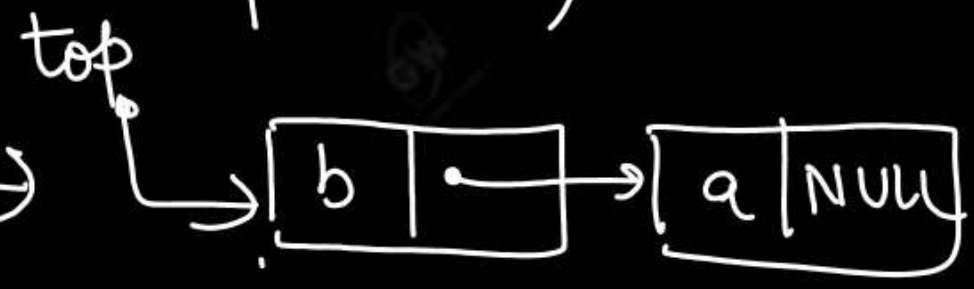
Struct node

```
{ int data;  
  struct node* next;  
};
```

struct node* top = NULL, *new;



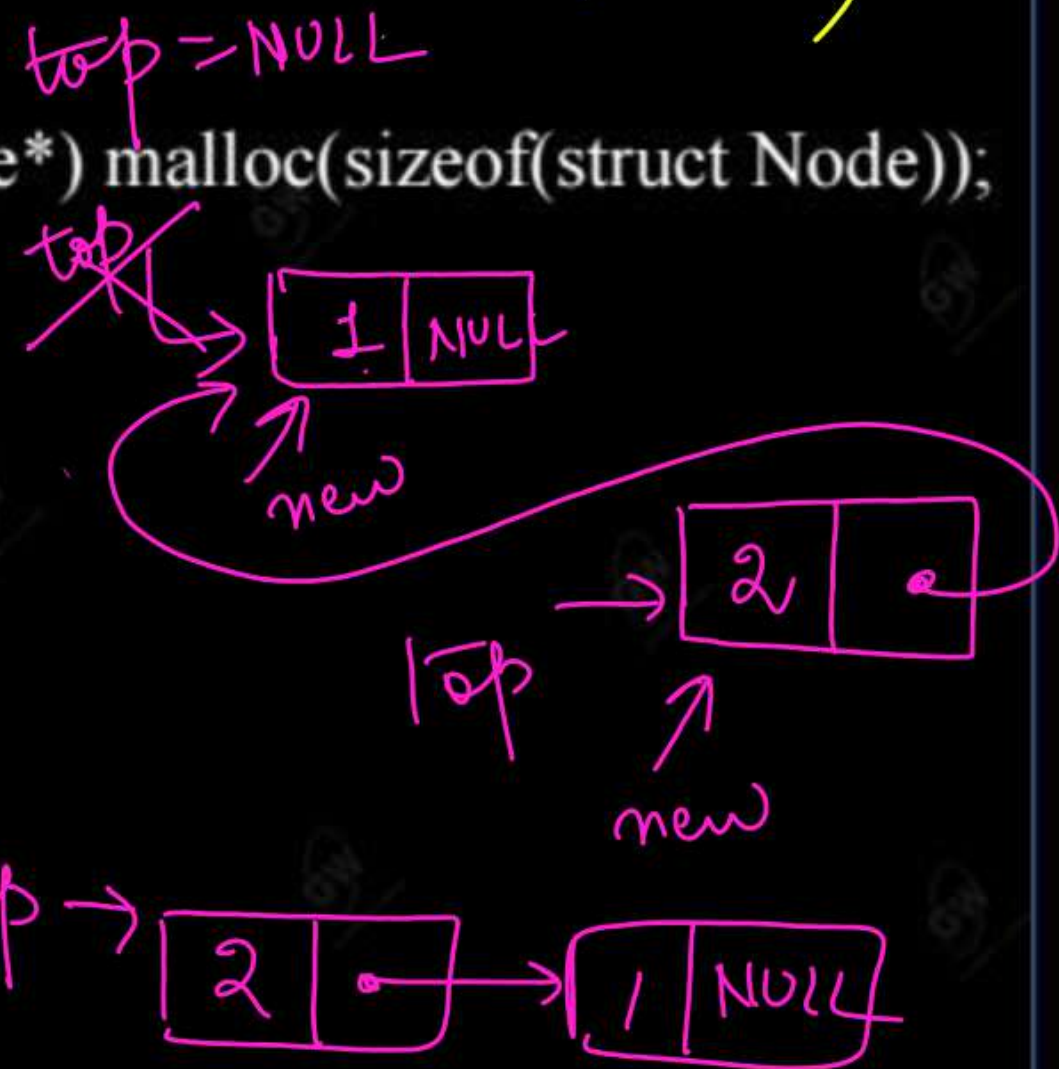
new → next = top;
top = new;



Linked list Representation of Stack (AKTU 2024-25, 22-23, 21-22, 18-19)

```
#include <stdio.h>
#include <stdlib.h>
struct Node
{
    int data;
    struct Node *next;
};
struct Node *top = NULL;
int isempty() // Returns 1 if
               // empty stack, and
               // returning 0 otherwise
{
    return (top == NULL);
}
```

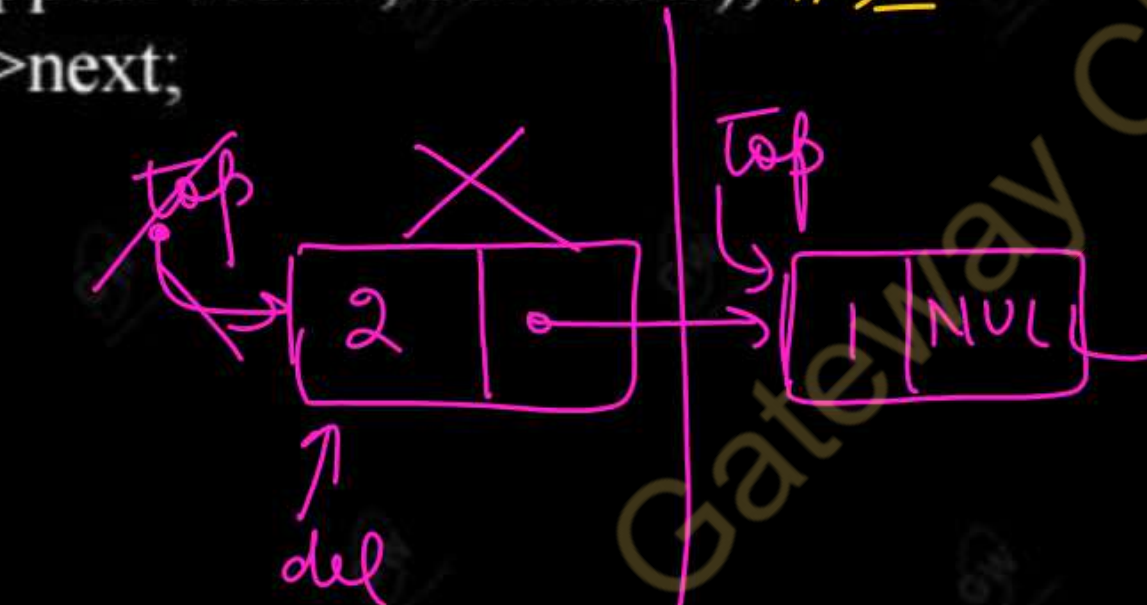
```
void push(int value)
{
    struct Node *new = (struct Node*) malloc(sizeof(struct Node));
    if (new == NULL)
    {
        printf("Heap Overflow\n");
        return;
    }
    new->data = value;
    new->next = top;
    top = new;
    printf("%d pushed onto stack\n", value);
}
```



```
void pop()
```

```
{
    if (isempty())
    {
        printf("Stack is Empty\n"); }
    else
```

```
{
    struct Node *del = top;
    printf("Popped: %d\n", del->data); // 2
    top = top->next;
    free(del);
}
```



```
void peek()
```

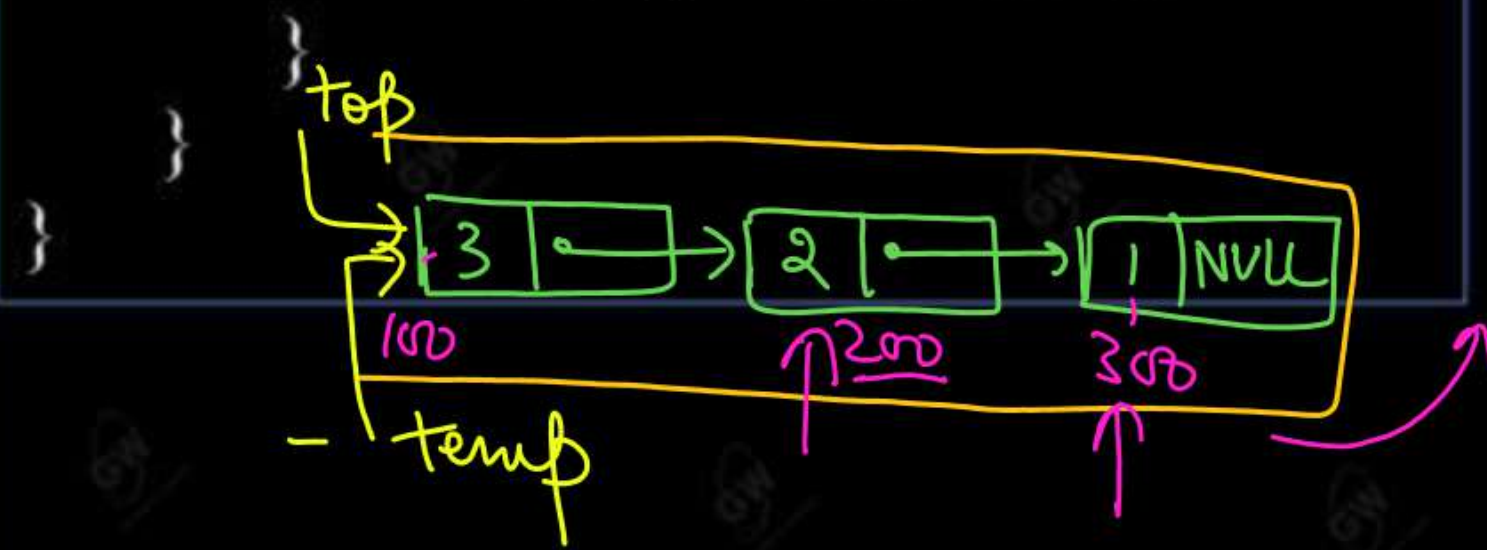
```
{
    if (isempty())
    {
        printf("Stack is Empty\n"); }
    else
    {
        printf("Element at top is: %d\n", top->data);
    }
}
```

```
void display()
```

```
{
    if (isempty())
    {
        printf("Stack is Empty\n"); }
    else
```

```
{
    struct Node *temp = top;
    printf("Stack elements: ");
    while (temp != NULL)
    {
```

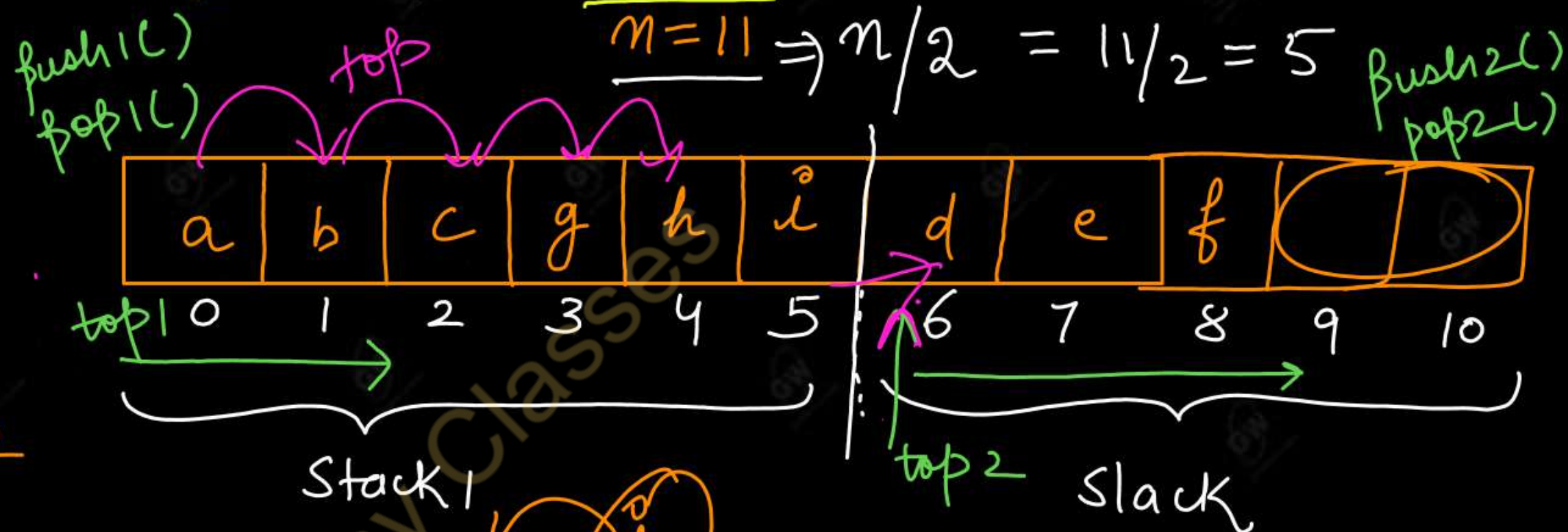
```
        printf("%d ", temp->data);
        temp = temp->next;
    }
```



```
int main()
{
    int choice, value;
    while (1)
    {
        printf("\n1. Push 2. Pop 3. Show Top 4. Display Stack 5. Exit\n");
        scanf("%d", &choice);
        switch (choice)
        {
            case 1: printf("Enter Value: ");
                    scanf("%d", &value);
                    push(value);
                    break;
            case 2: pop();
                    break;
            case 3: peek();
                    break;
            case 4: display();
                    break;
            case 5: printf("Exiting program.\n");
                    exit(0);
            default: printf("Invalid choice! Try again.\n");
        }
    }
    return 0;
}
```

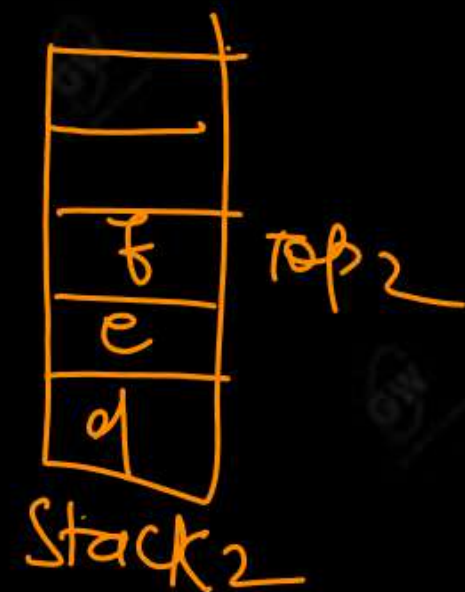
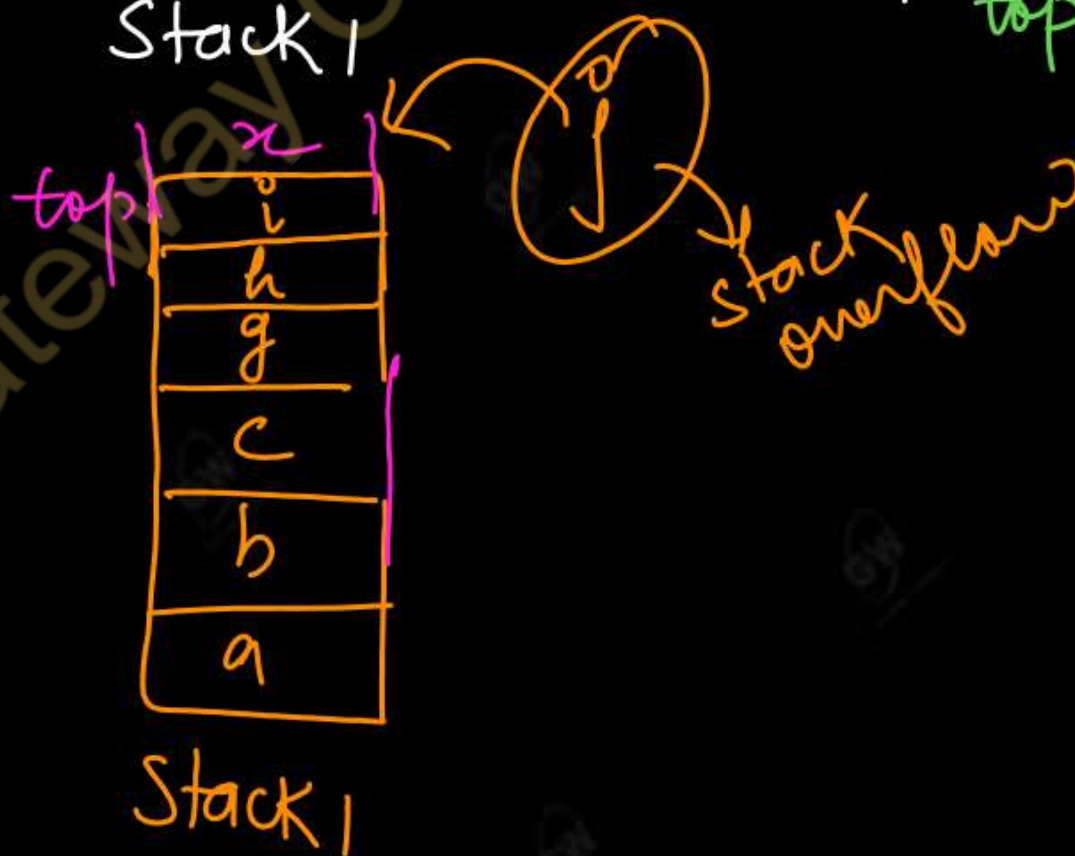
Q: Design a method for keeping two stacks within a single linear array so that neither stack overflow until all the memory is used. (AKTU2021-22)

I Approach

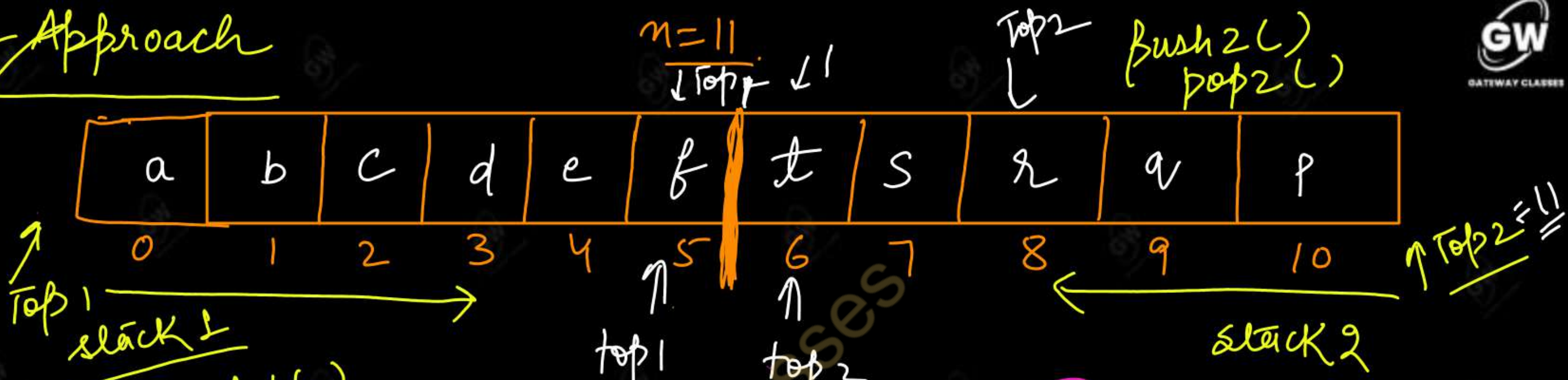


Stack 1) $\rightarrow 0 - n/2$

Stack 2) $\rightarrow n/2 + 1 - n - 1$



II Approach



$(Top1 + 1 == Top2)$ overflow

$Top1 = -1$
 $Top2 = 11$

Correct Approach

$$Top1 < Top2 - 1$$

validity for push operation

No-overflow situation

Data Structure

Unit-2 DPP-2

1. How do you push elements in a linked stack? **(AKTU2018-19)**
2. If a stack is implemented using Linked List, then show the logical representation of nodes as a stack. Illustrate PUSH and POP operations of this type of stack. **(AKTU2024-25,2022-23,2021-22,2018-19)**
3. Design a method for keeping two stacks within a single linear array so that neither stack overflow until all the memory is used. **(AKTU2021-22)**



AKTU

B.Tech III-Sem



GATEWAY CLASSES

CS IT & CS Allied

DATA STRUCTURE

Unit-2 Lecture-3

Today's Target

- Arithmetic expression (infix, postfix and prefix notions)
- Conversion from Infix to postfix expression using stack
- AKTU PYQs
- DPP



By Dr. Nidhi Parashar Ma'am

Ph.D (CSE)

- **M.Tech Gold Medalist**
- **Net Qualified**

Arithmetic expression (infix, postfix and prefix notions)

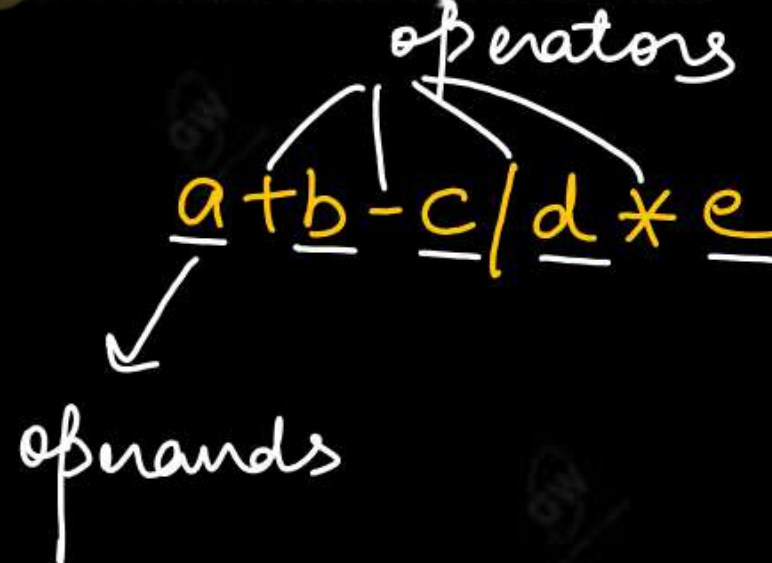
Infix, Postfix and Prefix Expressions/Notations

- Mathematical formulas involve complex expressions that require a clear understanding of the order of operations.
- To represent these expressions, there are three common expression notations:

1. Infix
2. prefix
3. postfix

Infix Expressions

Mathematical expressions where **operator** is placed **between its operands**.

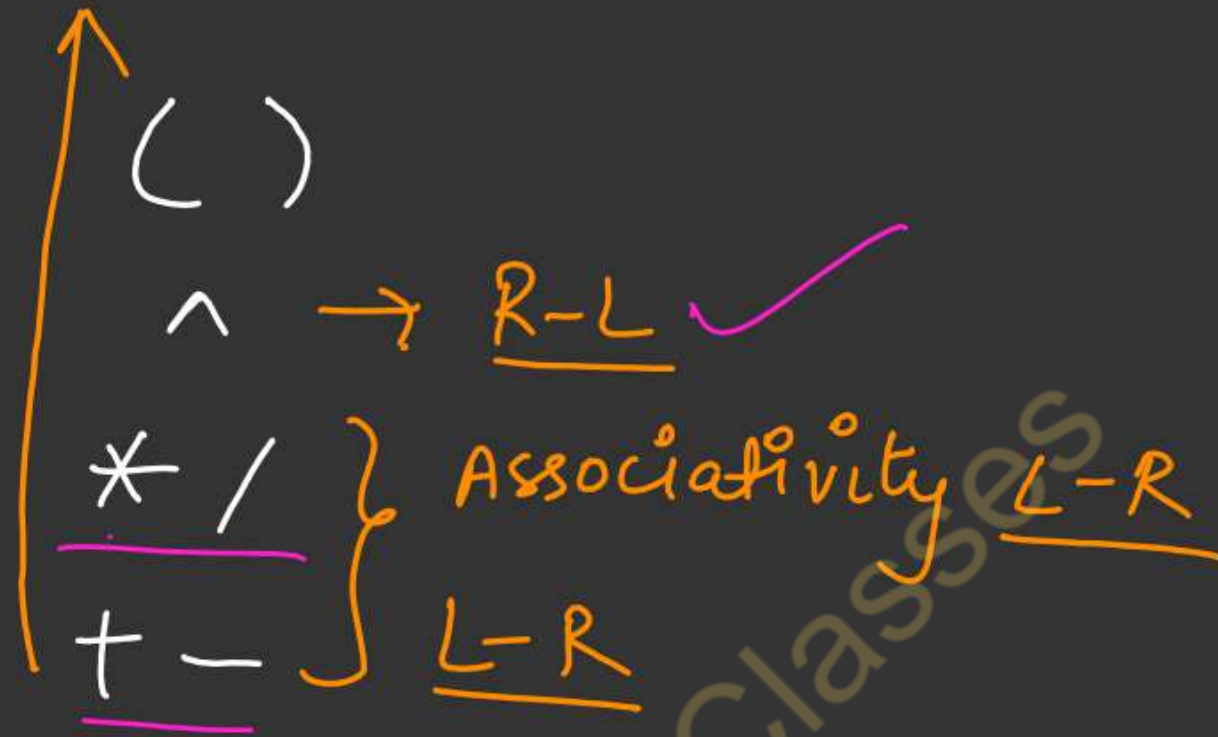


BOĐMÁŠ

Infix

$$(a+b) * c / d$$

Precedence ↑



$$a + b * c / d$$

$$\Rightarrow a + (b * c) / d$$

$$\Rightarrow a + (b * c) / d$$

$$\Rightarrow \underline{a + ((b * c) / d)}$$

Evaluating Infix Expressions

Evaluating infix expressions requires additional processing to handle the order of operations and parentheses.

Advantages:

- Easier to read and understand for humans.
- Widely used and supported by most programming languages and calculators.

Disadvantages: ()

- Requires parentheses to specify order of operations.
- Can be difficult to parse and evaluate efficiently.

$$2^3 + 5 * 2^2 - 12 / 6$$

$$2^3 + 5 * 4 - 12 / 6$$

$$8 + 5 * 4 - 12 / 6$$

$$8 + 20 - 12 / 6$$

$$8 + 20 - 2$$

$$28 - 2$$

$$= 26$$

$$\begin{array}{l} () \\ \wedge \\ * / \\ + - \end{array} \begin{array}{l} R-L \\ L-R \\ L-R \end{array}$$

Stack is more efficient to evaluate an infix arithmetical expression by first converting to a *prefix* or *postfix* expression and then evaluating these converted expressions.

- This approach eliminate repeated scanning of an infix expression in order to obtain its value.
- Three notations for representation of any complex arithmetic expression. These forms are as follows:
- **Infix:** $a + b$
- **Prefix (polish notation):** If the operator symbols are placed before its operands, then the expression is in *prefix form*. $+ a b$
- **Postfix (Reverse polish notation):** If the operator symbols are placed after its operands, then the expression is in *postfix form*. $a b +$

Convert to prefix:

$$(A+B) * C \Rightarrow \underbrace{+AB}_I \underbrace{*C}_II \Rightarrow *+ABC$$

$$A + (B * C) \Rightarrow \underbrace{A}_I + \underbrace{*BC}_II \Rightarrow +A*BC$$

$$(A+B) / (C-D) \Rightarrow \underbrace{+AB}_I / \underbrace{-CD}_II \Rightarrow /+AB-CD$$

operator operand₁ operand₂

Convert to postfix:

$$(A+B) * C \Rightarrow \underbrace{AB+}_I \underbrace{*C}_II \Rightarrow AB+C*$$

$$A + (B * C) \Rightarrow \underbrace{A}_I + \underbrace{BC*}_II \Rightarrow ABC*+$$

$$(A+B) / (C-D) \Rightarrow \underbrace{AB+}_I / \underbrace{CD-}_II \Rightarrow AB+CD- /$$

operand₁ operand₂ operator

Q: Why are parentheses needed to specify the order of operations in infix expressions but not in postfix operations? (AKTU2023-24)

- Infix notation is human-readable format, which need brackets/ Parenthesis based on BODMAS rule to understand the order of execution.
- The postfix or prefix are machine-readable format. they are actually conversions of the original standard equation where the brackets have already been taken into consideration and the formed prefix/postfix expression is the correct order of expansion of a given mathematical statement.
- Evaluation of postfix or prefix notations are done with the help of stack.

(AKTU2023-24, 2022-23, 2019-20)

Algorithm: Conv_REVERSE_POLISH (Q, P) Q: Infix arithmetic expression

P: Equivalent postfix expression P

1. Push "(" onto STACK, and add ")" to the end of Q.
2. Scan Q from left to right and repeat steps 3 to 9 for each element of Q until the STACK is empty.
3. If an operand is encountered, add it to P.
4. If a left parenthesis is encountered, push it onto STACK.
5. *If operator is encountered* If stack is empty or contains a left parenthesis on top, push the incoming operator onto the stack.
6. If incoming symbol is ')', pop the stack & add operators to P until left parenthesis is found.
7. *operator* If incoming symbol has higher precedence than top of the stack, push it on the stack.
8. If incoming symbol has lower precedence than top of the stack, pop & add the top to P. Then test the incoming operator against the new top of the stack.

9. If incoming operator has equal precedence with the top of the stack, use associativity rule.

→ Associativity L to R: pop & add top of stack to P & then push the incoming operator.

→ Associativity R to L: then push the incoming operator.

[END of Step-2 loop]

10. Exit

- This algorithm uses a stack to temporarily hold operators and left parentheses.
- We begin by pushing a left parenthesis onto STACK and adding a right parenthesis at the end of Q .

The algorithm is completed when STACK is empty

Q: Trace algorithm to convert infix expression Q into its equivalent postfix expression P

$$Q = A + B * C - D / F$$

(AKTU2023-24)

Symbol	Stack	Expression (P)
A	(	A
+	(+	A
B	(+	AB
*	(+ *	AB
C	(+ *)	ABC
-	(-	ABC * +
D	(-	ABC * + D
/	(- /	ABC * + D
F	(- /	ABC * + DF
)	-	ABC * + DF / -

Ans.
ABC * + DF / -

Q: Trace algorithm to convert infix expression Q into its equivalent postfix expression P

$$A + (B * C - (D / E \wedge F) * G) * H$$

(AKTU2022-23, 2019-20)

Symbol	Stack	Expression
A	(	A
+	(+	A
(	(+ (	A
B	(+ (	A B
*	(+ (*	A B
C	(+ (*	A B C
-	(+ (-	A B C *
(	(+ (- (	A B C *
D	(+ (- (	A B C * D

$$A + (B * C - (D / E \wedge F) * G) * H$$

Symbol	Stack	Expression
/	(+ (- (/	ABC * D
E	(+ (- (/	ABC * DE
^	(+ (- (/ ^	ABC * DE
F	(+ (- (/ ^	ABC * DEF
)	(+ (-	ABC * DEF ^ /
*	(+ (- *	ABC * DEF ^ /
G	(+ (- *	ABC * DEF ^ / G
)	(+	ABC * DEF ^ / G * -
*	(+ *	ABC * DEF ^ / G * -
H	(+ *	ABC * DEF ^ / G * - H
)		ABC * DEF ^ / G * - H * +

Ans.

ABC * DEF ^ / G * - H * +

$$A + (B * C - \underbrace{(D / E \wedge F)} * G) * H$$

$$A + (B * C - \underbrace{(D / EF \wedge)} * G) * H$$

$$A + (\underbrace{B * C} - \underbrace{DEF \wedge /} * G) * H$$

$$A + (\underbrace{BC *}_{\quad} - \underbrace{DEF \wedge /} * G) * H$$

$$A + (BC * - \underbrace{DEF \wedge / G *}_{\quad}) * H$$

$$A + \underbrace{BC * DEF \wedge / G * -}_{\quad} * \underbrace{H}_{\quad}$$

$$A + \underbrace{BC * DEF \wedge / G * - H *}_{\quad}$$

$$A BC * DEF \wedge / G * - H * +$$

1. Why are parentheses needed to specify the order of operations in infix expressions but not in postfix operations? **(AKTU2023-24)**
2. Write an algorithm to convert a valid arithmetic infix expression into an equivalent postfix expression. Trace your algorithm for following infix expression. $A+B*C-D/F$ **(AKTU2023-24)**
3. Write an algorithm for converting infix expression into postfix expression. Trace your algorithm for infix expression Q into its equivalent postfix expression P, Q: $A + (B * C - (D / E ^ F) * G) * H$ **(AKTU2022-23, 2019-20)**
4. Transform expression into its equivalent postfix expression using stack: $A + (BC - (D/E \uparrow F) * G) * H$. **(AKTU2019-20)**

Data Structure

Unit-2 DPP-3

5. Convert the infix expression $(A+B) * (C-D) \$ E * F$ to postfix. Give the answer without any spaces.

(AKTU2021-22)

6. Convert the following infix expression to reverse polish notation expression using stack.

$$x = \frac{-b + \sqrt{b^2 - 4ac}}{2a} \quad \text{(AKTU2021-22)} \quad \checkmark$$

7. Convert the following infix expression into postfix expression: $B - C / D + A * (F - G / H)$ **(AKTU2019-20)**

8. Convert the following arithmetic infix expression into its equivalent postfix expression.

Expression: $A - B / C + D * E + F$ **(AKTU2017-18)**



AKTU

B.Tech III-Sem



GATEWAY CLASSES

CS IT & CS Allied

DATA STRUCTURE

Unit-2 Lecture-4

Today's Target

- More solved problems of infix to postfix conversion
- Evaluation of postfix expression using stack
- Conversion of *infix* into *prefix* expression
- Conversion of *postfix* into *infix* expression
- Conversion of *prefix* into *infix* expression
- AKTU PYQs
- DPP



By Dr. Nidhi Parashar Ma'am

Ph.D (CSE)

- **M.Tech Gold Medalist**
- **Net Qualified**

Q: Convert infix expression to postfix. Give the answer without any spaces. (AKTU2021-22)

$(A+B) * (C-D) * E * F$ → highest precedence

Symbol	Stack	Postfix Expression
(	((	
A	((	A
+	((+	A
B	((+	AB
)	(	AB+
*	(*	AB+
(	(* (	AB+
C	(* (	AB+C
-	(* (-	AB+C
D	(* (-	AB+CD
)	(*	AB+CD-

()
^
*/
+-

$$(A+B) * (C-D) \$E * F$$

Symbol	Stack	Expression
\$	(* \$	AB+CD-
E	(* <u>\$</u>	AB+CD-E
*	(*	AB+CD-E\$*
F	(*	AB+CD-E\$*F
)	_	AB+CD-E\$*F*
		AB+CD-E\$*F*

Ans.

$$(A+B) * (C-D) \$E * F$$

$$\underbrace{AB+}_{\text{ }} * \underbrace{CD-}_{\text{ }} \$E * F$$

$$\underbrace{AB+}_{\text{ }} * \underbrace{CD-E\$}_{\text{ }} * F$$

$$\underbrace{AB+CD-E\$*}_{\text{ }} * F$$

$$AB+CD-E\$*F*$$

Q: Convert infix expression to postfix. Give the answer without any spaces. (AKTU2019-20)

$$B-C/D+A*(F-G/H)$$

Symbol	Stack	Expression
B	(	B
-	(-	B
C	(-	BC
/	(-/	BC
D	(-/	BCD
+	(+	BCD/-
A	(+	BCD/-A
*	(+*	BCD/-A
(	(+*(	BCD/-A

$$B-C/D+A*(F-G/H)$$

Symbol	Stack	Expression
F	(+ * (	BCD / - A F
-	(+ * (-	BCD / - A F
G	(+ * (-	BCD / - A F G
/	(+ * (- /	BCD / - A F G
H	(+ * (- /	BCD / - A F G H
)	(+ *	BCD / - A F G H / -
)	—	BCD / - A F G H / - * +
		Ans

Q: Convert the following infix expression to reverse polish notation expression using stack.

$$x = \frac{-b + \sqrt{b^2 - 4ac}}{2a} \quad (\text{AKTU2021-22})$$

$$\sqrt{b^2 - 4ac} = (b^2 - 4ac)^{1/2}$$

$$x = \left(-b + (b^2 - (4 * a * c))^{(1/2)} \right) / (2 * a)$$

Symbol	Stack	Expression
x	(	x
=	(=	x
(	(=(	x
-	(=(- unary minus	x
b	(=((-	x b
+	(=(+	x b -
(	(=(+ (	x b -
b	(=(+ (	x b - b
^	(=(+ (^	x b - b
2	(=(+ (^	x b - b 2
-	(=(+ (-	x b - b 2 ^

$$x = \frac{-b + (b^2 - (4 \times a \times c))^{1/2}}{(2 \times a)}$$

Symbol	Stack	Expression
(	(= (+ (- (	$x b - b^2$
4	(= (+ (- (	$x b - b^2^4$
*	(= (+ (- (*	$x b - b^2^4$
a	(= (+ (- (*	$x b - b^2^4 a$
*	(= (+ (- (*	$x b - b^2^4 a *$
c	(= (+ (- (*	$x b - b^2^4 a * c$
)	(= (+ (-	$x b - b^2^4 a * c *$
)	(= (+	$x b - b^2^4 a * c * -$
^	(= (+ ^	$x b - b^2^4 a * c * -$
(	(= (+ ^ (	$x b - b^2^4 a * c * -$
	(= (+ ^ (	$x b - b^2^4 a * c * - $

$$x = \frac{(-b + (b^2 - 4ac)^{1/2})}{(2a)}$$

Symbol	Stack	Expression
/	(= (+ ^ (/	$x b - b^2 \wedge 4 a * c * - - 1$
2	(= (+ ^ (/	$x b - b^2 \wedge 4 a * c * - 1 2$
)	(= (+ ^	$x b - b^2 \wedge 4 a * c * - 1 2 /$
)	(=	$x b - b^2 \wedge 4 a * c * - 1 2 / \wedge +$
/	(= /	$x b - b^2 \wedge 4 a * c * - 1 2 / \wedge +$
(	(= / (	$x b - b^2 \wedge 4 a * c * - 1 2 / \wedge +$
2	(= / (	$x b - b^2 \wedge 4 a * c * - 1 2 / \wedge + 2$
*	(= / (*	$x b - b^2 \wedge 4 a * c * - 1 2 / \wedge + 2$
a	(= / (*	$x b - b^2 \wedge 4 a * c * - 1 2 / \wedge + 2 a$
)	(= /	$x b - b^2 \wedge 4 a * c * - 1 2 / \wedge + 2 a *$
)	—	$x b - b^2 \wedge 4 a * c * - 1 2 / \wedge + 2 a * / =$

Ans.

Q: Evaluate the following arithmetic expression Q written in *infix* notation:

Infix :-

$$3 * (5 - 3)$$

(AKTU2023-24)

Postfix →

$$3 * 5 3 -$$

$$3 5 3 - *$$

Evaluation of Postfix Expression

- Postfix expression itself determines the precedence of operators due to the placement of the operator.
- Hence it is easier for the computer to perform the evaluation of a postfix expression.

Symbol	Stack
3	3
5	3 5
3	3 5 3
-	3 2
*	6
	popped Ans

← Top of stack

$$\begin{array}{r} B \quad A \\ 5 - 3 \\ = 2 \end{array}$$

$$\begin{array}{r} B \quad A \\ 3 * 2 \\ = 6 \end{array}$$

[This algorithm finds VALUE of an arithmetic expression P written in *postfix* notation.]

1. Scan P from left to right and repeat step 3 and 4 for each element.
2. If an operand is encountered, put it on STACK
3. If an operator is encountered (except NOT operator) then:
 - a. Remove the two top elements of STACK, where A is top element and B is next -to-top element.
 - b. Evaluate B and A for the encountered operator.
 - c. Place the result of evaluation back on STACK
4. If NOT operator is encountered then:
 - a. Remove top element of STACK, Evaluate for the encountered operator, Place the result of evaluation back on STACK

[End of step 2 loop].

4. Set VALUE equal to the top element on STACK.
5. Exit.

Q: Evaluate the following arithmetic expression Q written in *infix* notation:

$$10 * (8 + 4) - 6 / 3$$

Postfix $\rightarrow$ 10 8 4 + * 2 -

$$10 * (8 + 4) - 6 / 3$$

$$10 * \underline{84+} - 6 / 3$$

$$10 * \underline{84+} - 2$$

$$\underline{10 \ 84+*} - 2$$

$$10 \ 84+*2-$$

Symbol	Stack
10	10
8	10 8
4	10 8 4
+	10 12
*	120
2	120 2
-	<u>118</u> Ans
	popped.

$$\begin{array}{r} B \\ 8 \end{array} \quad \begin{array}{r} A \\ 4 \end{array}$$

$$8 + 4 = \underline{12}$$

$$\begin{array}{r} B \\ 10 \end{array} \quad \begin{array}{r} A \\ 12 \end{array}$$

$$\begin{array}{r} B \\ 120 \end{array} \quad \begin{array}{r} A \\ 2 \end{array}$$

$$120 - 2 = \underline{118}$$

Q: Evaluate the following postfix expression using stack.

$2\ 3\ 9\ * + 2\ 3\ ^ - 6\ 2\ / +$

Ans. 24

show each step and find equivalent prefix form of above expression. Where $^$ is an exponent operator.

(AKTU2020-21)

Symbol	Stack
2	2
3	2 3
9	2 3 9
*	2 27
+	29
2	29 2
3	29 2 3
^	29 8
-	21
6	21 6
2	21 6 2
/	21 3
+	24

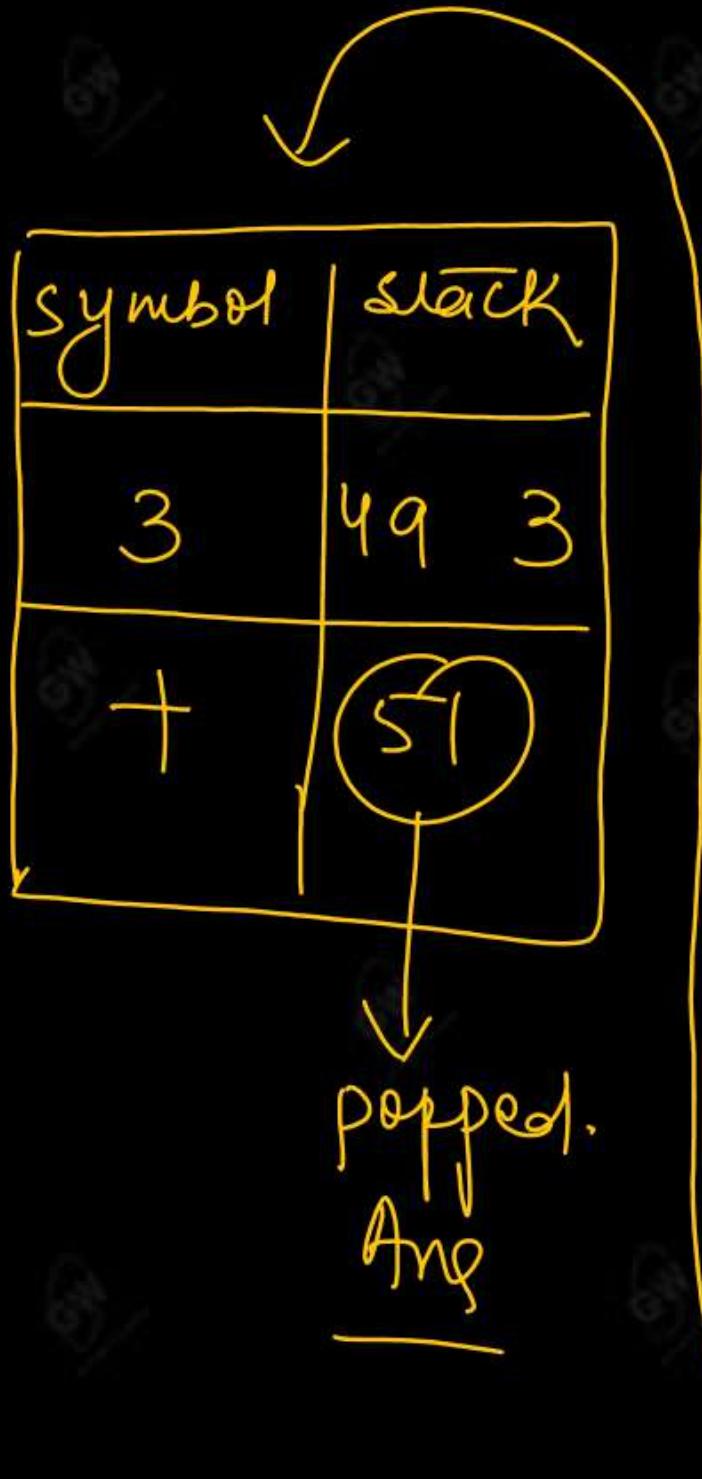
$$\begin{aligned} &\left\{ \begin{array}{l} B \\ 3 \end{array} \right. \begin{array}{l} A \\ 9 \end{array} \\ &\quad 3 * 9 = 27 \\ &\left\{ \begin{array}{l} B \\ 2 \end{array} \right. \begin{array}{l} A \\ 27 \end{array} \Rightarrow 2 + 27 = 29 \\ &\left\{ \begin{array}{l} B \\ 2 \end{array} \right. \begin{array}{l} A \\ 3 \end{array} \Rightarrow 2^1 3 = 8 \end{aligned}$$

24 Ans.

Q: Evaluate the following postfix expression using stack.

$$6\ 2\ 3\ +\ -\ 3\ 8\ 2\ /\ +\ * \ 2^3\ +$$

Ans. 52



Symbol	Stack
6	6
2	6 2
3	6 2 3
+	6 5
-	1
3	1 3
8	1 3 8
2	1 3 8 2
/	1 3 4
+	1 7
*	7
2	7 2
^	49

Data Structure

Unit-2 DPP

1. Write an algorithm to evaluate an arithmetic expression using stack and show how the expression $3*(5-3)$ will be evaluate. **(AKTU2023-24)**
2. Evaluate the following postfix expression using stack. $2\ 3\ 9\ * +\ 2\ 3\ ^ -\ 6\ 2\ / +$, show the contents of each and every steps. also find the equivalent prefix form of above expression. Where $^$ is an exponent operator. **(AKTU2020-21)**

Conv_POLISH (Q, P) Q: Infix arithmetic expression, P: Equivalent prefix expression P

1. Reverse Q (reverse brackets also), Now, Push '(' onto STACK and add ')' to the end of Q.
2. Scan Q from left to right and repeat steps 3 to 6 for each element until the STACK is empty.
3. If an operand is encountered add it to P.
4. If a left parenthesis is encountered, push it onto stack.
5. If an operator is encountered then:
 - a. Repeatedly pop from STACK and add to P each operator (on the top of STACK) which has higher precedence than the operator. *if same precedence then, $L \rightarrow R \implies$ push it onto the stack*
 - b. Add operator to STACK. *$R \rightarrow L \implies$ pop from stack and check with new top of the stack.*
6. If a right parenthesis is encountered then
 - a. Repeatedly pop from STACK and add to P until a left parenthesis is encountered.
 - b. Remove the left parenthesis
7. Exit *finally reverse the expression to get prefix form.*

Q: Trace algorithm to convert infix expression Q into its equivalent prefix expression P

$$A + (B * C)^D$$

$$A + \underbrace{*BC}^D$$

$$\underbrace{A +}^{\wedge} \underbrace{*BCD}$$

$$\boxed{+ A^{\wedge} * B C D}$$

Ans.

Q: Trace algorithm to convert infix expression Q into its equivalent prefix expression P

$$(A + (B * C)^D)$$

$$(D^{(C * B)} + A)$$

Symbol	Stack	Expression
D	(	D
^	(^	D
(	(^C	D
C	(^C	DC
*	(^(C*	DC
B	(^(C*	DCB
)	(^	DCB*
+	(+	DCB*^
A	(+	DCB*^A

Prefix
 $+A^{*}BCD$ Ans

Reverse this expression

DCB*^A+

$DCB*^A+$

Q: Trace algorithm to convert infix expression Q into its equivalent prefix expression P

$$(A + B * C - D + E / (F + G))$$

$$(A + B * C - D + E / \underbrace{+FG})$$

$$(A + \underbrace{*BC} - D + E / \underbrace{+FG})$$

$$(A + \underbrace{*BC} - D + \underbrace{/E+FG})$$

$$(\underbrace{+A*BC} - D + \underbrace{/E+FG})$$

$$(\underbrace{-+A*BCD} + \underbrace{/E+FG})$$

$$+-+A*BCD/E+FG$$



Q: Trace algorithm to convert infix expression Q into its equivalent prefix expression P

$$(A + B * C - D + E / (F + G))$$

$$\underline{((G + F) / E + D - C * B + A)}$$

Symbol	Stack	Expression
(	((	
G	((	G
+	((+	G
F	((+	G F
)	(	G F +
/	(/	G F +
E	(/	G F + E
+	(+	G F + E /
D	(+	G F + E / D

Q: Trace algorithm to convert infix expression Q into its equivalent prefix expression P

$$(A + B * C - D + E / (F + G))$$

$$\underline{((G + F) / E + D - C * B + A)}$$

Symbol	Stack	Expression
-	C + -	G F + E / D
(	(+ -	G F + E / D (
*	(+ - *	G F + E / D (
B	(+ - *	G F + E / D (B
+	(+ - +	G F + E / D (B *
A	(+ - +	G F + E / D (B * A
)	—	G F + E / D (B * A + - +
		↓ Reverse it
		+ - + A * B C D / E + F G

[This algorithm finds VALUE of an arithmetic expression P written in *prefix* notation. Here the expression will read from right to left]

1. Read the next element.
 2. If element is operand, then, Push the element in the stack
 3. If element is operator, then
 - a. Pop two operands from the stack, *where A is top element and B is next –to-top element.*
 - b. Evaluate A and B for the encountered operator.
 - c. Push the results of the expression in the stack
 4. If no more elements, then Pop the result
- Else
- Go to step 1.

Q: Evaluate the following arithmetic expression written in *prefix* notation:

$$+ 2 * 3 + 4 5$$

← Scan from Right to Left

Symbol	Stack
5	5
4	5 4
+	9
3	9 3
*	27
2	27 2
+	(29) popped
	<u>Ans</u>

B op A

$$\begin{array}{cc} B & A \\ \{ & \\ 5 & 4 \\ \} & \end{array}$$

$$A + B = 5 + 4 = 9$$

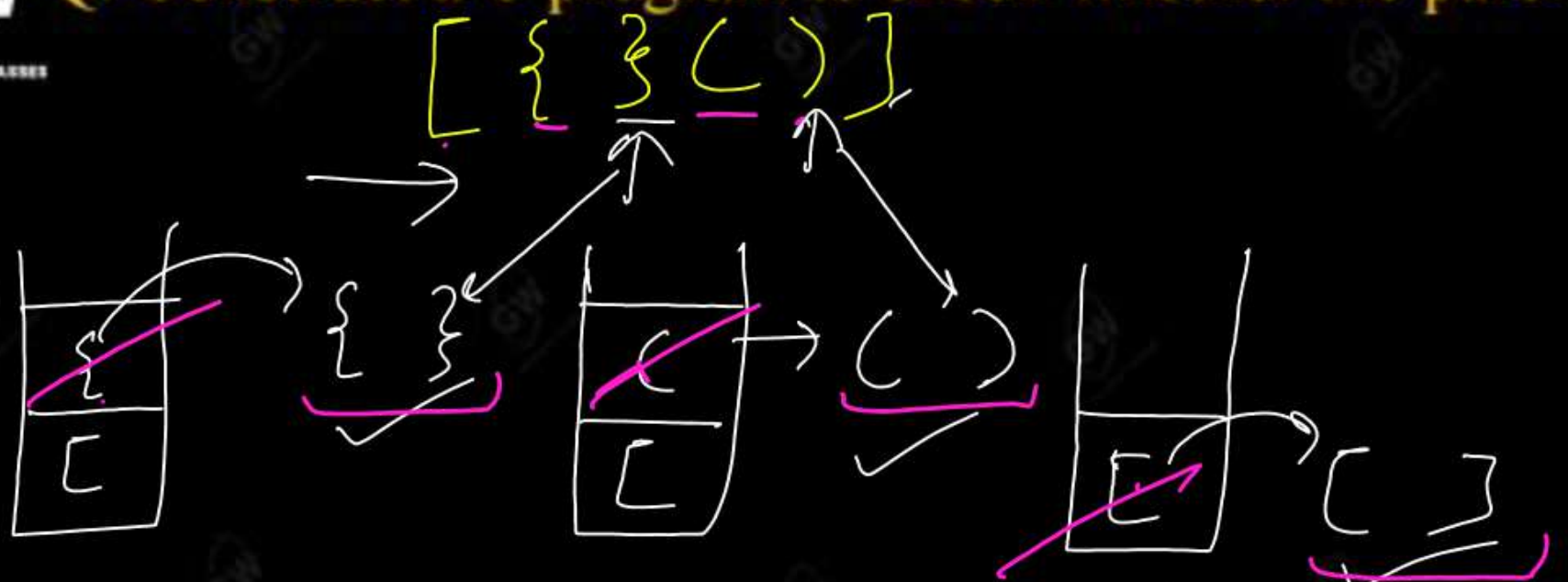
$$\begin{array}{cc} B & A \\ 9 & 3 \\ \Rightarrow & 3 * 9 = 27 \end{array}$$

1. Used for reversal of a given list. We can accomplish this task by pushing each element onto the stack as it is read. When the line is finished, elements are then popped off the stack, so they come off in reverse order.
2. For the evaluation of arithmetic expressions. It accomplishes by converting first the given expression in reverse polish notation and then evaluates the expression.
3. For the zero address instruction implementation in computer organization.
4. Used for the address holding in function calling procedure of programming.
5. Used to implement recursive procedures.
6. To check whether the expression has balanced parentheses or not.

Handwritten diagram illustrating balanced parentheses: $[(\{) \}]$. The opening and closing brackets are matched with lines, and the entire expression is enclosed in a large bracket. There are 'X' marks above and below the diagram, indicating a check or correction.

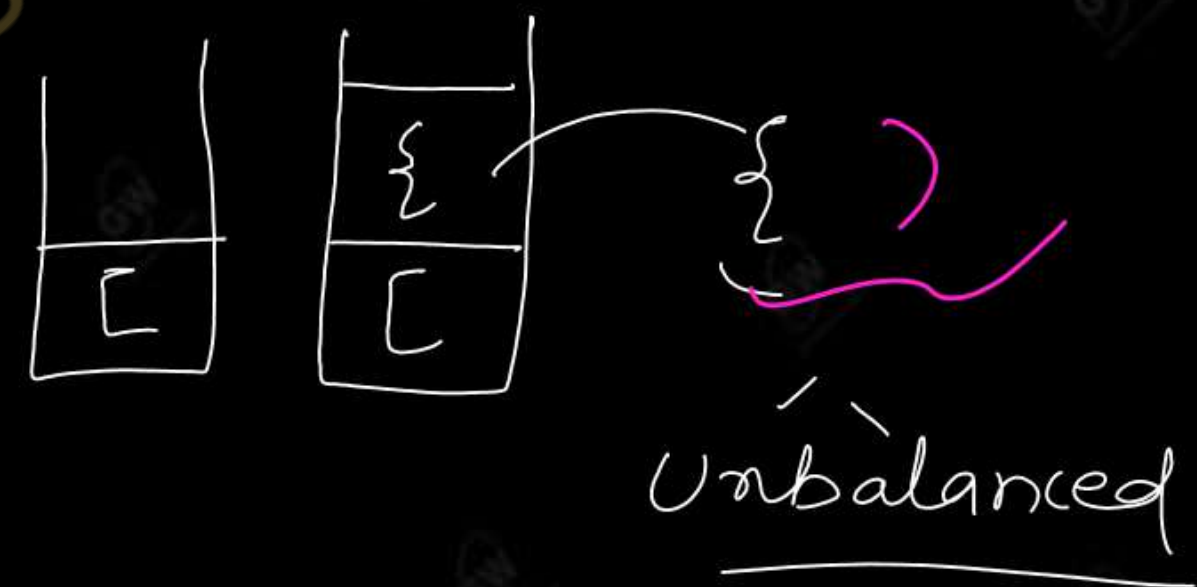
Q. Construct a C program to check whether the parentheses are balanced in an expression.

(AKTU2024-25)



Balanced

$[\{ \}]$



Unbalanced

Logic

1) $\{ ' (' [\Rightarrow$ push on stack

2) $\{ ') '] \Rightarrow$ pop from stack and match both parentheses
 a) If matching left parenthesis is popped —
 b) Unbalanced.

C program to check whether the parentheses are balanced in an expression

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX 100
char stack[MAX];
int top = -1; ✓

void push(char data)
{
    if (top >= MAX - 1)
    {
        printf("Stack overflow\n");
        return;
    }
    top++;
    stack[top] = data;
}
```

```
char pop()
{
    char data;
    if (top == -1)
    {
        printf("Empty stack\n");
        return;
    }
    data = stack[top];
    top--;
    return data;
}
```

// Function to check if two characters form a matching pair

int is_matching_pair(char ch1, char ch2)

{

if (ch1 == '(' && ch2 == ')')

return 1;

else if (ch1 == '{' && ch2 == '}')

return 1;

else if (ch1 == '[' && ch2 == ']')

return 1;

else

return 0;

}

parenthesis popped from stack

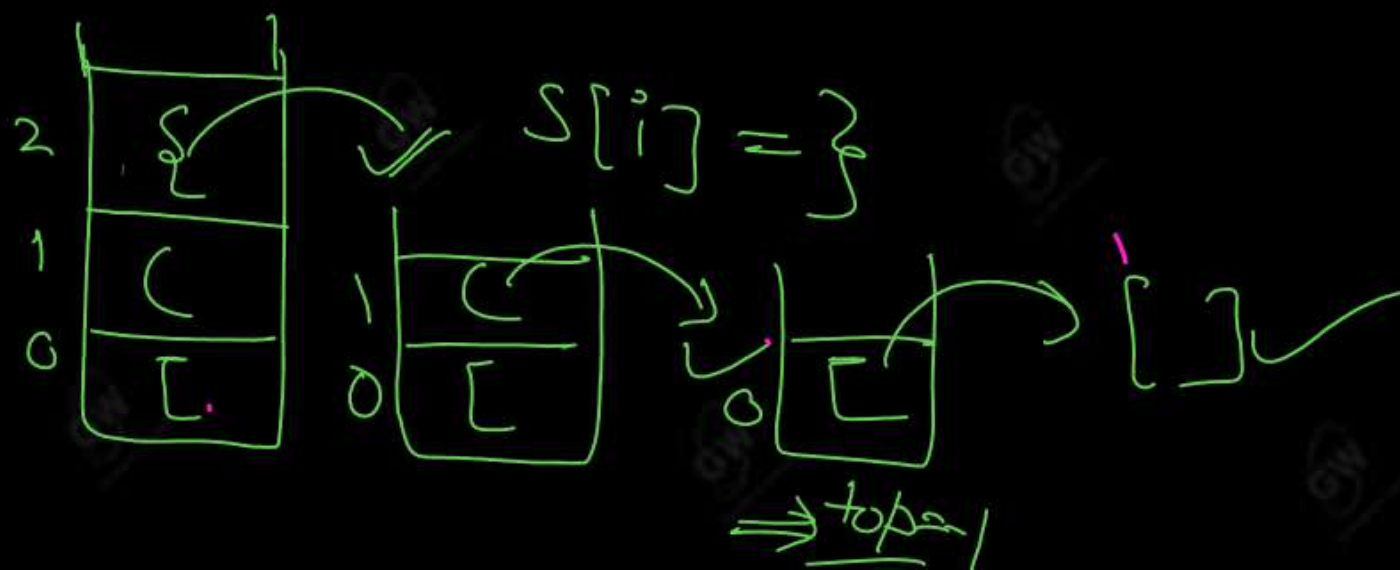
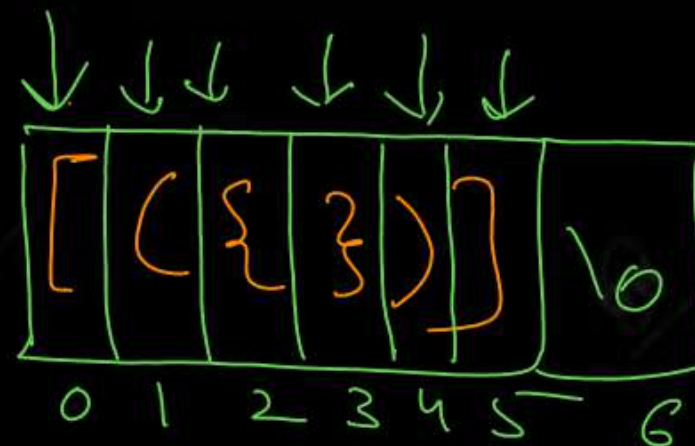
parenthesis newly encountered

returns 1
successfully
finding
matching
parenthesis
otherwise
returns 0

// Function to check if expression is balanced

```
int isBalanced(char *s)
{
    int i;
    for (i = 0; i < strlen(s); i++)
    {
        if (s[i] == '(' || s[i] == '{' || s[i] == '[')
        {
            push(s[i]);
        }
        else if (s[i] == ')' || s[i] == '}' || s[i] == ']')
        {
            if (top == -1) // stack empty
            {
                return 0; // if stack becomes empty
            }
            else if (!is_matching_pair(pop(), s[i]))
            {
                return 0;
            }
        }
    }
}
```

```
if (top == -1)
    return 1;
else
    return 0;
```



```
int main()
{
    char s[MAX];
    printf("Enter the expression: ");
    scanf("%s", s); ✓

    if (isBalanced(s))
        printf("The expression is balanced\n");
    else
        printf("The expression is not balanced\n");

    return 0;
}
```

s ⇒

[({ })] \0

strlen(s) = 6

Data Structure

Unit-2 DPP-5

1. Give some applications of stack. **(AKTU2019-20)**
2. Construct a C program to check whether the parentheses are balanced in an expression.
(AKTU2024-25)

Gateway Classes



AKTU

B.Tech III-Sem



GATEWAY CLASSES

CS IT & CS Allied

DATA STRUCTURE

Unit-2 Lecture-6

Today's Target

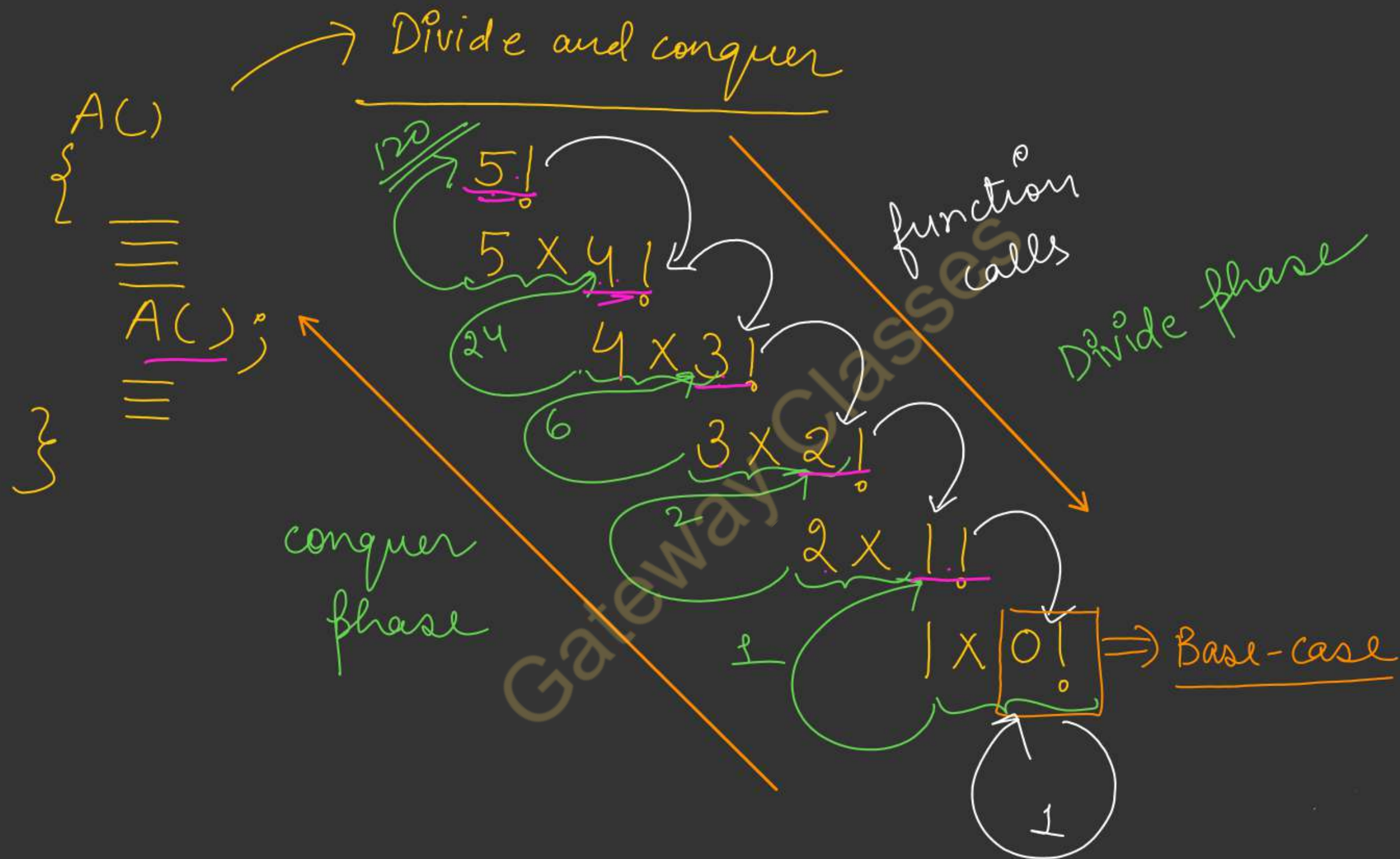
- Introduction to Recursion ✓
- Principles of Recursion ✓
- Programming examples of recursion ✓
- AKTU PYQs
- DPP



By Dr. Nidhi Parashar Ma'am

Ph.D (CSE)

- **M.Tech Gold Medalist**
- **Net Qualified**



What is Recursion?

- Recursion is a powerful technique used in programming, where a function calls itself during its execution.

Idea

Principles of Recursion

- We use recursion to solve bigger problem into smaller sub-problems. Recursion is based on concept of divide and conquer. It involves solving a problem by breaking it into smaller instances of the same problem. This is done by eventually reaching a base case where a straightforward solution is available.
- One thing we have to keep in mind, that if each sub-problem is following same kind of patterns, then only we can use the recursive approach.
- Each recursive call works on a smaller subproblem until the base case is reached, and the results are combined to solve the original problem.

Q: Write a C program to calculate factorial of number using recursive and non-recursive functions.

(Non-Recursive)
(Iterative)

(AKTU2021-22, 2018-19)

Sol.

→ $1 \times 2 \times 3 \times 4 \times 5$

```
#include <stdio.h>

int factorial (int n)
{
    int i, fact = 1;
    for (i = 1; i <= n; i++)
    {
        fact = fact * i;
    }
    return (fact);
}
```

funⁿ defiⁿ

```
int main ()
{
    int n;
    printf ("Enter the number");
    scanf ("%d", &n);
    if (n < 0)
        printf ("Invalid");
    else
        printf ("factorial is = %d", factorial(n));
    return 0;
}
```

funⁿ call

Q: Write a C program to calculate factorial of number using recursive and non-recursive functions.

(Recursive)

(AKTU2021-22,2018-19)

Recursive

```
#include <stdio.h>

int factorial (int n)
{
    if (n == 0) // Base case
        return 1;
    else
        return (n * factorial(n-1));
    // Recursive case
}
```

```
int main ()
{
    int n;
    printf ("Enter the number");
    scanf ("%d", &n);
    if (n < 0)
        printf ("Invalid");
    else
        printf ("factorial is = %d", factorial(n));
    return 0;
}
```

funⁿ call

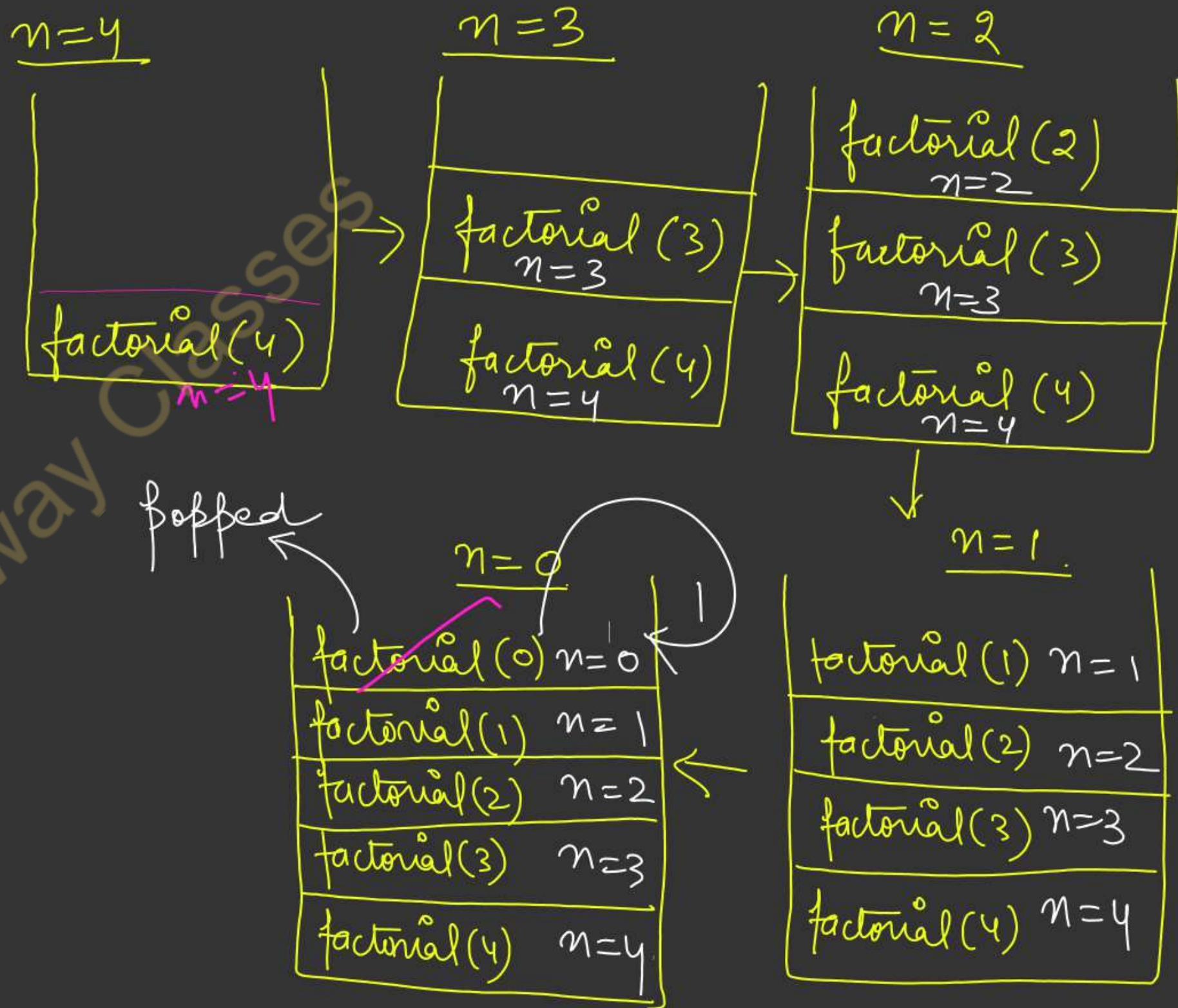
$n=4$ ↗

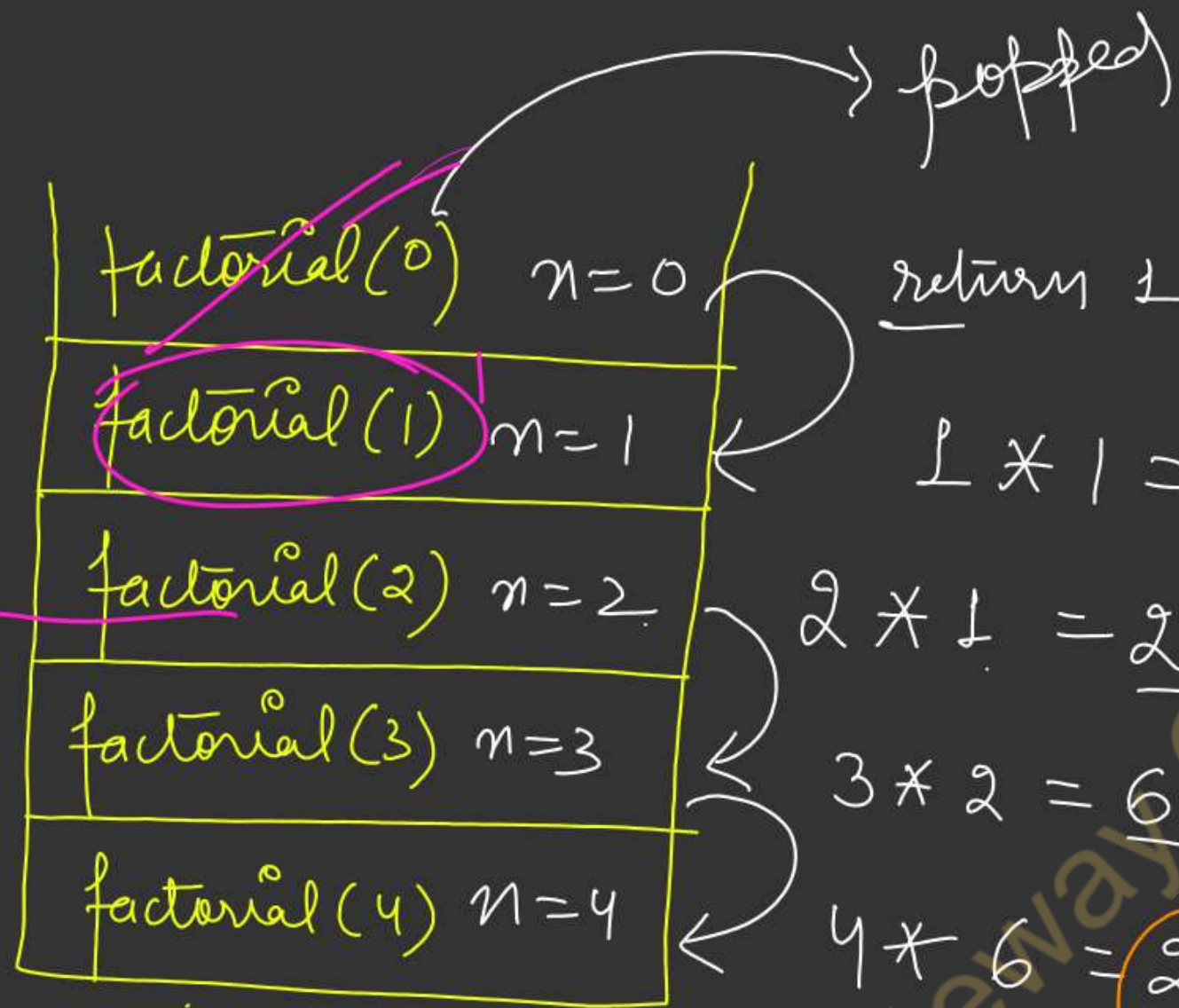
```

int factorial(int n)
{
    if (n == 0)
        return 1;
    else
        return (n * factorial(n-1));
}
  
```

For every funⁿ call

- 1) local var
- 2) Return addresses
- 3) funⁿ call





return 1.

$$1 \times 1 = 1 \Rightarrow 1 \times \text{factorial}(0)$$

$$2 \times 1 = 2 \Rightarrow 2 \times \text{factorial}(1)$$

$$3 \times 2 = 6 \Rightarrow 3 \times \text{factorial}(2)$$

$$4 \times 6 = 24 \Rightarrow 4 \times \text{factorial}(3)$$

returned to main()

factorial(2)

$$2 \times \text{factorial}(1)$$

The data structure used to perform **recursion** is a **stack**.

Why a stack?

- Due to its LIFO (Last In First Out) property it remembers its 'caller' so knows whom to return when the function has to return.
- Recursion makes use of system stack for storing the return addresses of the function calls.
- When a function is called, important info about that function (parameters, local variables, and return address — i.e., where to continue execution after the function finishes) is stored in a structure called the call stack.
- Each recursive call adds ("pushes") a new **stack frame** onto the call stack.
- When a function finishes, its stack frame is popped from call stack, and control returns to the previous function call.
- Every recursive function has its equivalent iterative (non-recursive) function.

Recursion follows a set of fundamental principles that help in solving complex problems by breaking them into smaller subproblems. These principles ensure that recursive functions work correctly and efficiently.

1. Base Case (Stopping Condition)

- Every recursive function must have a **base case**, which defines when the recursion should stop. Without a base case, the function will keep calling itself indefinitely, leading to a **stack overflow error**.

2. Recursive Case (Problem Reduction):

- In recursion, the function should **call itself** with a **reduced version** of the problem. This is known as the **recursive case**. The recursive case must be defined properly so that it reduces the complexity of the problem in each step.

3. Progress Toward the Base Case (Convergence Property):

- Each recursive call should **move closer to the base case**. If there is no progress toward the base case, the recursion will **never terminate**.
- To ensure this, the function should **reduce** the problem size in each recursive call. If the reduction is not happening, the function might get stuck in an infinite loop.

4. Stack Memory Usage (Call Stack):

- Each recursive call is **pushed onto the call stack**, and execution resumes when a function **returns** its result. Too many recursive calls may **overflow** the stack. After reaching the base case, the function starts **returning values in reverse order**.

5. Correctness by Mathematical Induction:

1. **Base Case Validity**: If the base case is correct, it ensures correctness for small inputs.
2. **Inductive Step**: If recursion works for $n-1$, it must work for n .

Valid / Invalid recursion

```
int fact (int n)
```

```
{
    return (n * fact (n - 1));
}
```

There is no base case

Invalid

```
int fact (int n)
```

```
{
    if (n == 0) // Base case
        return 1;
    else
        return (n * fact (n + 1));
}
```

Invalid

*Never leads
to Base case*

```
int fact (int n)
```

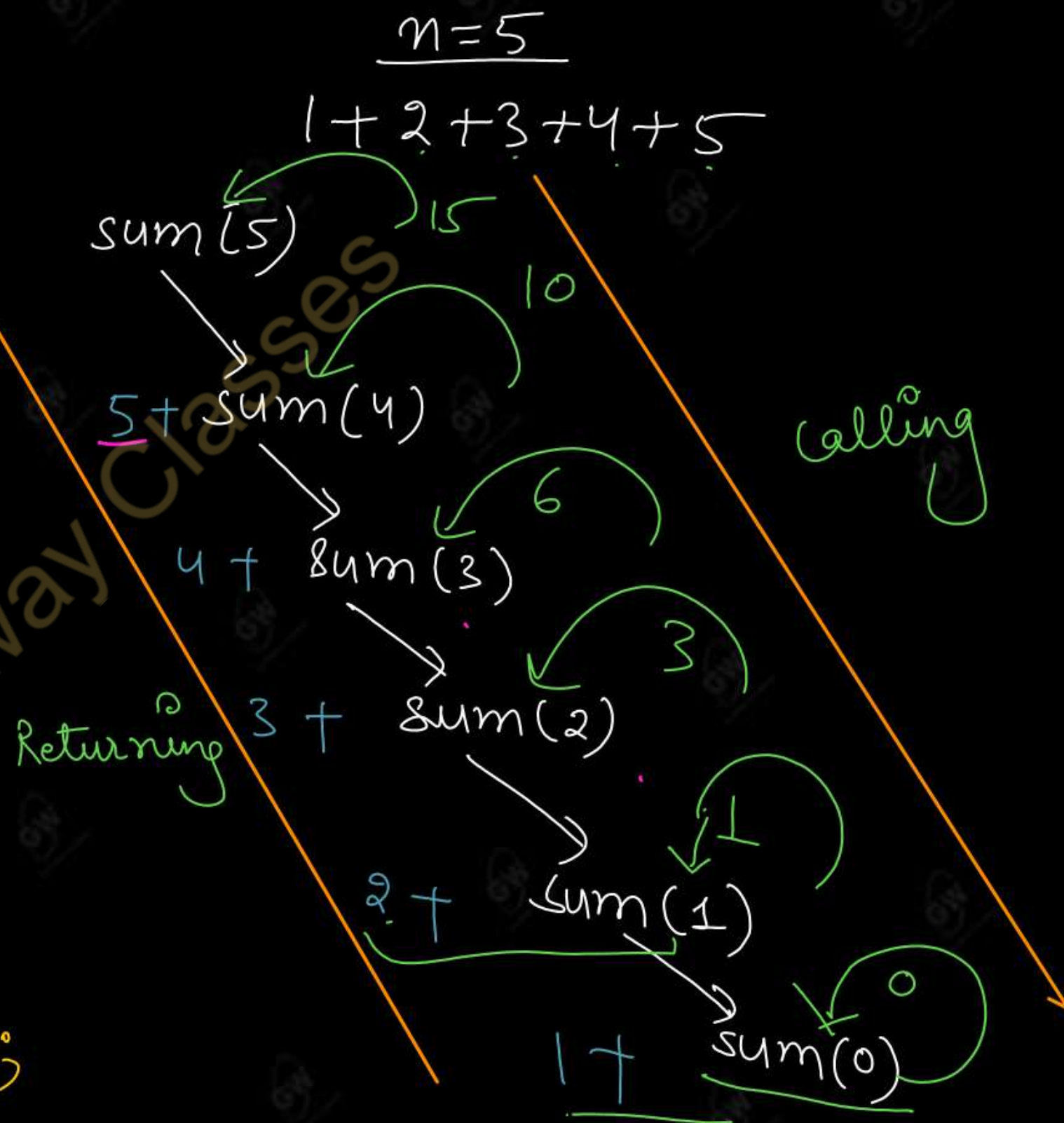
```
{
    if (n == 0)
        return 1;
    else
        return (n * fact (n - 1));
}
```

Valid

Q: Write recursive program to find sum of first N natural numbers.

```
#include <stdio.h>
int sum(int n)
{
    if (n == 0)
        return (n);
    else
        return (n + sum(n-1));
}

int main
{
    int n;
    printf("Enter how many natural numbers to print");
    scanf("%d", &n);
    printf("The sum of %d natural numbers is = %d", n, sum(n));
    return 0;
}
```



Q: Write recursive program to find sum of digits of the given number. Also calculate the time complexity.

(AKTU2017-18)

Iterative
logic

$$n = 325$$

$$s = 3 + 2 + 5$$

$$s = 0$$

$$n = 325$$

$$d = 325 \% 10 = 5$$

$$s = 0 + 5 = 5$$

$$n = 325 / 10 = 32$$

$$s = s + n \% 10$$

$$n = n / 10$$

$$n = 32$$

$$d = 32 \% 10 = 2$$

$$s = 5 + 2$$

$$n = 32 / 10 = 3$$

$$n = 3$$

$$d = 3 \% 10 = 3$$

$$s = 5 + 2 + 3$$

$$n = 3 / 10 = 0$$

$$n = 0$$

X

while ($n \neq 0$)

$$d = n \% 10;$$

$$s = s + d;$$

$$n = n / 10;$$

Q: Write recursive program to find sum of digits of the given number. Also calculate the time complexity.

Time Complexity
=
 $O(\log n)$

(AKTU2017-18)

```
#include <stdio.h>
```

```
int sod(int n)
```

```
{  
    if (n == 0)  
        return 0;
```

```
    else
```

```
        return (n % 10 + sod(n / 10));  
}
```

```
int main()  
{
```

```
    int n;
```

```
    printf("Enter number");
```

```
    scanf("%d", &n);
```

```
    printf("The sum of digits of %d is = %d",  
           n, sod(n));  
}
```

$n = 325$
 $\text{sod}(325)$

$5 + \text{sod}(32)$

$2 + \text{sod}(3)$

$3 + \text{sod}(0)$

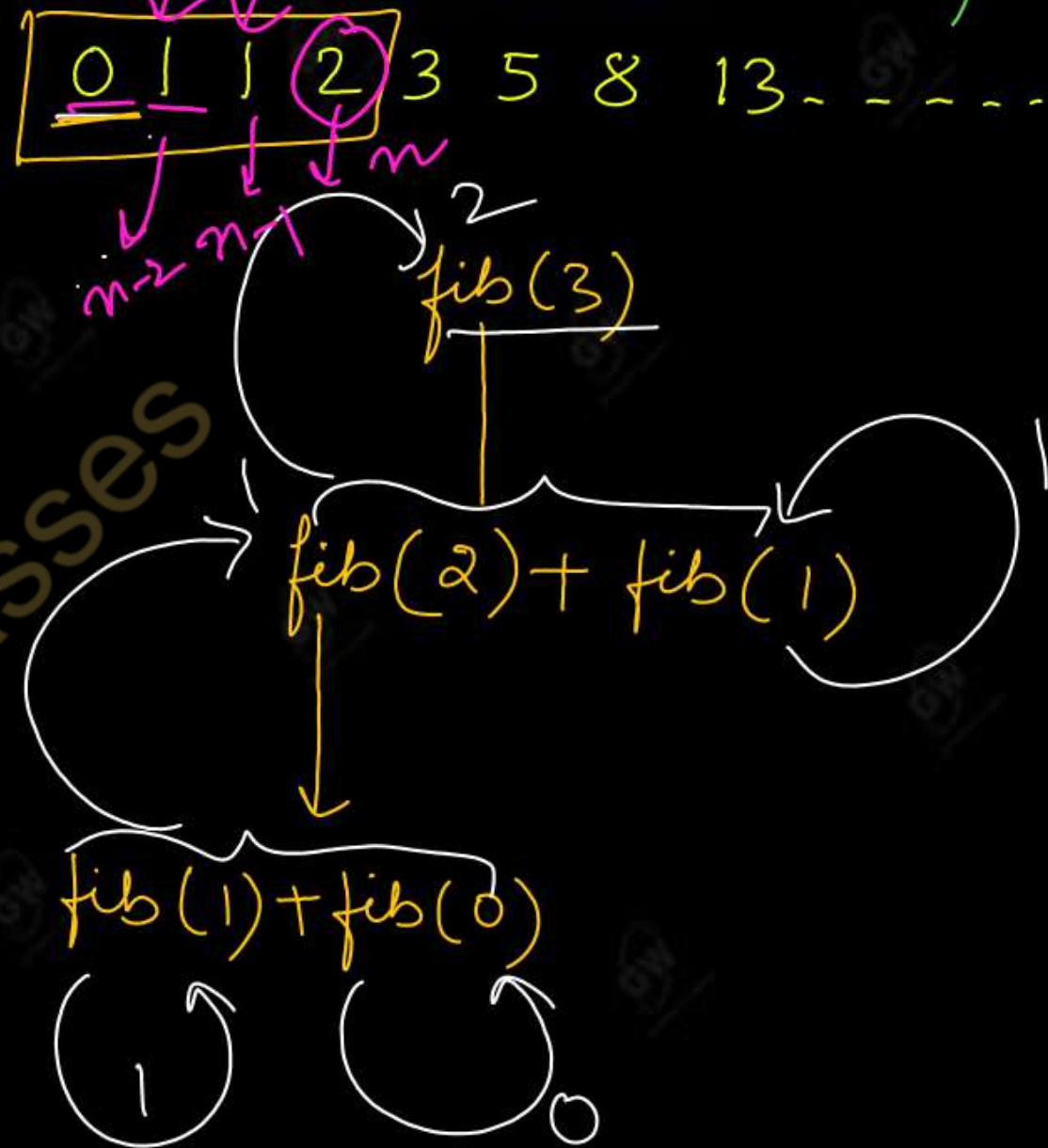
function
calling

Returning

Q: Write recursive program to print first N numbers of Fibonacci series. (AKTU)

```
#include <stdio.h>
int fib(int n)
{
    if (n == 0 || n == 1)
        return n;
    else
        return (fib(n-1) + fib(n-2));
}
```

```
int main()
{
    int n, i;
    printf("Enter number of terms to print");
    scanf("%d", &n);
    for (i = 0; i < n; i++)
    {
        printf("%d ", fib(i));
    }
    return 0;
}
```



0 1 1

Q. Differentiate between iteration and recursion. (AKTU 2019-20)

	Recursion	Iteration
Definition	Calling a function again and again	Calling ^{executing} a specific block of code again and again
Format	A function <u>calls itself</u> unless a base case is achieved	A block is called as long as a condition is true
Speed	slow due to involvement of stack	Faster compared to recursion as stack not involved
Termination	Recursion terminates when base condition is achieved	Iteration stops when conditional statement turns false
Infinite loop	Infinite recursion will crash the complete system, because stack will be full after some time	Infinite iteration will only execute the statement infinite time, but system will not crash
Size of code	Size of code is reduced	Size of code is larger
Usability	Recursion can only be used with functions, by calling a function inside itself.	Iteration can only be implemented by using loops only.

Data Structure

Unit-2 DPP-6

1. What is Recursion? **(AKTU2021-22)**
2. Write a C program to calculate factorial of number using recursive and non-recursive functions. **(AKTU2021-22)**
3. Write a program in C to compute the factorial of given number recursively. **(AKTU2018-19)**
4. Differentiate between iteration and recursion. **(AKTU2019-20)**
5. What is recursion? Write a recursive program to find sum of digits of the given number. Also calculate the time complexity. **(AKTU2017-18)**



AKTU

B.Tech III-Sem



GATEWAY CLASSES

CS IT & CS Allied

DATA STRUCTURE

Unit-2 Lecture-7

Today's Target

- Type of recursion(Direct, Indirect, Head, Tail recursion)
- AKTU PYQs
- DPP



By Dr. Nidhi Parashar Ma'am

Ph.D (CSE)

- M.Tech Gold Medalist
- Net Qualified

Types of Recursion

Recursive functions can be classified on the basis of

- a) Based on functions call itself – **Direct / Indirect**
- b) Based on pending operation at each recursive call – **Tail Recursive/ Head Recursive**
- c) Based on the structure of the function calling pattern – **Linear / Tree**

Direct Recursion

- If a function calls itself directly from within itself is known as Direct Recursion.
- When the method invokes itself, it is direct.
- Example: Here, function 'func' is calling itself.

```
int func(int num)
```

```
{
```

```
    if (num == 0)
```

```
        return 1;
```

```
    else
```

```
        return func(num - 1);
```

```
}
```

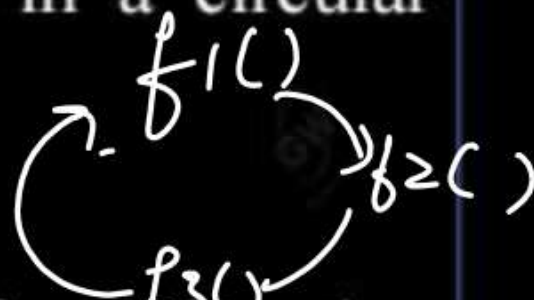
Base case

Recursive case

```
A ()
{
    |||
    |||
    |||
    = A();
}
```

Indirect Recursion

- In Indirect Recursion, more than one function call one another mutually in a circular manner.

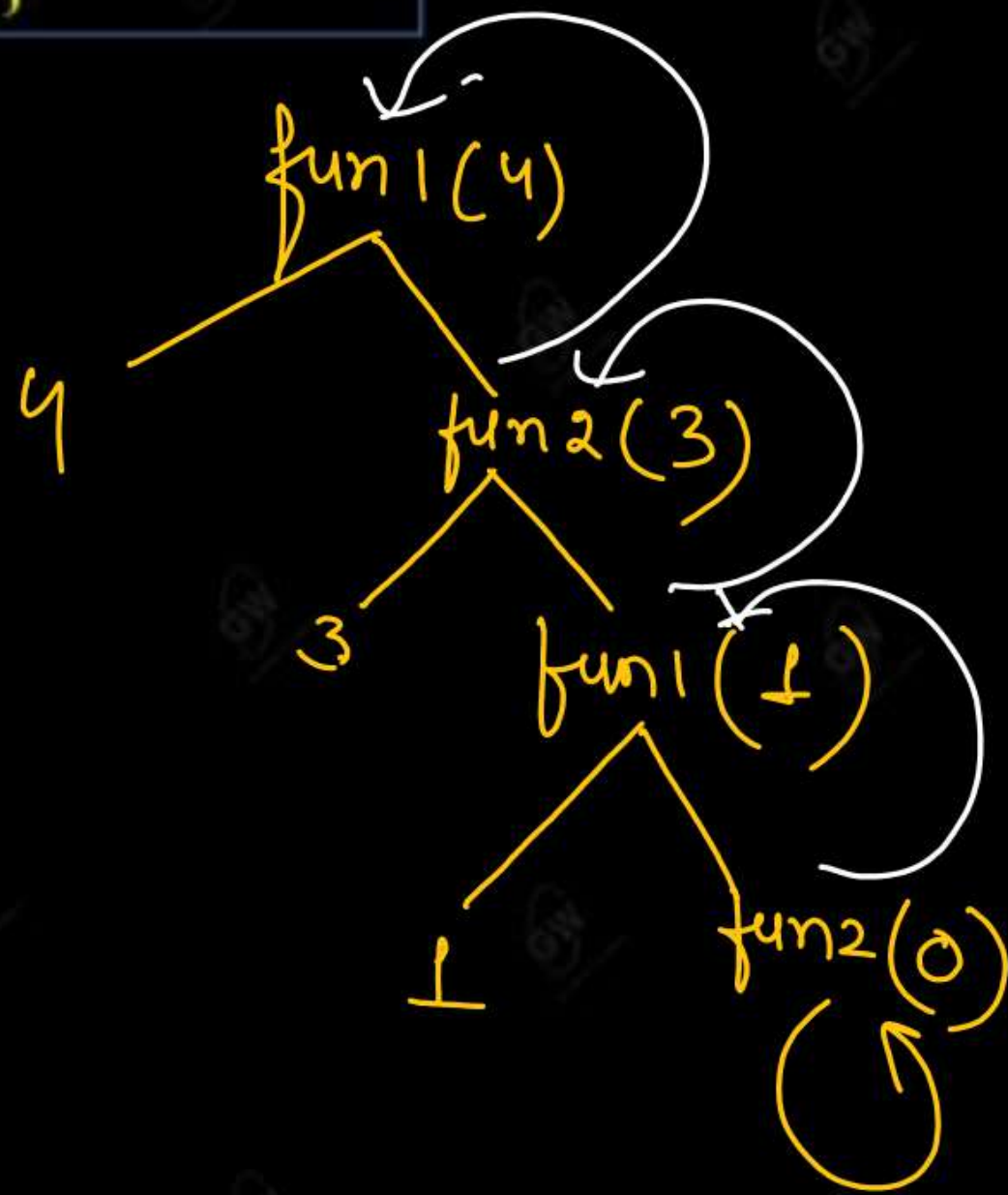


- For example, if a function 'fun1()' calls function 'fun2()', which calls function 'fun3()' which again calls 'fun1()' is called indirect recursion.

```
#include <stdio.h>
void fun2(int n);
void fun1(int n)
{
    if (n > 0)
    {
        printf("%d ", n);
        fun2(n - 1);
    }
}

void fun2(int n)
{
    if (n > 1)
    {
        printf("%d ", n);
        fun1(n - 2);
    }
}
```

```
int main()
{
    fun1(4);
    return 0;
}
```



Head/Top Recursion

- If a recursive function is calling itself and that recursive call is the first statement in the function then it's known as Head Recursion.
- Nothing will perform before calling of a recursive function.

Time Complexity For Head Recursion: $O(n)$

Space Complexity For Head Recursion: $O(n)$

```
#include <stdio.h>
```

```
void fun(int n)
```

```
{
```

```
    if (n > 0)
```

```
    {
```

```
        fun(n - 1);
```

// Recursive call

```
        printf("%d ", n);
```

```
    }
```

```
}
```

```
int main()
```

```
{
```

```
    fun(5);
```

```
    return 0;
```

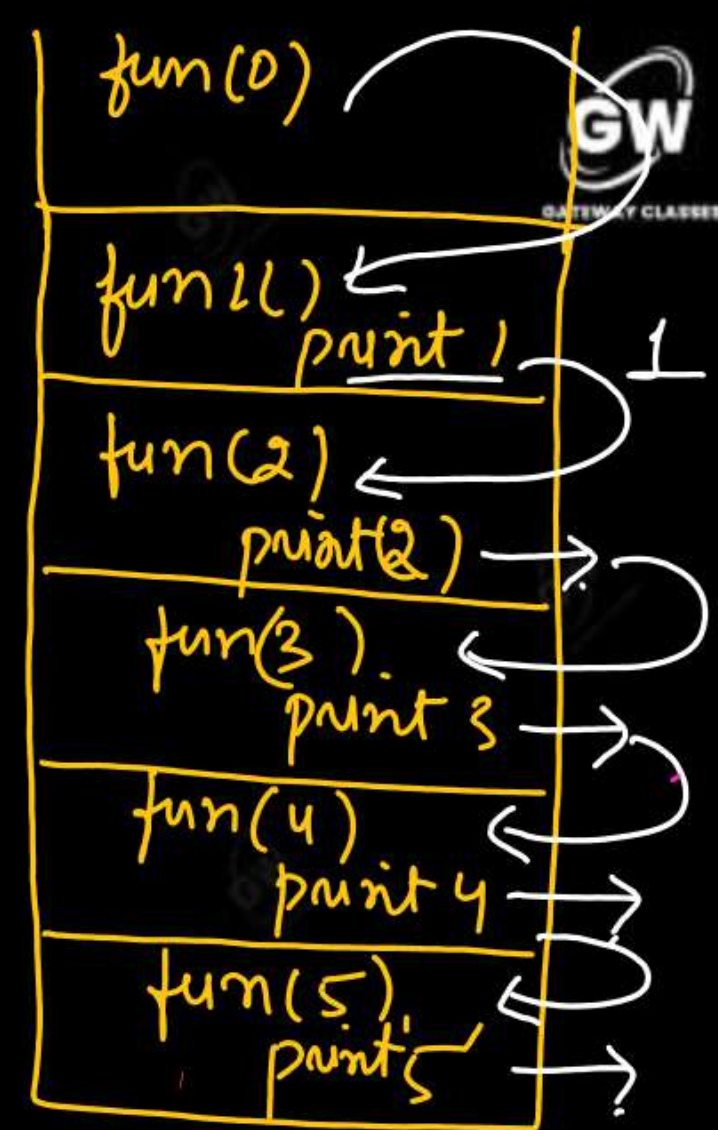
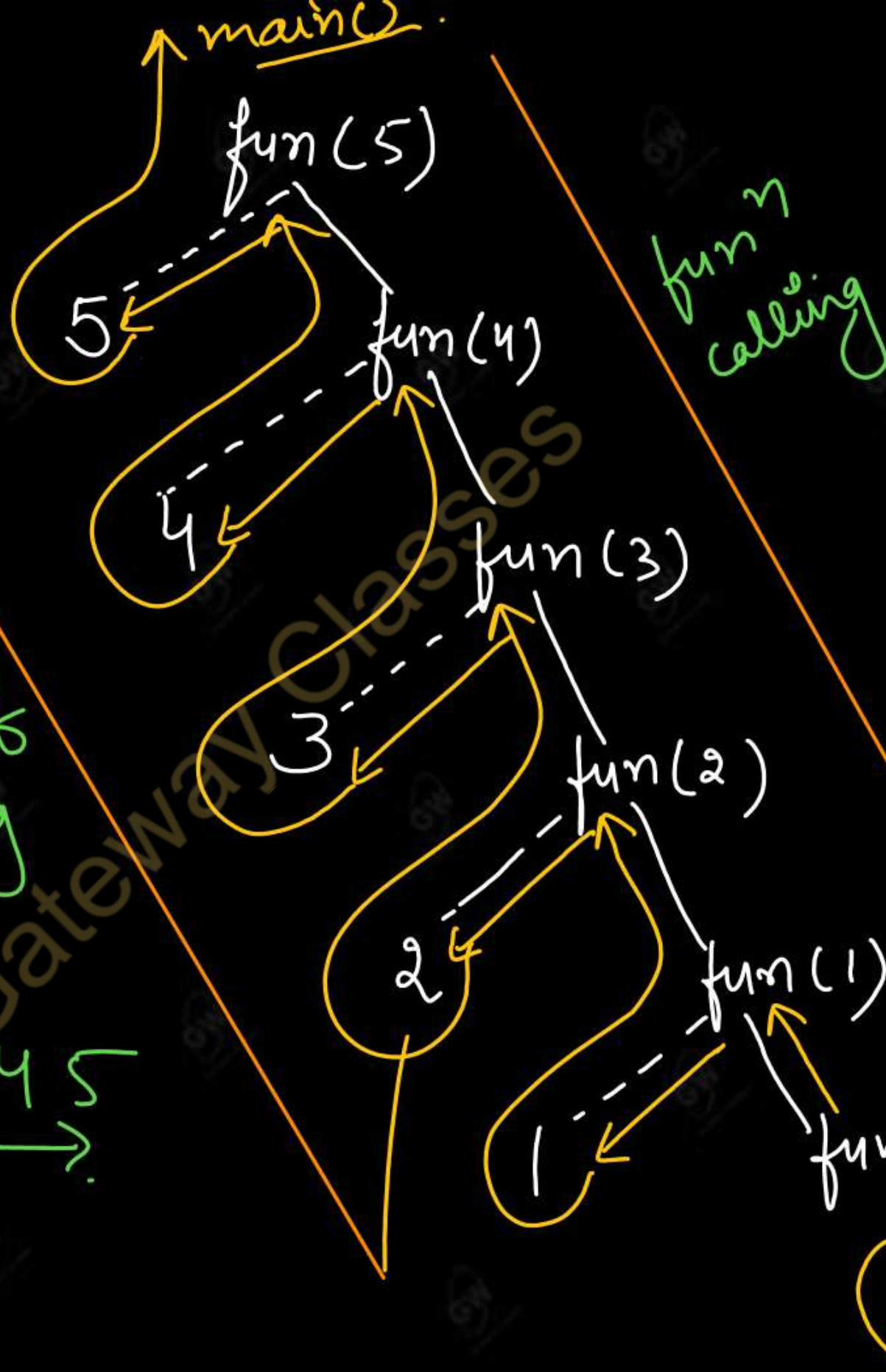
```
}
```

```
#include <stdio.h>
void fun(int n)
{
    if (n > 0)
    {
        fun(n - 1);
        printf("%d ", n);
    }
}

int main()
{
    fun(5);
    return 0;
}
```

order of
printing

1 2 3 4 5



Space Complexity
 $\Rightarrow O(n)$

Tail Recursion

(Imp)

- Tail Recursion is an example of Direct Recursion.
- If a recursive function is calling itself and that recursive call is the last statement in the function then it's known as Tail Recursion.
- In other words, no operations are pending when the recursive function returns to its caller.
- It is an efficient method as compared to others, as the stack space required is less and even compute overhead will get reduced.

Time Complexity For Tail Recursion: $O(n)$

Space Complexity For Tail Recursion: $O(n)$

```
#include <stdio.h>
```

```
void fun(int n)
```

```
{
```

```
    if (n > 0)
```

```
    {
```

```
        printf("%d ", n);
```

```
        fun(n - 1); // Last statement in function
```

```
    }
```

```
}
```

```
int main()
```

```
{
```

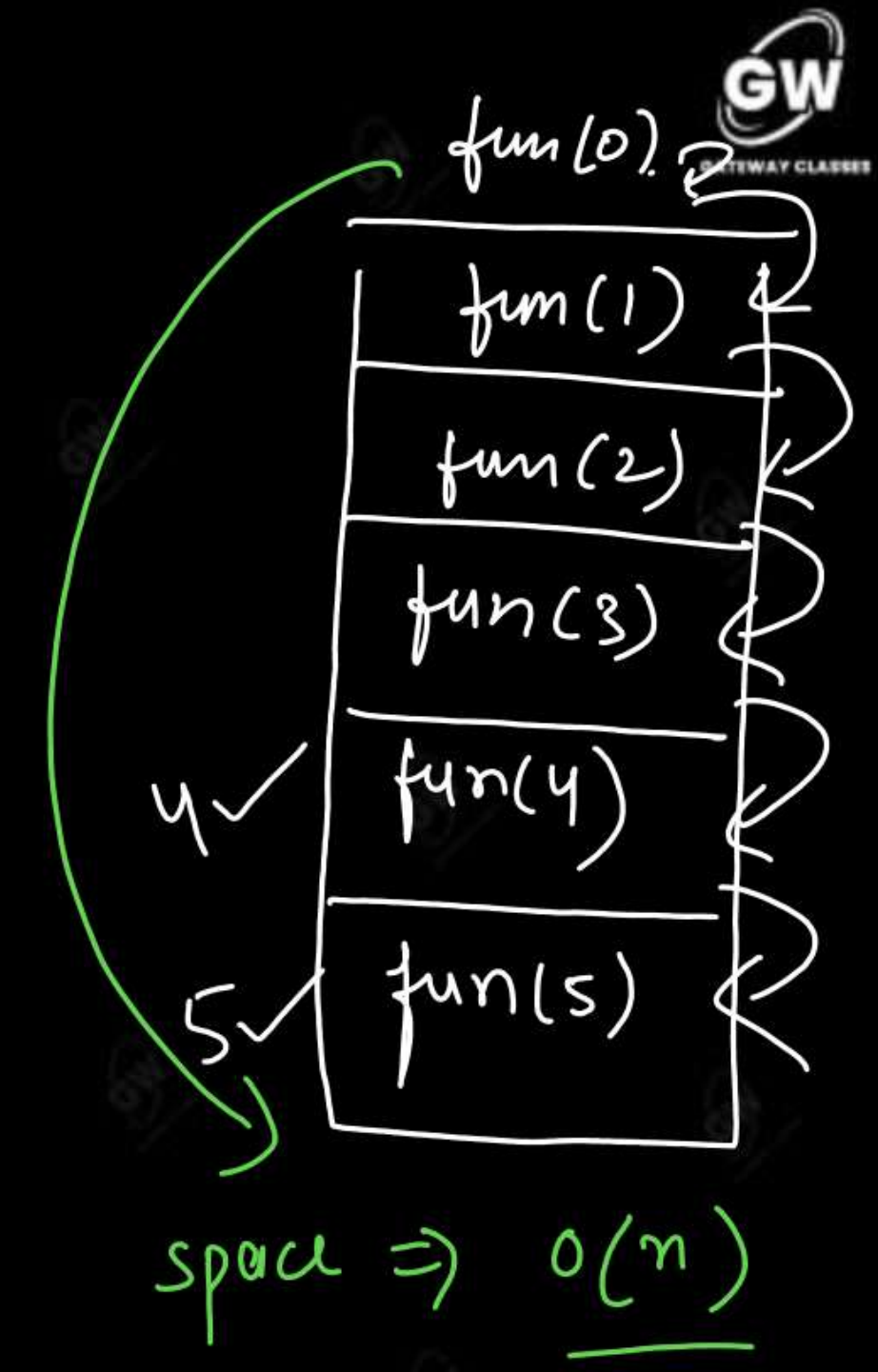
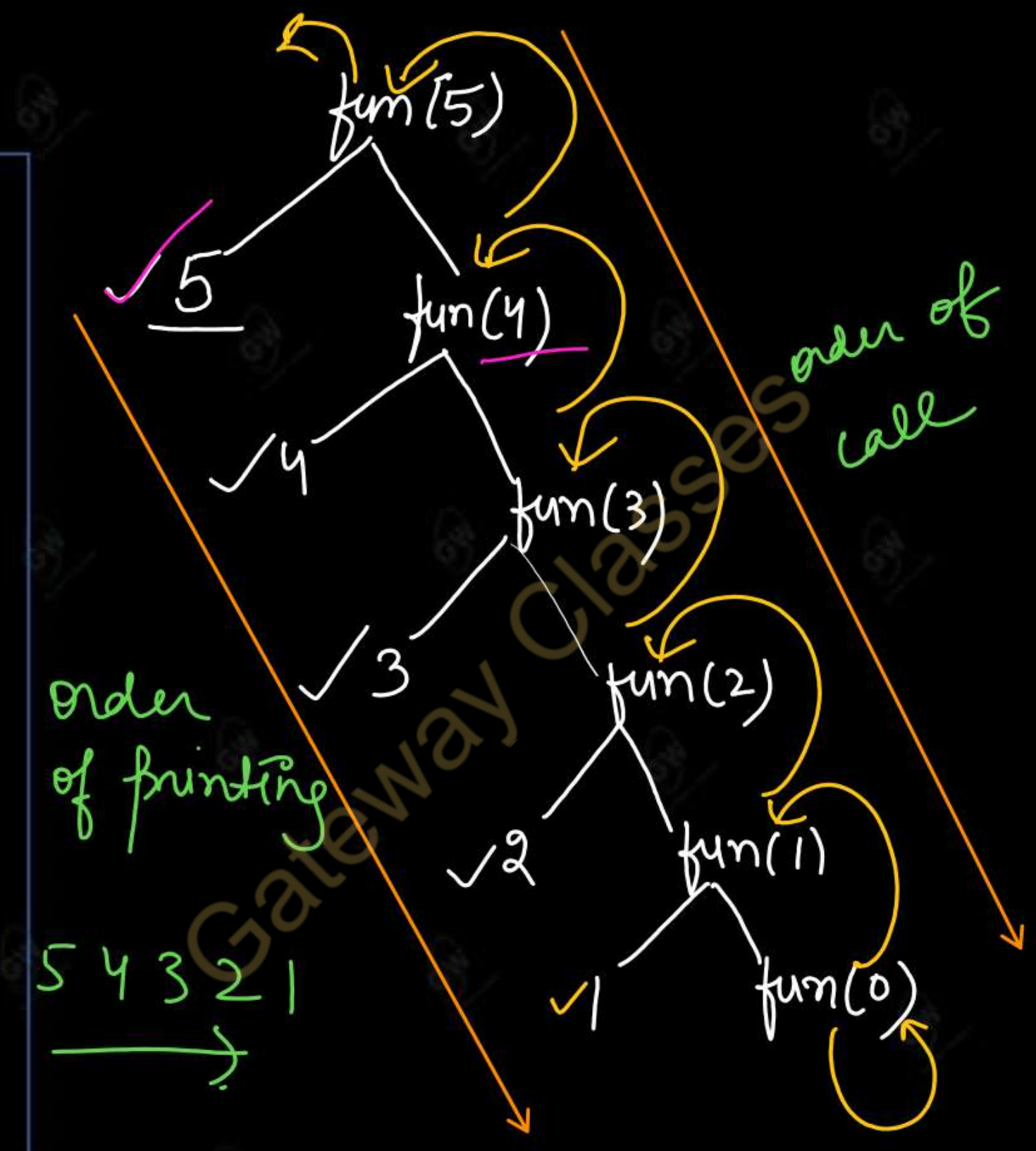
```
    fun(5);
```

```
    return 0;
```

```
}
```

```
#include <stdio.h>
void fun(int n)
{
    if (n > 0)
    {
        printf("%d ", n);
        fun(n - 1);
    }
}

int main()
{
    fun(5);
    return 0;
}
```



Is factorial Tail Recursive?

NO

```
int factorial(int n)
{
    if (n == 0)
        return 1;
    else
        return (n * factorial(n - 1));
}
```

↓

```
{ x = factorial(n - 1);
  return (n * x); }
```

Is following function Tail Recursive?

```
#include <stdio.h>
void fun(int n)
{
    if (n > 0)
    {
        fun(n - 1);
        printf("%d ", n);
        fun(n - 2);
    }
}
```

NO

Tail Recursion can be easily converted into iterative solution

```
#include <stdio.h>
void fun(int n)
{
    if (n > 0)
    {
        printf("%d ", n);
        fun(n - 1);
    }
}

int main()
{
    fun(5);
    return 0;
}
```



```
#include <stdio.h>
void fun(int n)
{
    while (n > 0)
    {
        printf("%d ", n);
        n--;
    }
}

int main()
{
    fun(5);
    return 0;
}
```

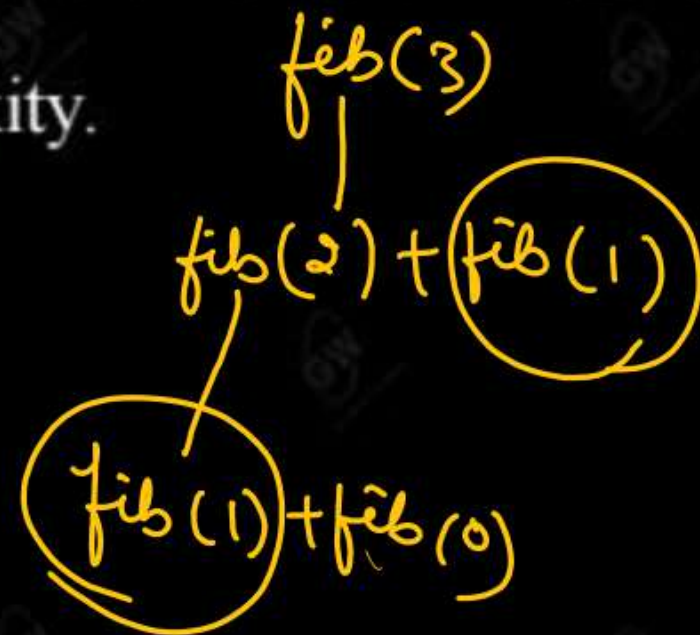
Space Complexity
 $O(1)$

output -
5 4 3 2 1

Which is better?

- Generally, tail recursions are always better.
- Even though they both have same time complexity and Auxiliary space, tail recursions takes an edge in terms of memory in function stack.
- Head recursions will wait in function stack memory until the post recursion code statements are executed which causes a latency in overall results, whereas tail recursions will be terminated in function stack over execution.

1. It makes code easier to write.
2. Solves naturally recursive problems:
 - As recursion breaks more difficult problem into less complex steps and comes up with easier version of its solution, so it solves naturally recursive problems.
3. Reduce time complexity:
 - It's observed that recursion sometimes increases the time it took for a function to complete the task. But, If we calculate the Fibonacci number using recursion then the time taken must be greater, If you memorize the result then you can reduce the time complexity.
4. Better at tree traversal



Disadvantages of Recursion

1. Takes more memory

- Each function call is stored in the system stack.
- Too many recursive calls can cause stack overflow.

2. Slower execution

- Every recursive call takes extra time (function call overhead).
- Iterative solutions are usually faster.

3. Hard to debug

- Tracing the flow of recursive functions is more difficult compared to loops.

4. Not always efficient

- Some recursive algorithms (like naive Fibonacci) do repeated work, making them inefficient.

Factorial

Non-Recursive

Time Comp. $\rightarrow O(n)$

Space comp. $\rightarrow O(1)$

Recursive

$O(n)$

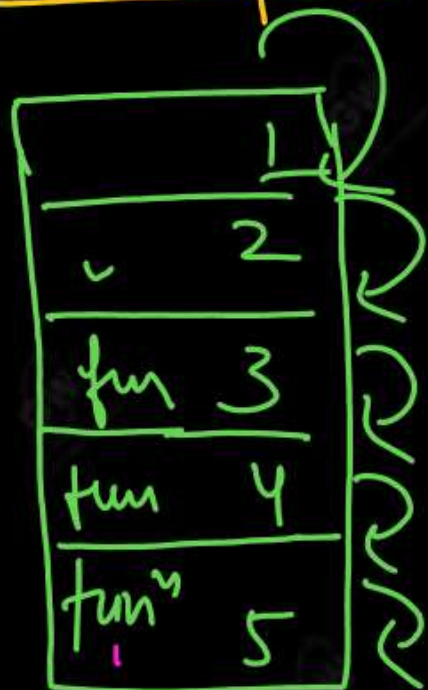
$O(n)$

↓
due to stack

Q: Describe a procedure to convert a recursive algorithm to a non recursive Algorithm.

- (AKTU2018-19)
- 1) Tail recursion can be directly converted to iterative solution. (As explained in slide (10))
 - 2) Using Stack

Example.



```
void fun(int n)
{
    if (n > 0)
    {
        fun(n-1);
        printf("%d", n);
    }
}
```

output $\Rightarrow$ 12345

output

1 2 3 4 5

```

#include <stdio.h>
#define MAX 10
int top = -1;
int a[MAX];
void push(int x)
{
    if (top == MAX - 1)
        printf("Overflow");
    else
    {
        top++;
        a[top] = x;
    }
}
int pop()
{
    if (top == -1)
        printf("Underflow");
    else
        return(a[top]);
    top--;
}

```

```

int main()
{
    int n = 5, i;
    for (i = n; i >= 1; i--)
    {
        push(i);
    }
    for (i = 1; i <= n; i++)
    {
        printf("%d", pop());
    }
    return 0;
}

```

4	1
3	2
2	3
1	4
0	5

output:-

1 2 3 4 5

Data Structure

Unit-2 DPP

1. What do you understand by tail recursion? **(AKTU2022-23,2021-22, 2020-21,2019-20)**
2. Discuss disadvantages of recursion with some suitable example. **(AKTU2021-22)**
3. Describe a procedure to convert a recursive algorithm to a non recursive Algorithm. **(AKTU2018-19)**

Gateway Classes

Data Structure

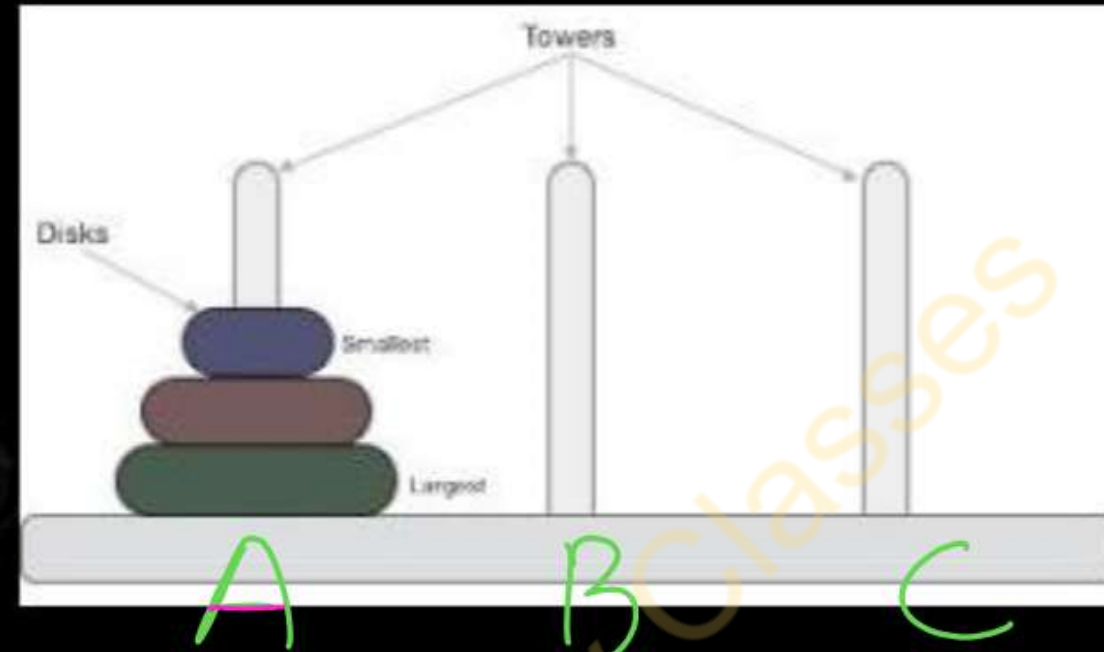
Unit-2 DPP

1. What do you understand by tail recursion? **(AKTU2022-23,2021-22, 2020-21,2019-20)**
2. Discuss disadvantages of recursion with some suitable example. **(AKTU2021-22)**
3. Describe a procedure to convert a recursive algorithm to a non recursive Algorithm. **(AKTU2018-19)**

Gateway Classes

Tower of Hanoi (Imp)

It is a mathematical puzzle which consists of three towers (pegs) and more than one rings.
disks



- These disks are of different sizes and stacked in an ascending order, i.e. the smaller one placed over the larger one. In other variations of the puzzle where number of disks increase, but tower count remains same.

Objective

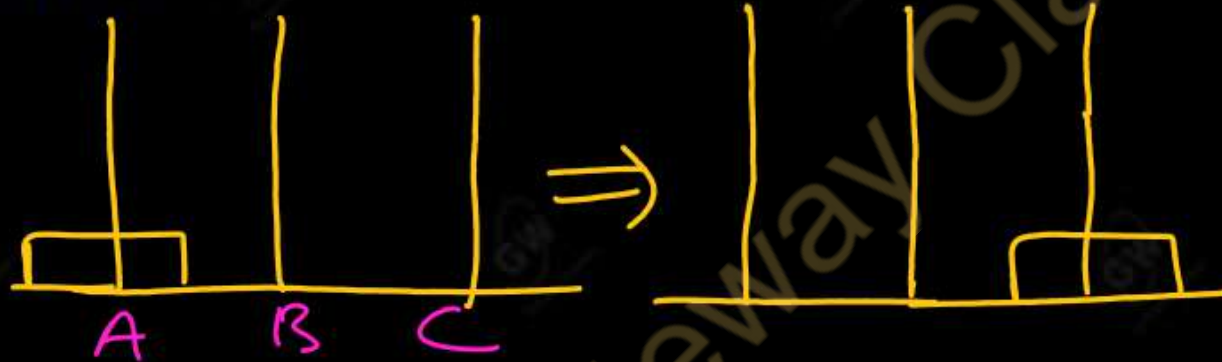
To move all disks to some another tower without violating the sequence of arrangement.

Rules to be followed

1. Only one disk can be moved among the towers at any given time.
2. Only the "top" disk can be removed.
3. No large disk can be placed over a small disk.

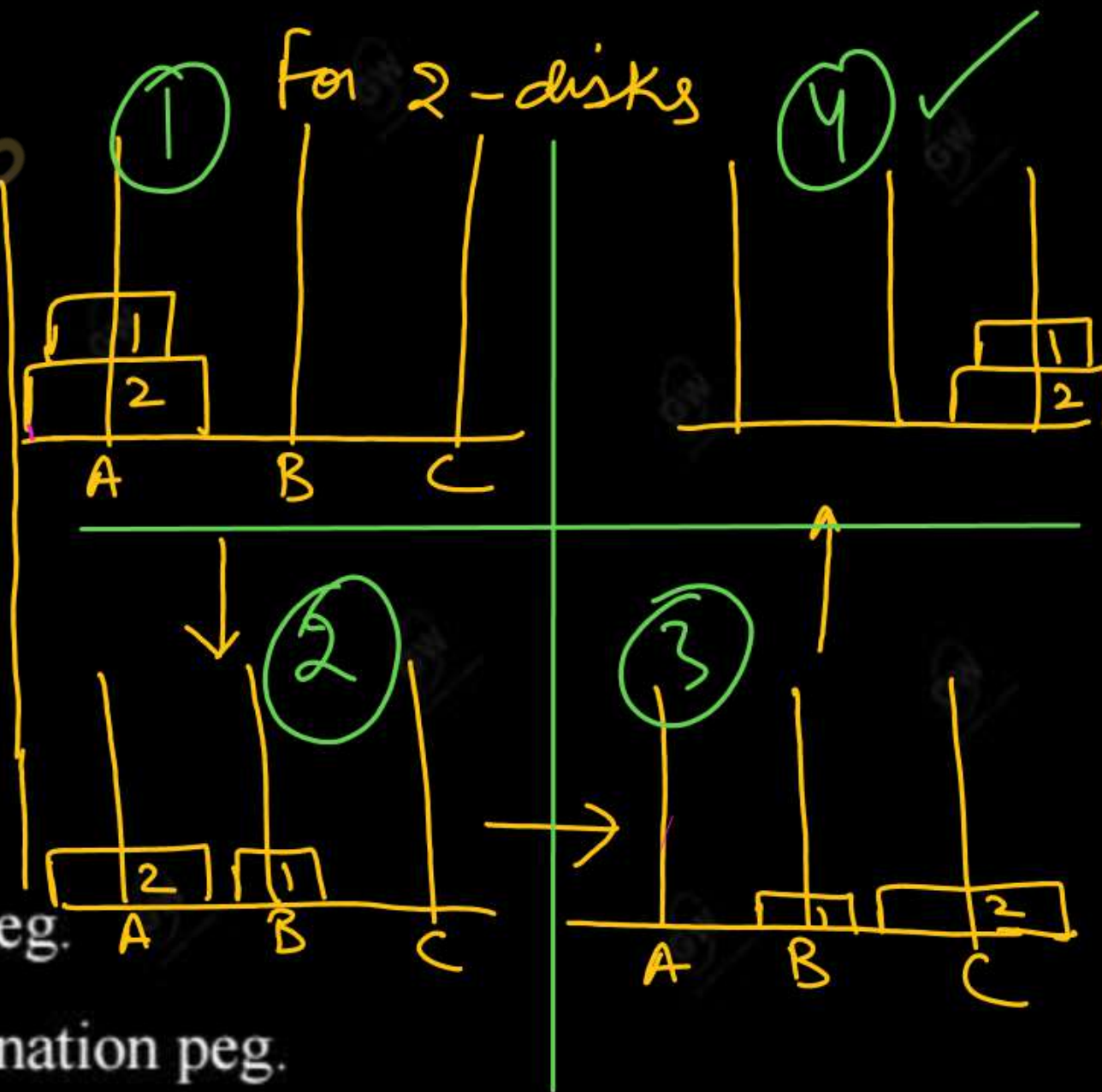
Algorithm

➤ If we have only one disk:

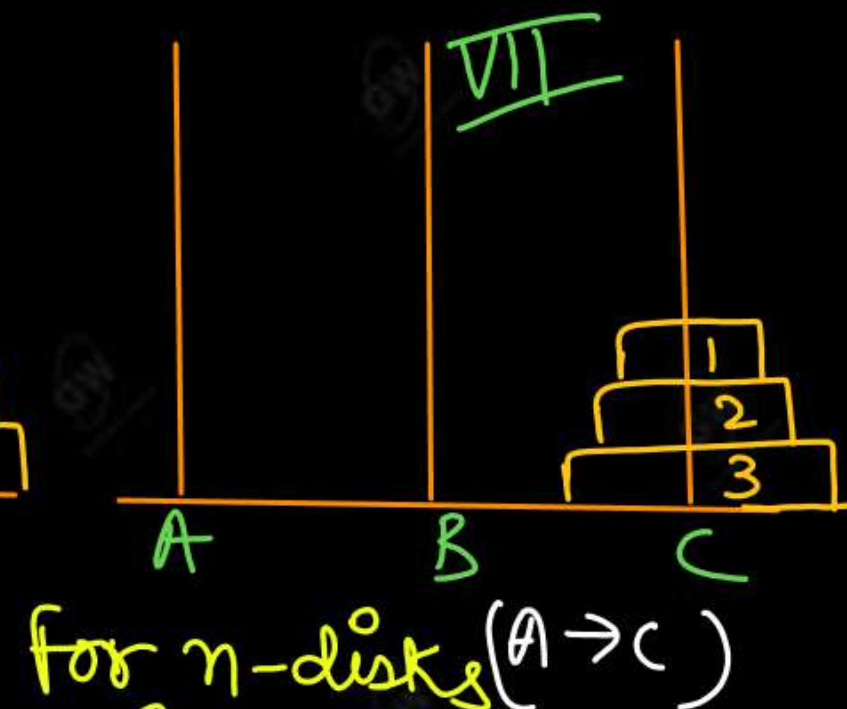
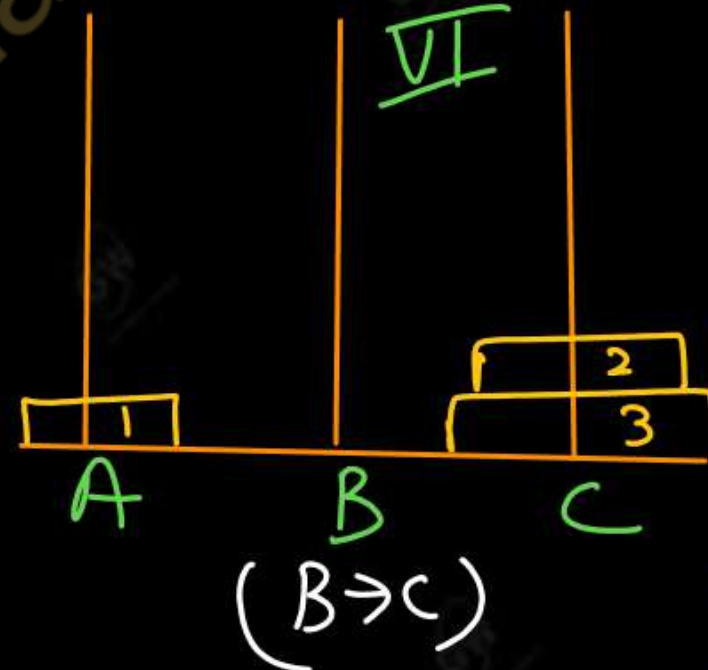
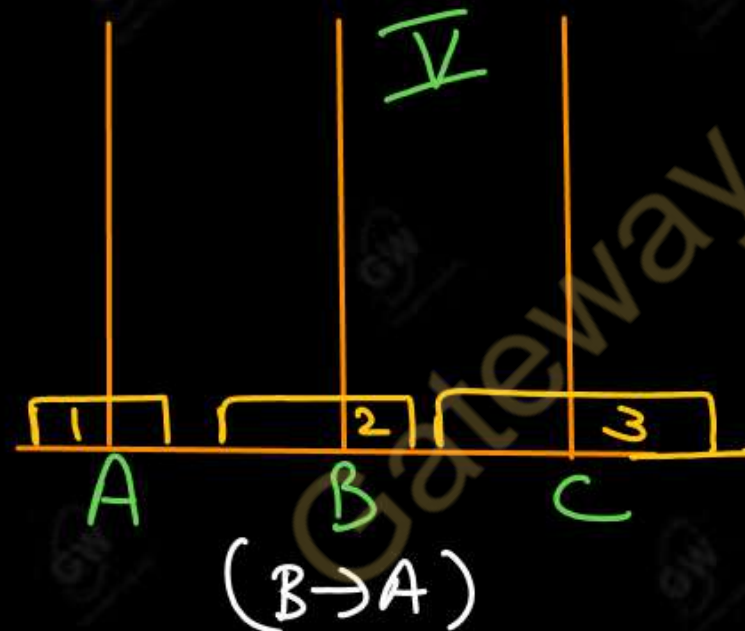
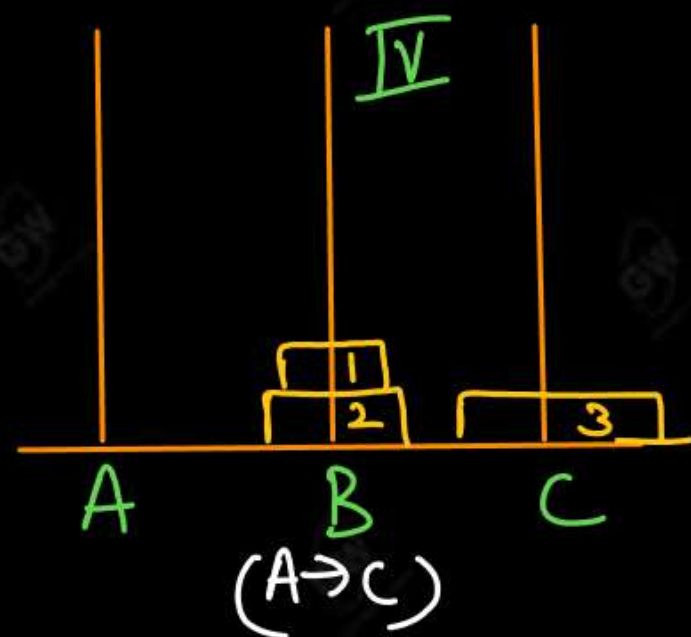
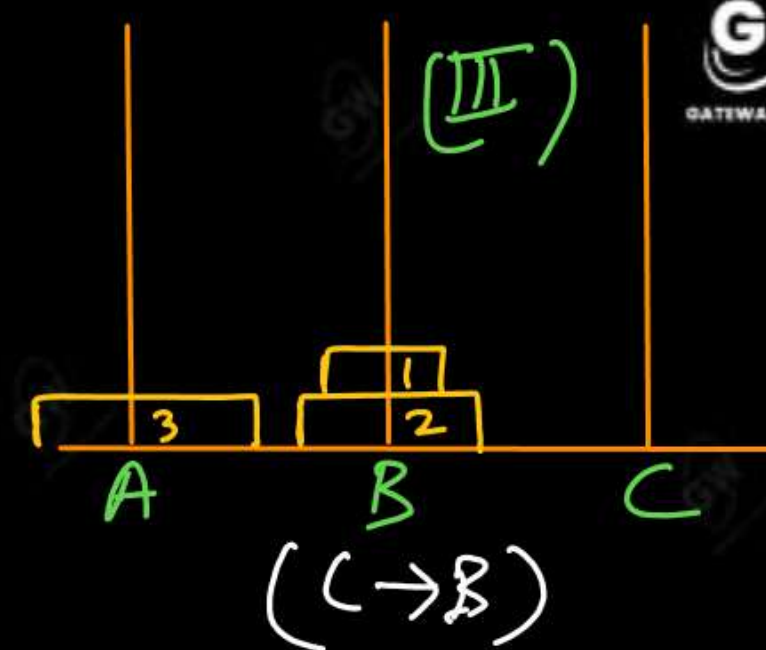
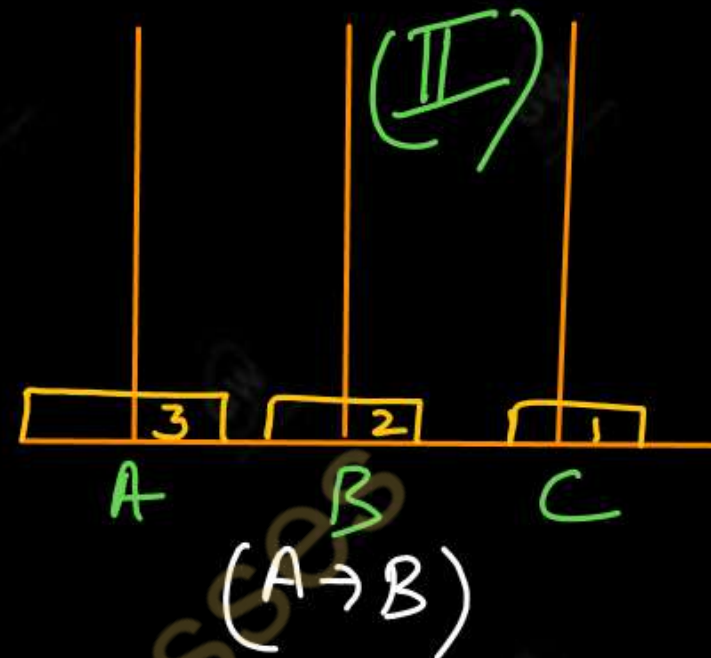
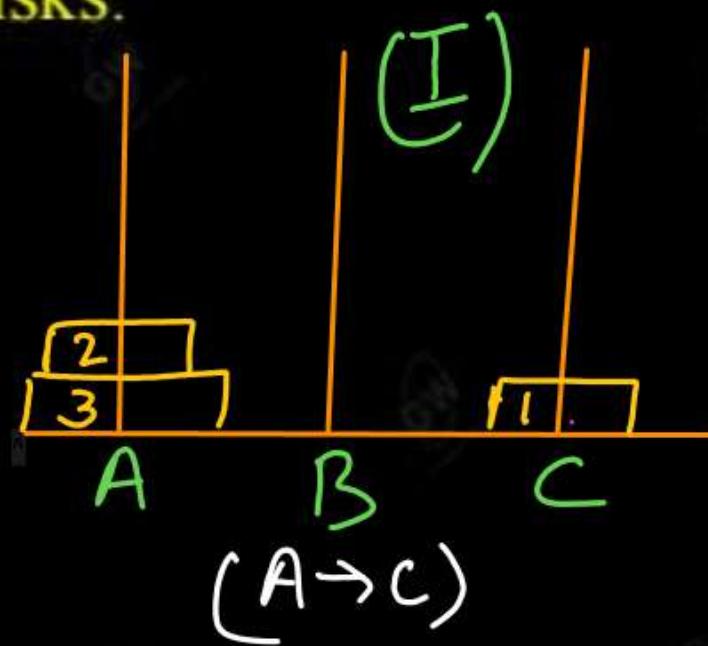
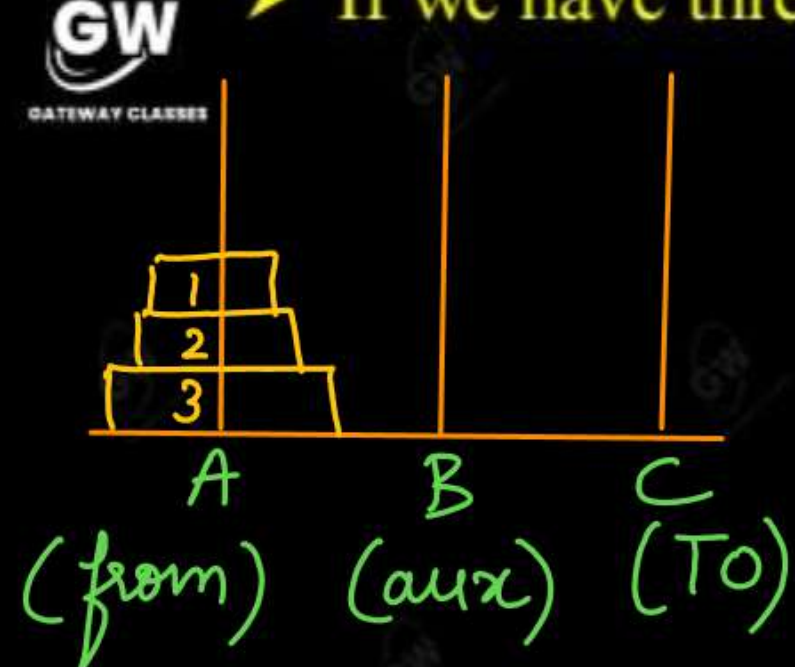


➤ If we have 2 disks:

1. First, we move the smaller (top) disk to aux peg.
2. Then, we move the larger (bottom) disk to destination peg.
3. And finally, we move the smaller disk from aux to destination peg.



➤ If we have three disks:



Minimum steps to solve Tower of Hanoi puzzle with n disks: $2^n - 1$ steps

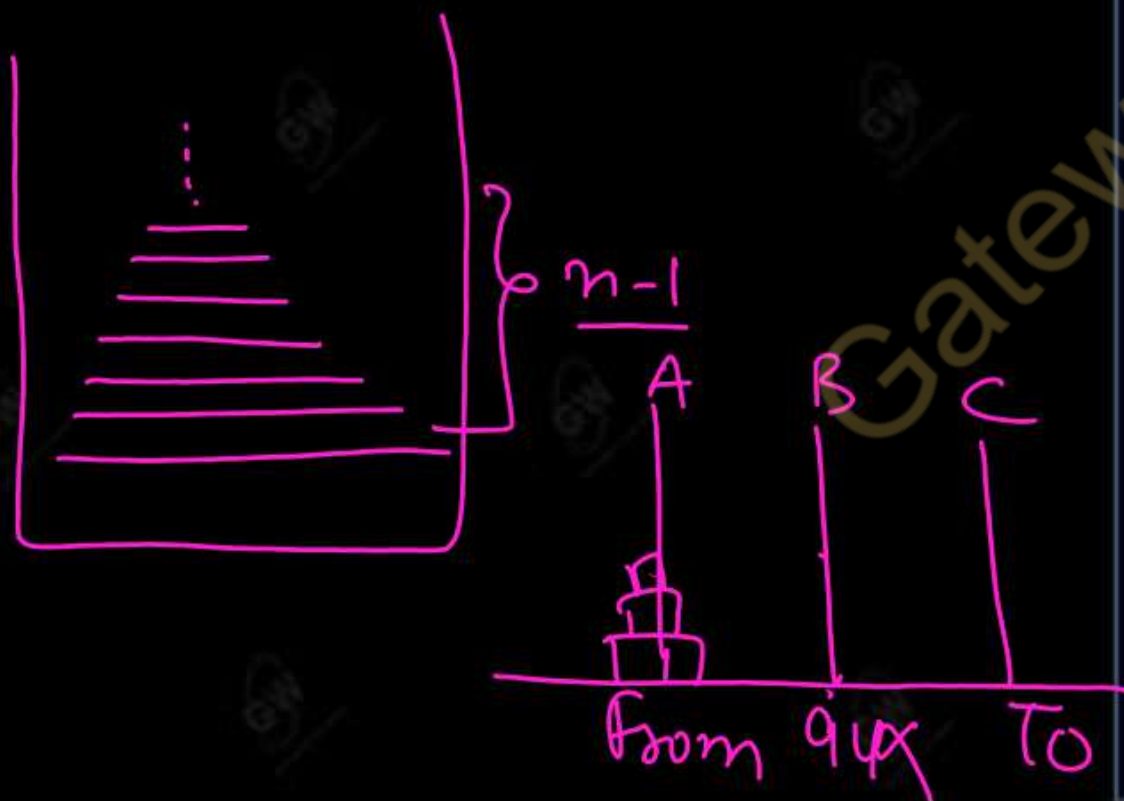
n disks

The steps to follow are –

Step 1 – Move $n-1$ disks from source to aux

Step 2 – Move n th disk from source to dest

Step 3 – Move $n-1$ disks from aux to dest



A recursive algorithm for Tower of Hanoi

START

Procedure TOH(disk, source, aux, dest)

IF disk == 1,

THEN move disk from source to dest

ELSE

TOH(disk - 1, source, dest, aux) // Step 1

Move disk from source to dest // Step 2

TOH(disk - 1, aux, source, dest) // Step 3

END IF

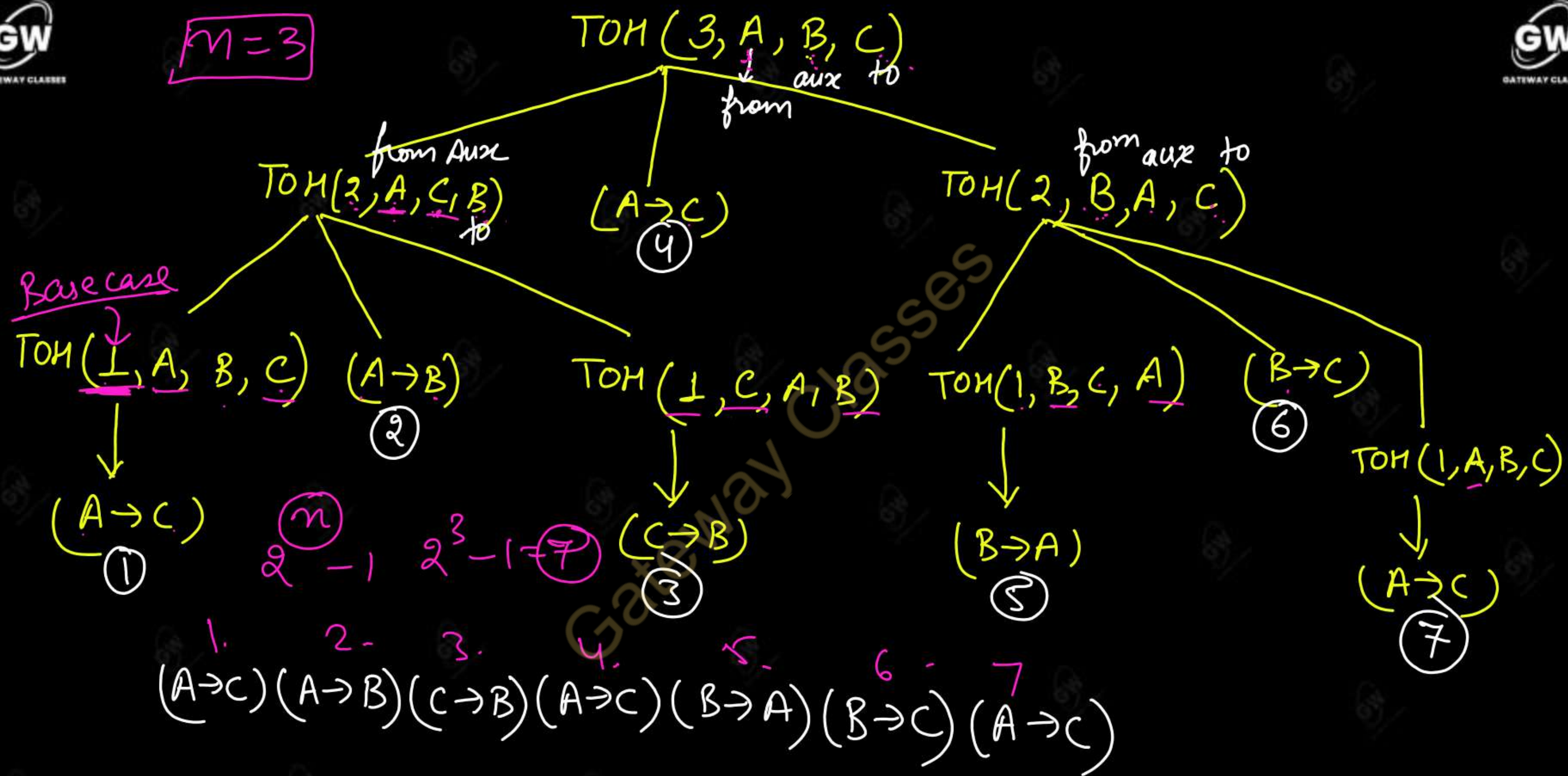
END Procedure STOP

```
#include <stdio.h>
void towerOfHanoi(int n, char fromA, char auxB, char toC)
{
    if (n == 1) // Base condition
    {
        printf("\n Move disk 1 from rod %c to rod %c", from, to);
        return;
    }

    towerOfHanoi(n-1, from, to, aux); ①
    printf("\n Move disk %d from rod %c to rod %c", n, from, to); ②
    towerOfHanoi(n-1, aux, from, to); ③
}

int main()
{
    int n = 3; // Number of disks
    towerOfHanoi(n, 'A', 'B', 'C'); // A, B and C are names of rods
    return 0;
}
```

$M=3$



Q: If the Tower of Hanoi is operated on $n=10$ disks, calculate the total number of moves.

(AKTU2018-19)

$$n = 10$$

$$\text{No. of moves} = 2^n - 1$$
$$= 2^{10} - 1 = 1024 - 1$$

$$= \frac{1023}{1} \text{ Ans}$$

Data Structure

Unit-2 DPP-8

1. Write the recursive solution for Tower of Hanoi problem. **(AKTU2019-20)**
2. Explain Tower of Hanoi problem and write a recursive algorithm to solve it. **(AKTU2019-20)**
3. Describe a procedure to convert a recursive algorithm to a non recursive Algorithm.
(AKTU2018-19)
4. If the Tower of Hanoi is operated on $n=10$ disks, calculate the total number of moves.
(AKTU2018-19)

- **Queue** is a non-primitive linear data structure that permits insertion of an element at one end and deletion of an element at the other end.
- The end at which the deletion of an element take place is called **front**, and the end at which insertion of a new element can take place is called **rear**.
- Queue is also referred to as *First-In-First-Out (FIFO)* list.
- Consider a railway reservation booth, at which we have to get into the reservation queue. New customers got into the queue from the rear end, whereas the customers who get their seats reserved leave the queue from the front end. It means the customers are serviced in the order in which they arrive the service center.



AKTU

B.Tech III-Sem



GATEWAY CLASSES

CS IT & CS Allied

DATA STRUCTURE

Unit-2 Lecture-9

Today's Target

- Introduction to queue data structure
- Array Implementation of Linear Queue
- Linked list implementation of Linear queue
- AKTU PYQs
- DPP



By Dr. Nidhi Parashar Ma'am

Ph.D (CSE)

- **M.Tech Gold Medalist**
- **Net Qualified**

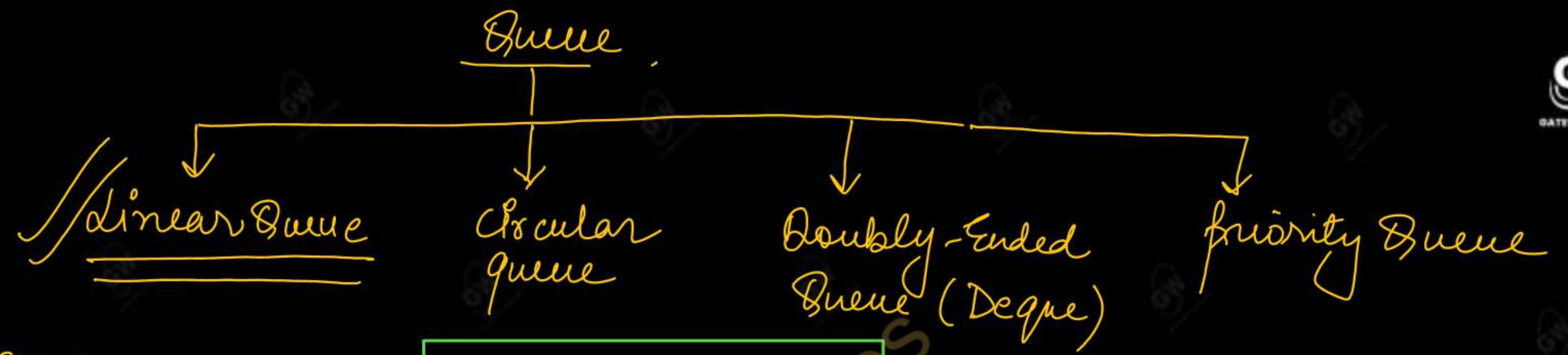
- The Queue ADT is a linear data structure that follows the FIFO (First In, First Out) principle.
- It allows elements to be inserted at one end (rear) and removed from the other end (front).

Operations:



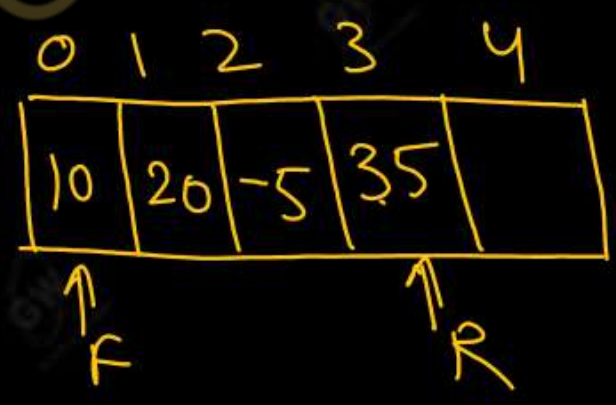
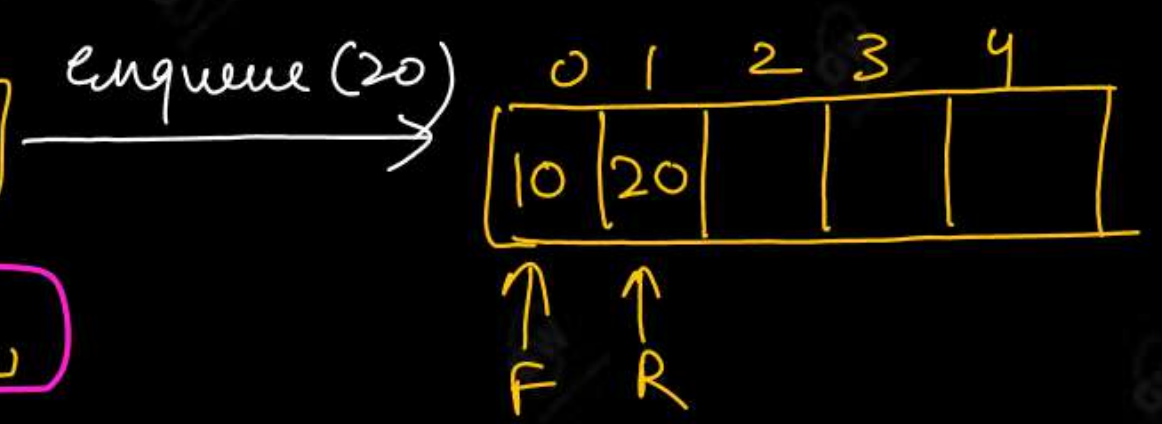
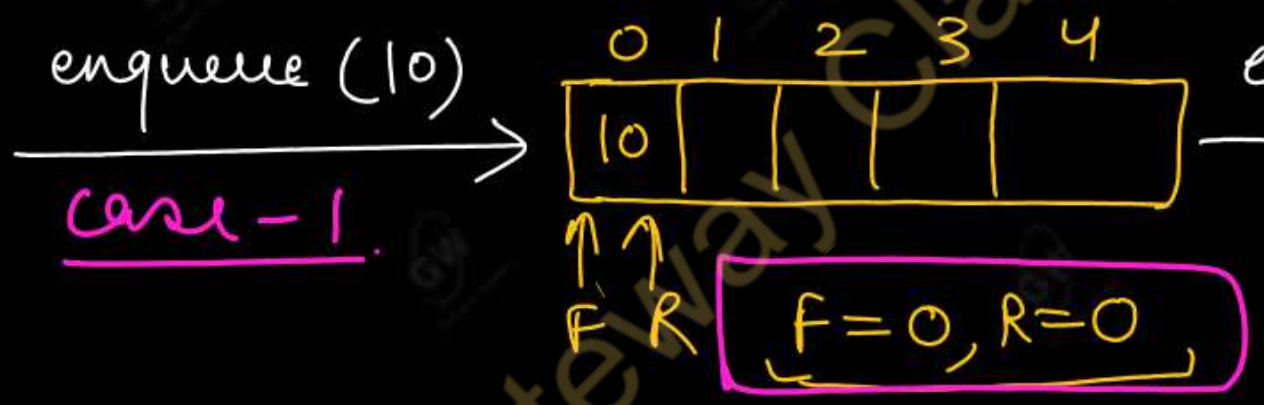
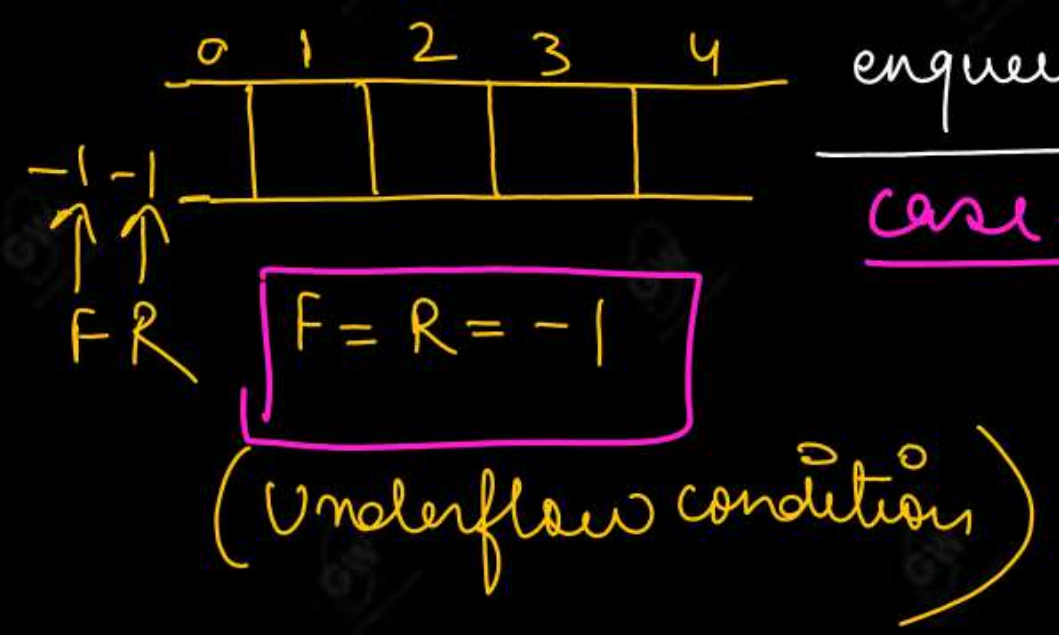
- **enqueue()**: Insert an element at the end of the queue.
- **dequeue()**: Remove and return the first element of the queue, if the queue is not empty.
- **peek()**: Return the element of the queue without removing it, if the queue is not empty.
- **size()**: Return the number of elements in the queue.
- **isEmpty()**: Return true if the queue is empty; otherwise, return false.
- **isFull()**: Return true if the queue is full; otherwise, return false. Only relevant for fixed-capacity queues (e.g., array-based).





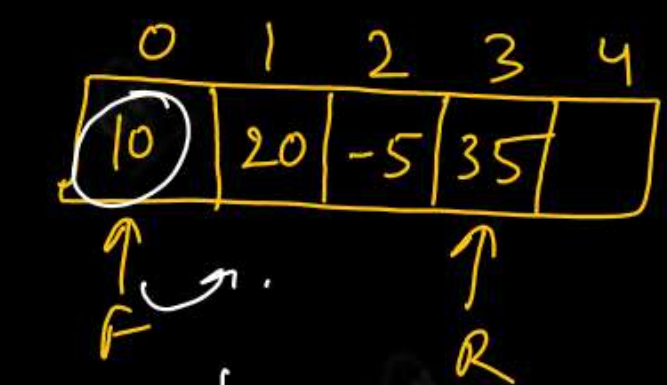
Empty queue

Linear Queue

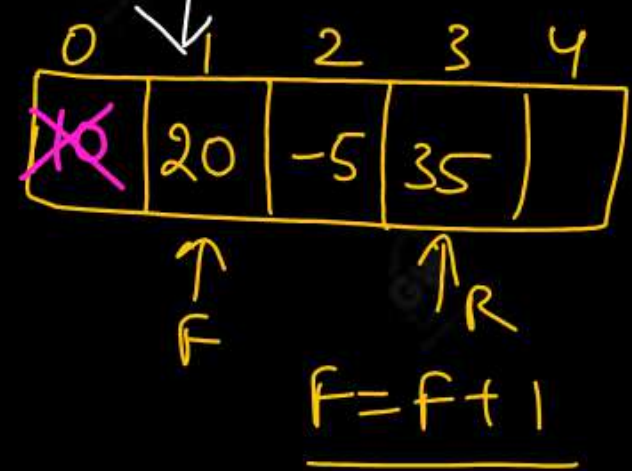


- ① enqueue (-5)
- ② enqueue (35)

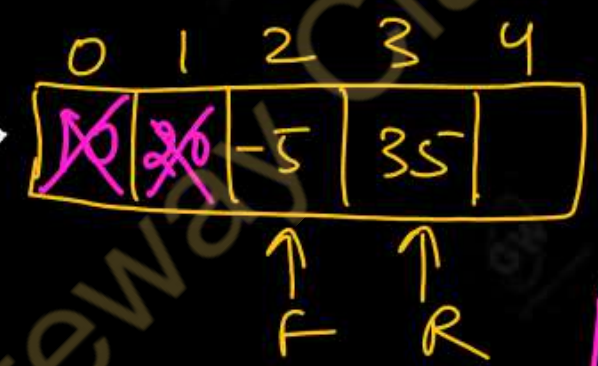
$R = R + 1$



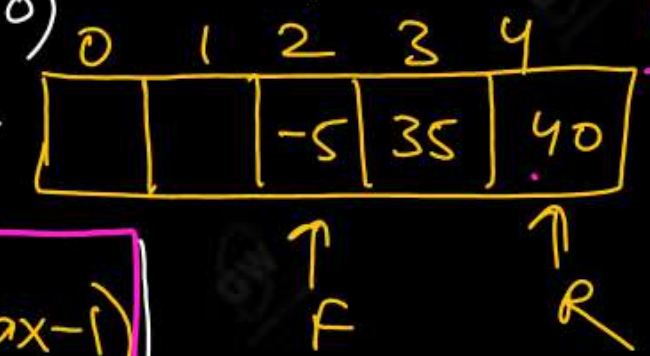
dequeue()



dequeue()



enqueue(40)



MAX = 5

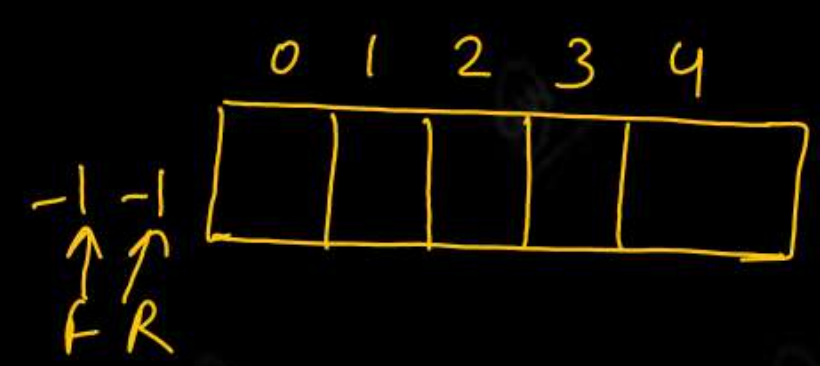
~~if (R == Max - 1)~~
overflow

enqueue(60)

R = R + 1 ⇒ R = 5

invalid index

overflow

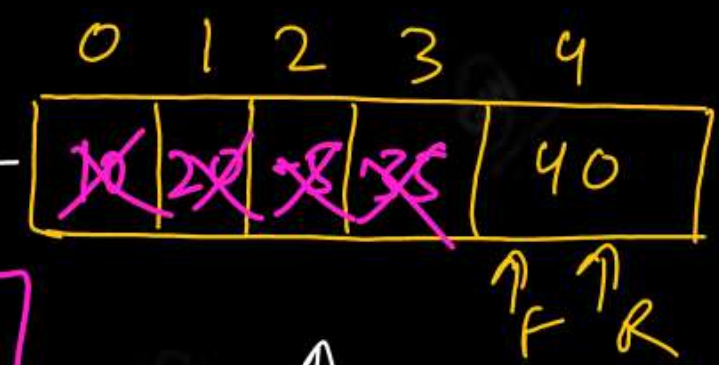


set F = R = -1

only one element present in queue

F == R

dequeue



dequeue()
dequeue()

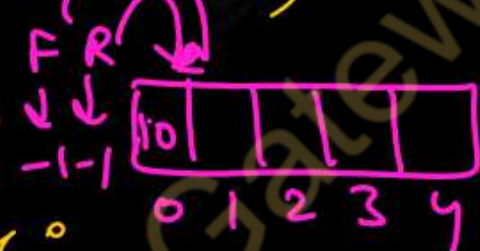
Array Implementation of Linear Queue

```
#include <stdio.h>
#define MAX 5
int queue[MAX];
int front = -1, rear = -1;
```

void enqueue (int x)

```
{
    if (rear == MAX - 1)
    {
        printf("Queue is Full \n");
    }
    else if (front == -1 && rear == -1)
    {
        front = rear = 0;
        queue[rear] = x;
    }
    else
    {
        rear = rear + 1;
        queue[rear] = x;
    }
}
```

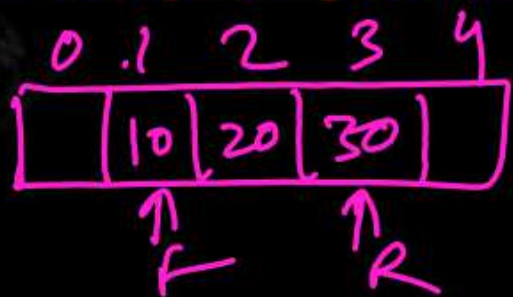
overflow condition



void dequeue()

```
{
    if (front == -1 && rear == -1)
    {
        printf("Queue is Empty \n");
    }
    else if (front == rear) // If only one element in Queue
    {
        printf("Deleted element is = %d", queue[front]);
        front = rear = -1;
    }
    else
    {
        printf("Deleted Element is = %d", queue[front]);
        front = front + 1;
    }
}
```

Array Implementation of Linear Queue



```
void display()
{
    int i;
    if (Front == -1 && Rear == -1)
    {
        printf("Queue is Empty");
    }
    else
    {
        for (i = Front; i <= Rear; i++)
        {
            printf("%d\n", queue[i]);
        }
    }
}
```

```
void peek()
{
    if (Front == -1 && Rear == -1)
    {
        printf("Queue is Empty");
    }
    else
    {
        printf("Element at front position is = %d", queue[Front]);
    }
}
```

Array Implementation of Linear Queue

```
int main()
{
    int choice, x;
    printf("Enter 1 to insert into queue\n");
    printf("Enter 2 to delete from queue\n");
    printf("Enter 3 to display all queue elements\n");
    printf("Enter 4 to perform peek operation\n");
    printf("Enter 5 to exit\n");
    printf("Enter your choice");
    scanf("%d", &choice);
```

```
while(1)
{
    switch(choice)
    {
        case 1: printf("Enter element to insert");
                scanf("%d", &x);
```

```
                enqueue(x);
                break;
        case 2: dequeue();
                break;
        case 3: display();
                break;
        case 4: peek();
                break;
        case 5: exit(0);
        default: printf("Invalid choice");
    }
    return 0;
}
```

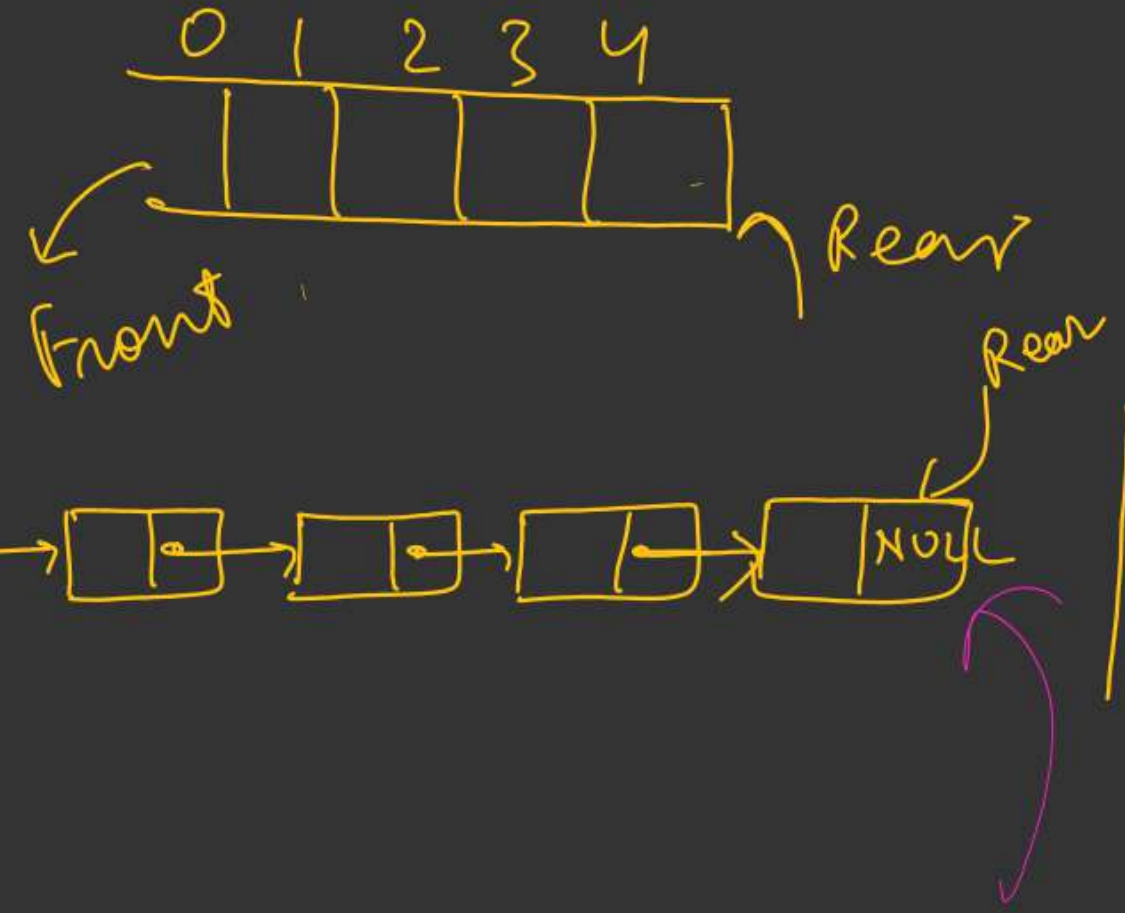
Time Complexity

enqueue() $\longrightarrow O(1)$

dequeue() $\longrightarrow O(1)$

display() $\longrightarrow O(n)$

peek() $\longrightarrow O(1)$



Linked list Implementation of Linear Queue

```
#include <stdio.h>

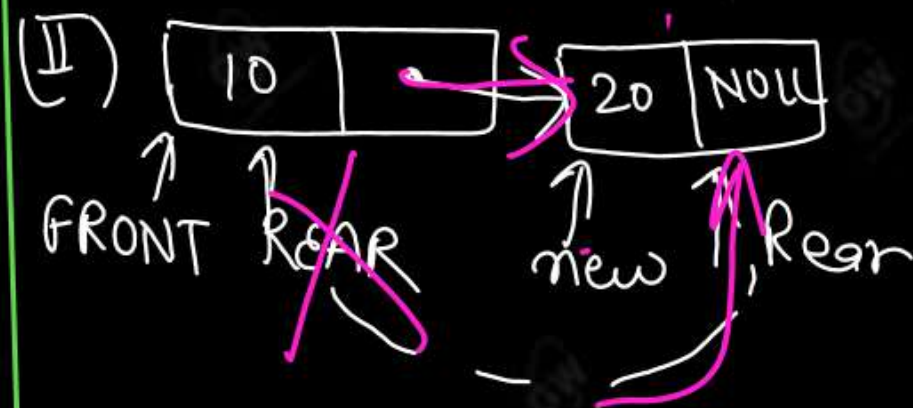
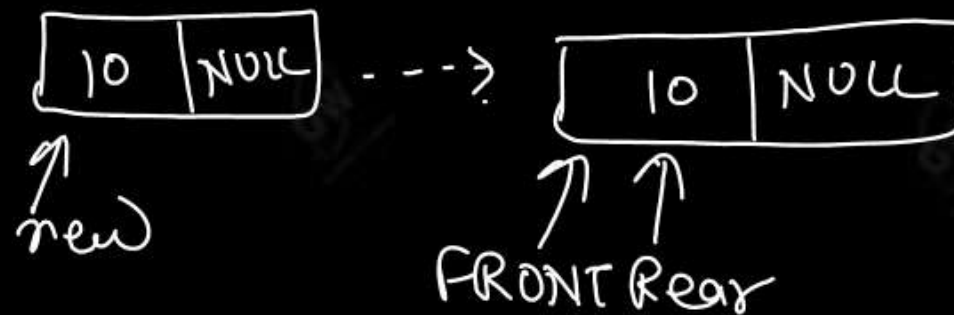
struct node
{
    int data;
    struct node *next;
};

struct node *Front = NULL;
struct node *Rear = NULL;

void enqueue(int x)
{
    struct node *new;
    new = (struct node*) malloc(sizeof(struct node));
    new->data = x;
    new->next = NULL;
}
```

```
if (Front == NULL && Rear == NULL)
{
    Front = Rear = new;
}
else
{
    Rear->next = new;
    Rear = new;
}
```

Front = NULL, Rear = NULL



Linked list Implementation of Linear Queue

```
void dequeue()
{ struct node *temp;
```

```
(I) if (Front == NULL & Rear == NULL) // Empty Queue
{ printf("Empty Queue"); }
```

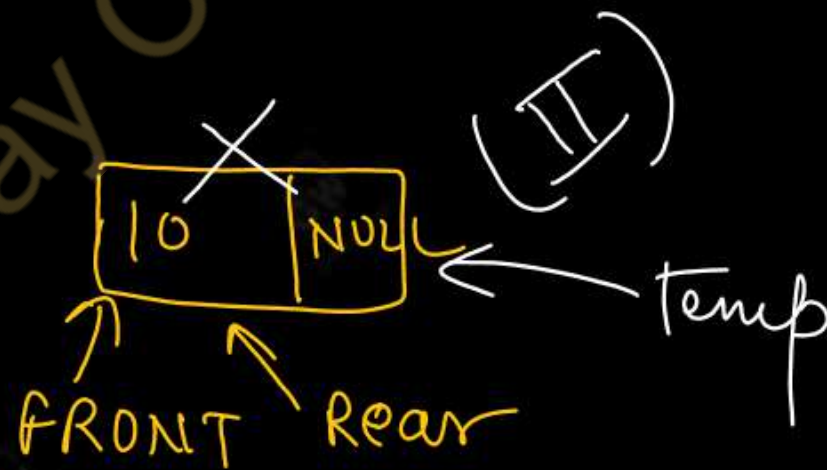
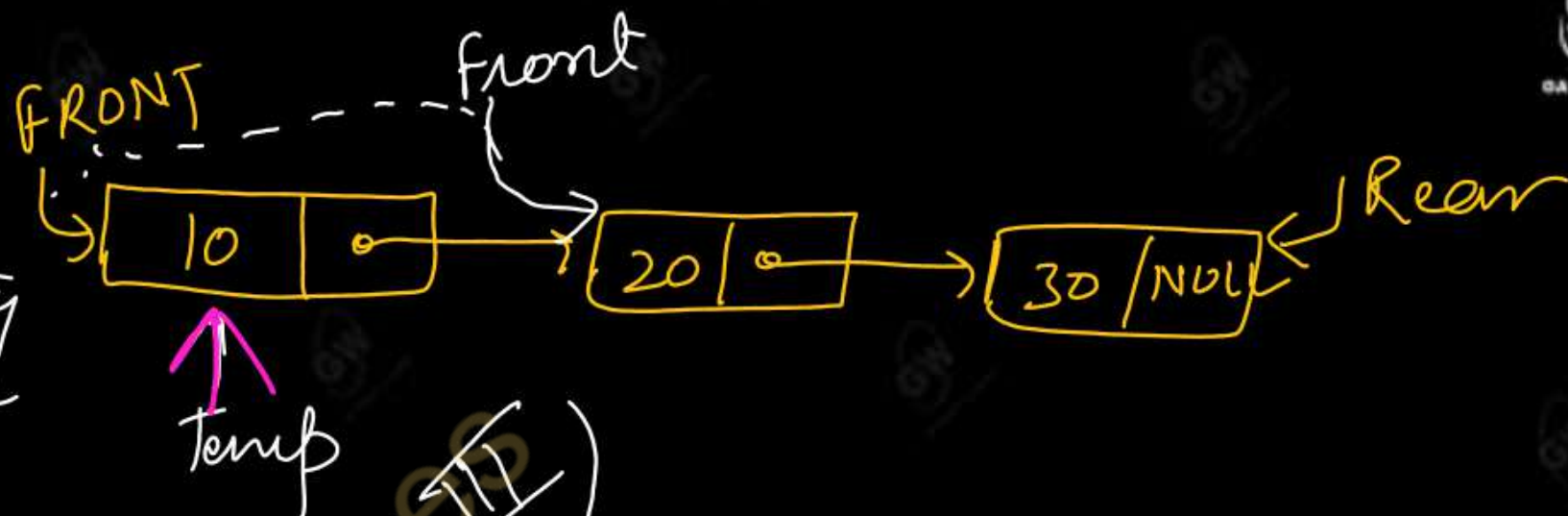
```
(II) else if (Front == Rear) // only one element in Queue
{
    temp = Front;
```

```
    Front = Rear = NULL;
    free(temp);
```

```
}
else {
    temp = Front; ✓
```

```
(III) printf("Deleted element = %d", temp->data);
```

```
    Front = Front->next;
    free(temp);
}
```

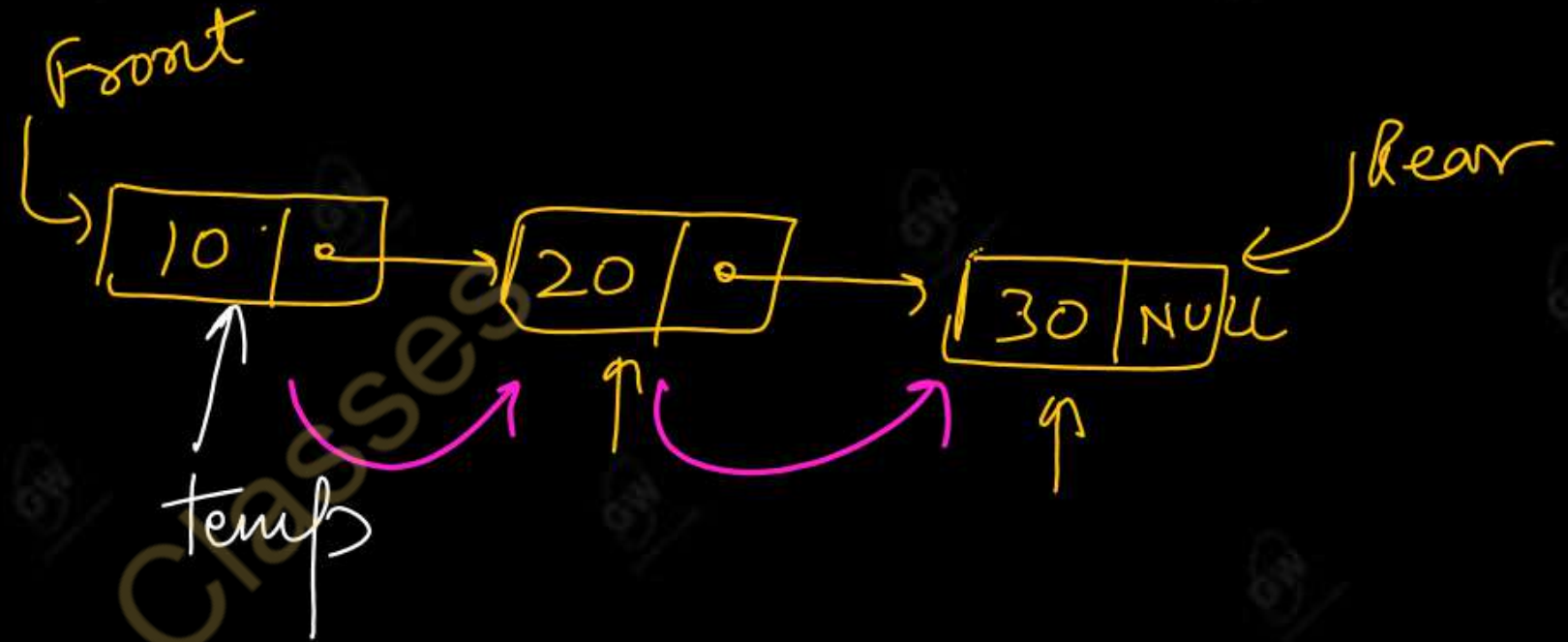


Linked list Implementation of Linear Queue

```
void display()
{
    struct node * temp = Front;
    if (Front == NULL && Rear == NULL)
    { printf("Empty Queue"); }
}
```

```
else
{
```

```
    while (temp != NULL)
    {
        printf("%d", temp->data);
        temp = temp->next;
    }
}
```



```
void peek()
{
    if (Front == NULL && Rear == NULL)
    { printf("Empty Queue"); }
    else { printf("%d", Front->data); }
}
```

Linked list Implementation of Linear Queue

```
int main()
{
    int choice, x;
    printf("Enter 1 to insert into queue\n");
    printf("Enter 2 to delete from queue\n");
    printf("Enter 3 to display all queue elements\n");
    printf("Enter 4 to perform peek operation\n");
    printf("Enter 5 to exit\n");
    printf("Enter your choice");
    scanf("%d", &choice);
```

```
while(1)
{
    switch(choice)
    {
        case 1: printf("Enter element to insert");
                scanf("%d", &x);
```

```
enqueue(x);
        break;
        case 2: dequeue();
                break;
        case 3: display();
                break;
        case 4: peek();
                break;
        case 5: exit(0);
        default: printf("Invalid choice");
    }
    return 0;
}
```

1. Waiting Lines in Daily Life (FIFO System)

Examples:

- Ticket counters (cinema halls, railway stations, airports)
- Bank queues (customers wait for service)
- Customer service call centers

2. Traffic Management

- **Traffic Signals:** Vehicles wait in line, and the front vehicle moves first when the signal turns green.
- **Toll Plazas:** Vehicles line up for toll payment, processed one by one.
- **Parking Lots:** Vehicles enter and exit in an orderly queue.

3. Printers in Offices and Homes

When multiple documents are sent to a printer, they are placed in a queue, where the first document is printed before the next.

1. Process Scheduling in Operating Systems

CPU scheduling in OS follows the queue data structure to manage processes efficiently.

- **Ready Queue:** Stores processes waiting for CPU execution.
- **Job Queue:** Holds all processes submitted for execution.
- **I/O Queue:** Manages processes waiting for input/output operations.

2. Disk Scheduling

In computer storage systems, disk scheduling algorithms use queues to manage read/write requests efficiently.

Example: Hard drive stores read/write requests in a queue and processes them sequentially or based on priority.

Unit-2 DPP-9

1. Construct a Queue using a linked list? Include functions insert(), delete(), print(). Explain how a circular queue overcomes the limitations of a simple queue. (AKTU2024-25)

2. Write a program in C for implementation of queue. Your program should at least contain ADD, CREATE, DELETE, FULL, EMPTY functions. (AKTU2019-20,2018-19)

void create()
{
 front = rear = -1;
}

int isEmpty()
{
 if (front == -1 && rear == -1)
 return 1;
 else
 return 0;
}

int isFull()
{
 if (rear == MAX-1)
 return 1;
 else
 return 0;
}



AKTU

B.Tech III-Sem



GATEWAY CLASSES

CS IT & CS Allied

DATA STRUCTURE

Unit-2 Lecture-10

Today's Target

- Introduction to circular queue data structure
- Array Implementation of Circular Queue
- Linked list implementation of Circular queue
- AKTU PYQs
- DPP

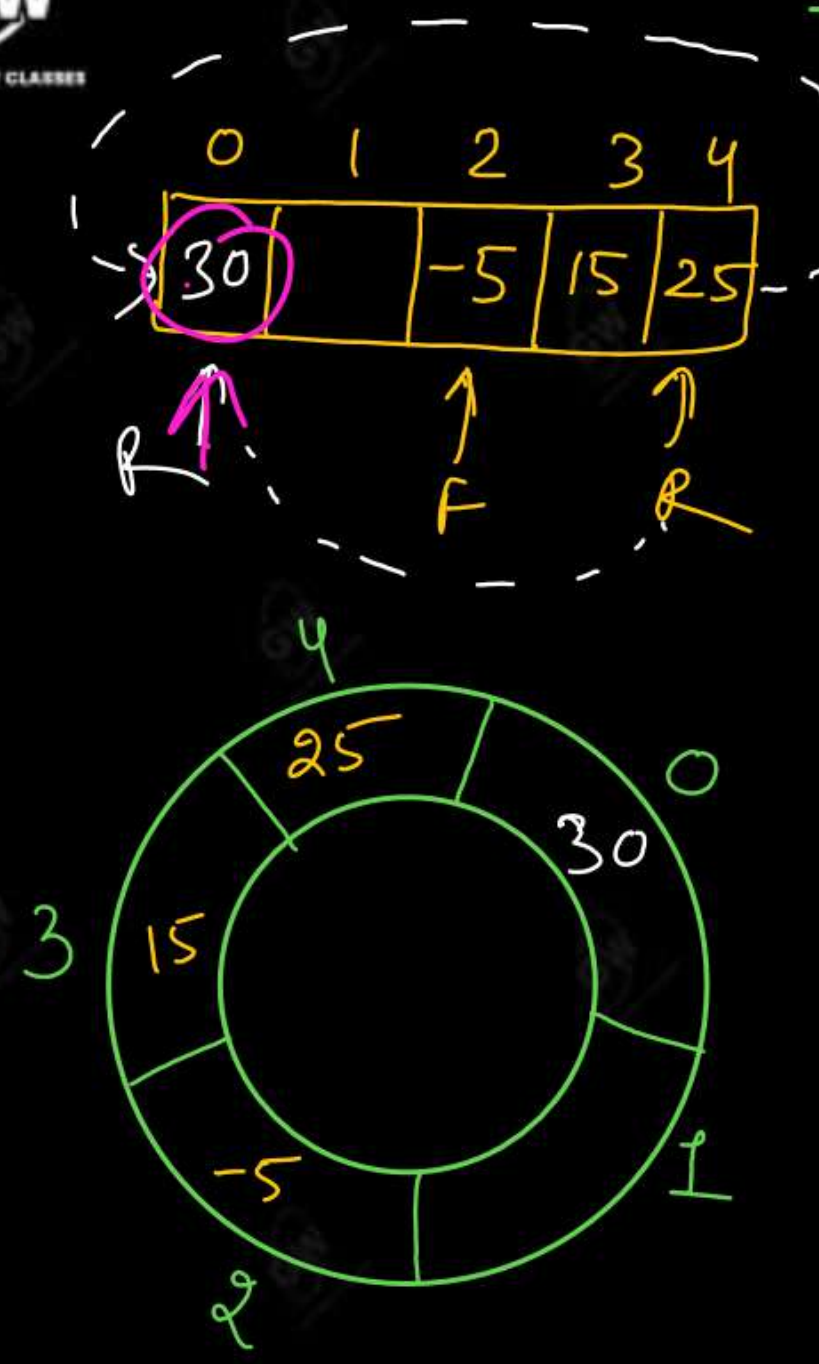


By Dr. Nidhi Parashar Ma'am

Ph.D (CSE)

- **M.Tech Gold Medalist**
- **Net Qualified**

How Circular Queue solves the problem?



In linear Queue

To enqueue (30)

$R = 4$

$R = (R + 1) \% N$

$= (4 + 1) \% 5$

$= 5 \% 5 \Rightarrow 0$

for enqueue() / Add() / insert()

$Rear = Rear + 1$

for dequeue() / Remove() / Delete()

$Front = Front + 1$

In circular Queue

for enqueue() / Add() / ~~Delete()~~ ^{Insert}

$Rear = (Rear + 1) \% N \rightarrow \text{Max size of array}$

for dequeue() / Remove() / Delete()

$Front = (Front + 1) \% N$

0	1	2	3	4
30		-5	15	25
↑ R		↑ F		

enqueue(40)

$$R = (R + 1) \% N$$

$$= 0 + 1 \% 5$$

$$1 \% 5 \Rightarrow \textcircled{1}$$

0	1	2	3	4
30	40	-5	15	25
↑ R	↑ F			

Circular Queue Empty condition
if (Front == -1 && Rear == -1)

Circular Queue Full condition

$$\text{if } ((Rear + 1) \% N == Front)$$

Important

Now if we want to insert one more element, then it has no space remaining as Queue is full.

0	1	2	3	4
30	40	-5	15	25

↑ ↑
R F

↓ dequeue()
↓ dequeue()

0	1	2	3	4
30	40	/	/	25

↑ ↑
R F

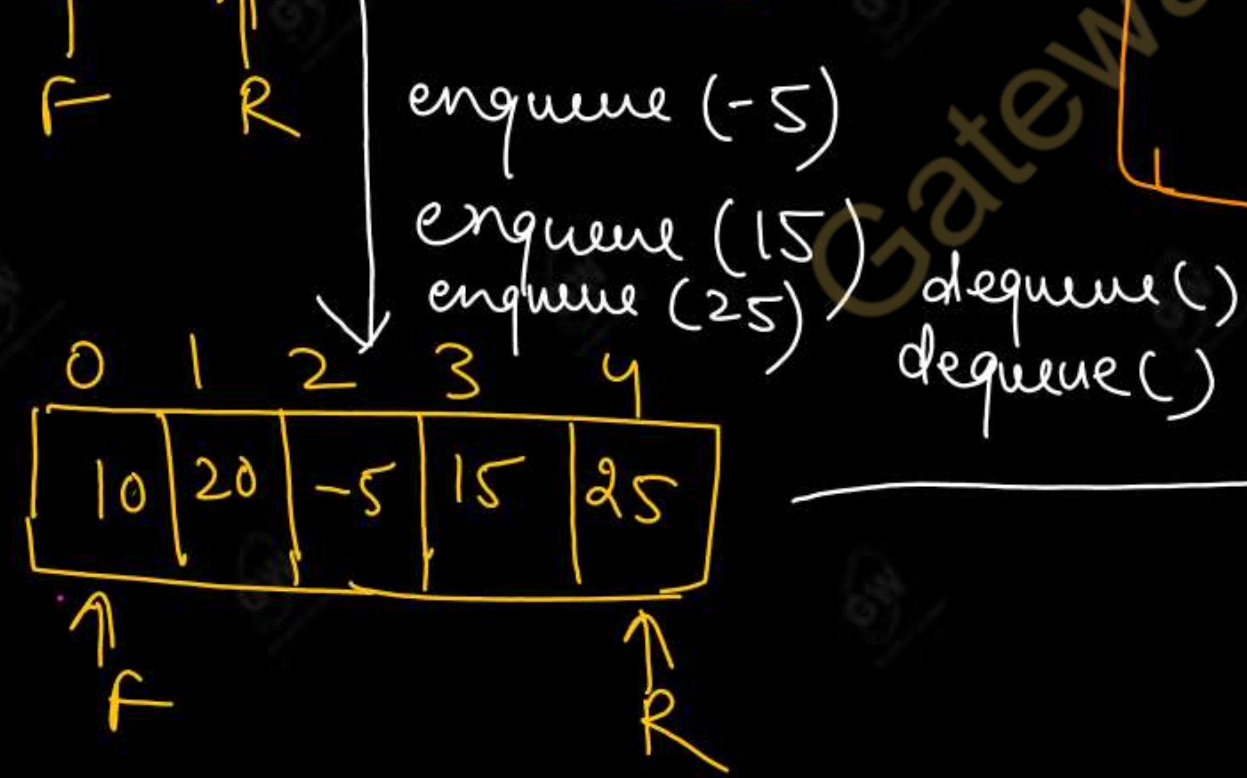
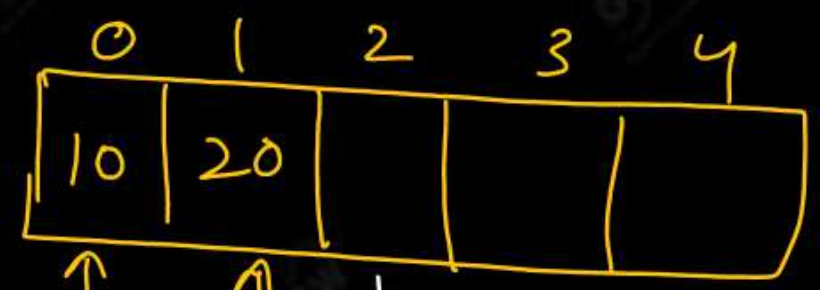
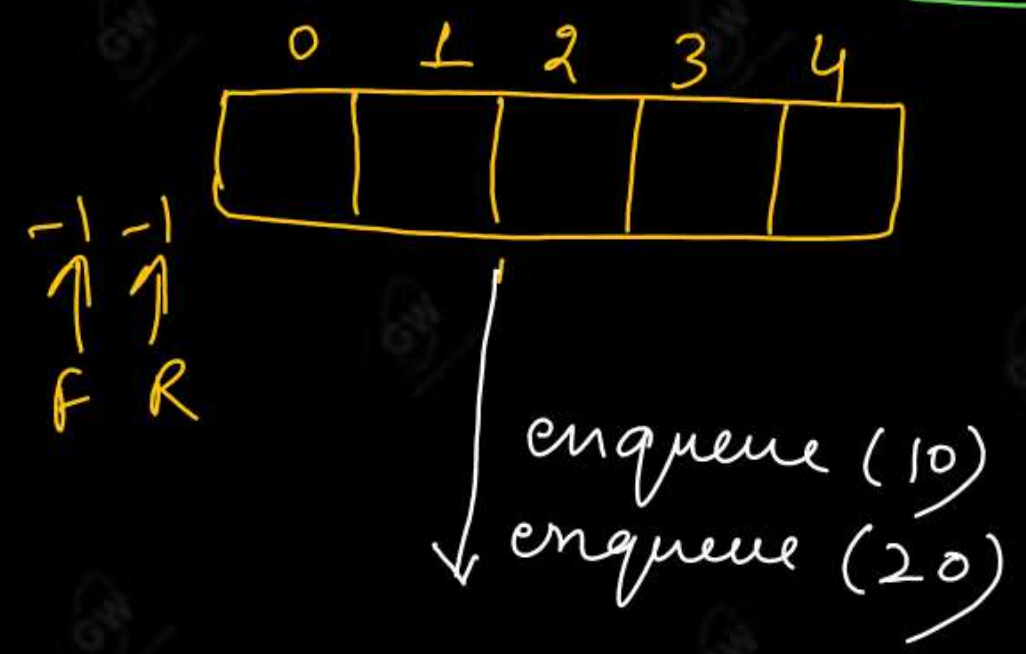
dequeue()

0	1	2	3	4
30	40	/	/	/

↑ ↑
F R

$$\begin{aligned}
 F &= (F+1) \% N \\
 &= (4+1) \% 5 \\
 &= 0
 \end{aligned}$$

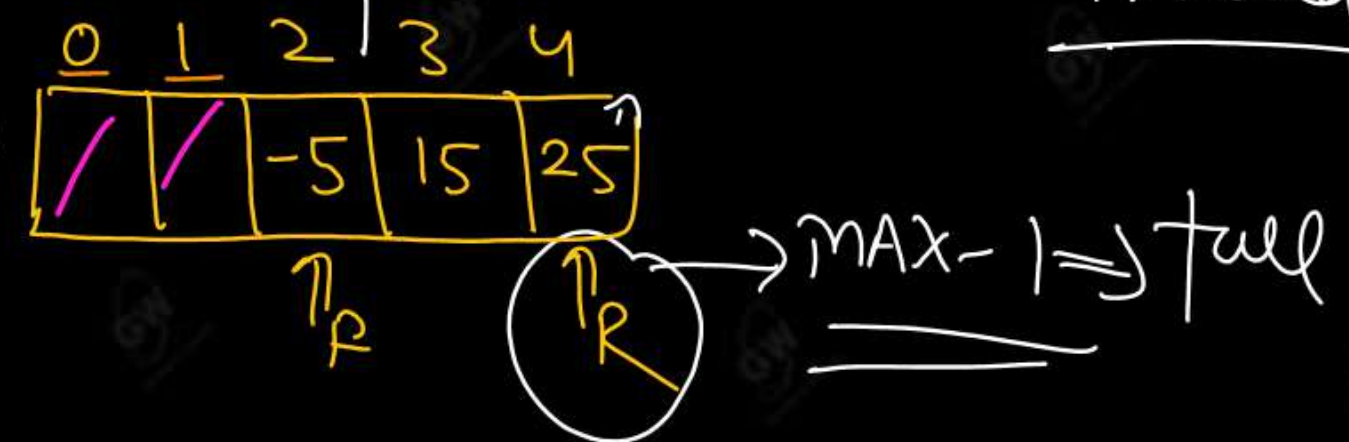
Drawback of linear Queue



It leads to a drawback of linear Queue as it has space to store two elements at index position 0 and 1, but still showing Queue Full.

Queue Full

To insert one more element
 $R = R + 1 \Rightarrow R = 4 + 1 = 5$
 Invalid



Array Implementation of Circular Queue

```
#include <stdio.h>
```

```
#define MAX 5
```

```
int queue[MAX];
```

```
int front = -1, rear = -1;
```

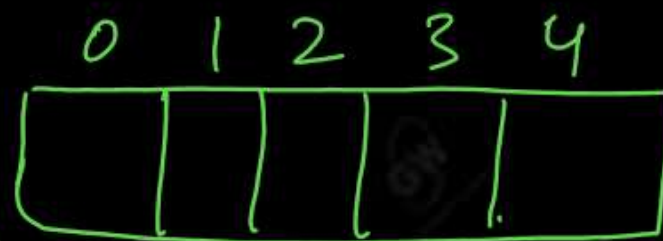
```
void enqueue(int x)
```

```
{ if ((rear + 1) % MAX == front) // Queue full condition
  { printf("Overflow\n"); }
  (I) else if (front == -1 && rear == -1) // Empty Queue
  { front = rear = 0;
    queue[rear] = x; }
```

```
else
```

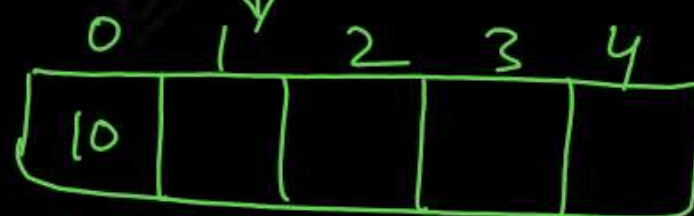
```
(II) { rear = (rear + 1) % MAX;
      queue[rear] = x; }}
```

(I)



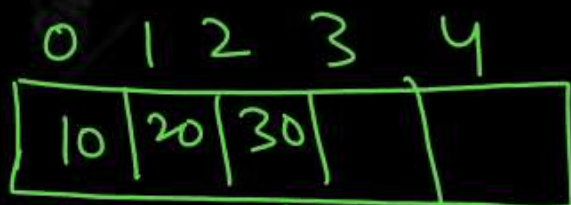
-1 -1
↑ ↑
F R

enqueue(10)



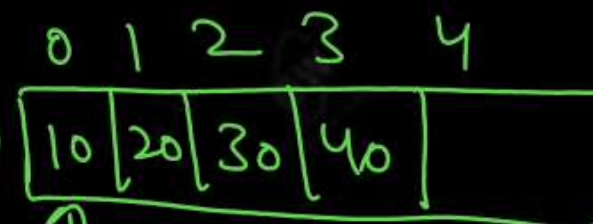
↑ ↑
F R

(II)



↑ ↑
F R

enqueue(40)



↑ ↑
F R

Array Implementation of Circular Queue

```
void dequeue()
```

```
{ if (front == -1 && rear == -1) // Empty Queue
```

```
{ printf("Underflow\n"); }
```

(I) else if (front == rear) // only one element in the queue

```
{ printf("Deleted element: %d\n", queue[front]);
```

```
front = rear = -1;
```

```
}
```

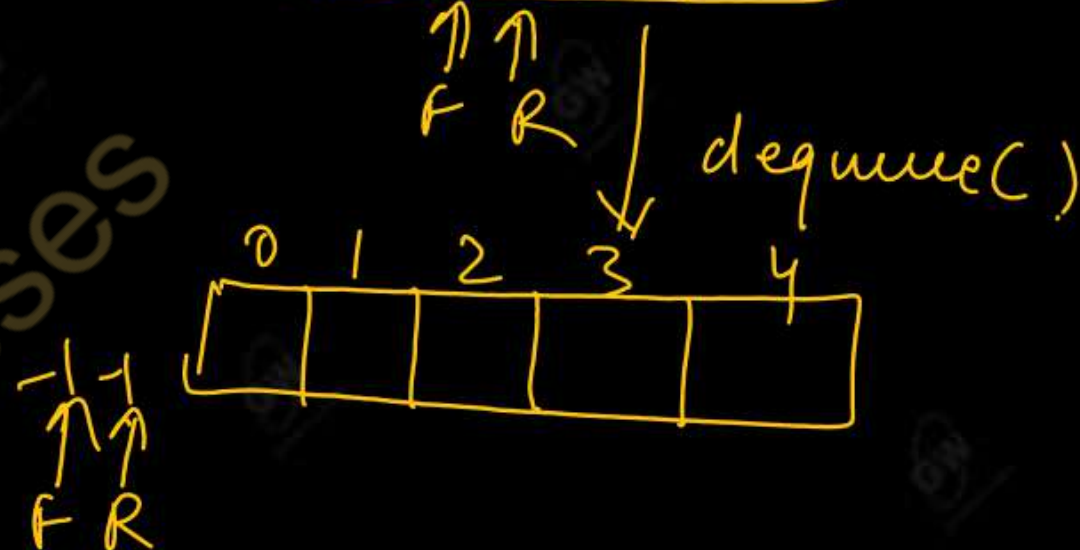
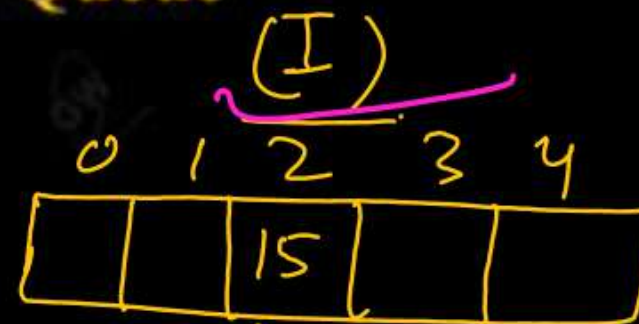
```
else
```

(II) { printf("Deleted element: %d\n", queue[front]);

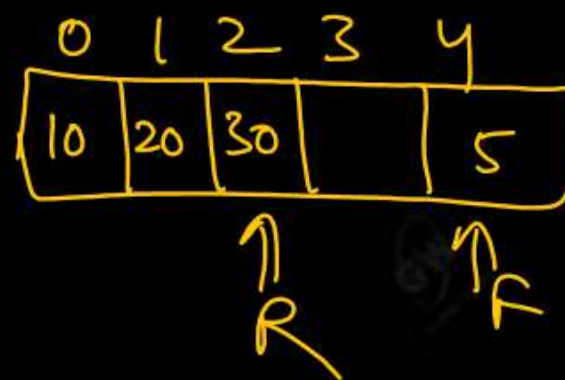
```
front = (front + 1) % MAX;
```

```
}
```

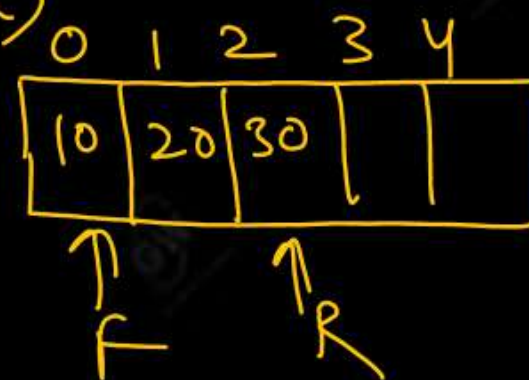
```
}
```



(II)



dequeue()

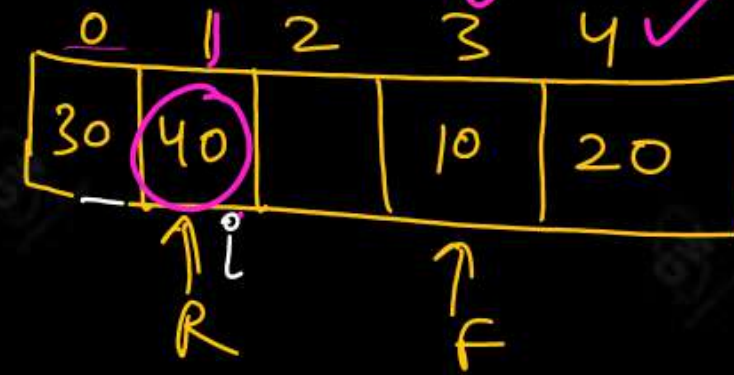


Array Implementation of Circular Queue

```
void display()
{ int i;
  if (front == -1 && rear == -1)
  { printf("Queue is empty\n"); }
  else
  { i = front;
    while (i != rear)
    { printf("%d\n ", queue[i]);
      i = (i + 1) % MAX;
    }
    printf("%d\n ", queue[i];)
  }
}
```

void peek() // Exactly same as in case of linear Queue

```
{ if (front == -1 && rear == -1)
  { printf("Queue is empty\n"); }
  else
  { printf("%d\n ", queue[front];)
  }
```



$0 \leq i \leq 4 \quad R=1$
 $3 + 1 \% 5 = 4$

Output

10 4 30 40

Array Implementation of Circular Queue

```
int main()
{
    int choice, x;
    while (1)
    {
        printf("\n--- Queue Menu (Linked List) ---\n");
        printf("1. Insert element\n");
        printf("2. Delete element\n");
        printf("3. Peek element\n");
        printf("4. Display queue\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice)
        {
            case 1: printf("Enter element to insert: ");
                    scanf("%d", &x);
```

```
                    enqueue(x);
                    break;
            case 2: dequeue();
                    break;
            case 3: peek();
                    break;
            case 4: display();
                    break;
            case 5: return 0;
            default: printf("Invalid choice\n");
        }
    }
}
```

Linked list Implementation of Circular Queue

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
struct node
```

```
{ int data;
```

```
  struct node *next;
```

```
};
```

```
struct node *front = 0;
```

```
struct node *rear = 0;
```

```
void enqueue (int x)
```

```
{
```

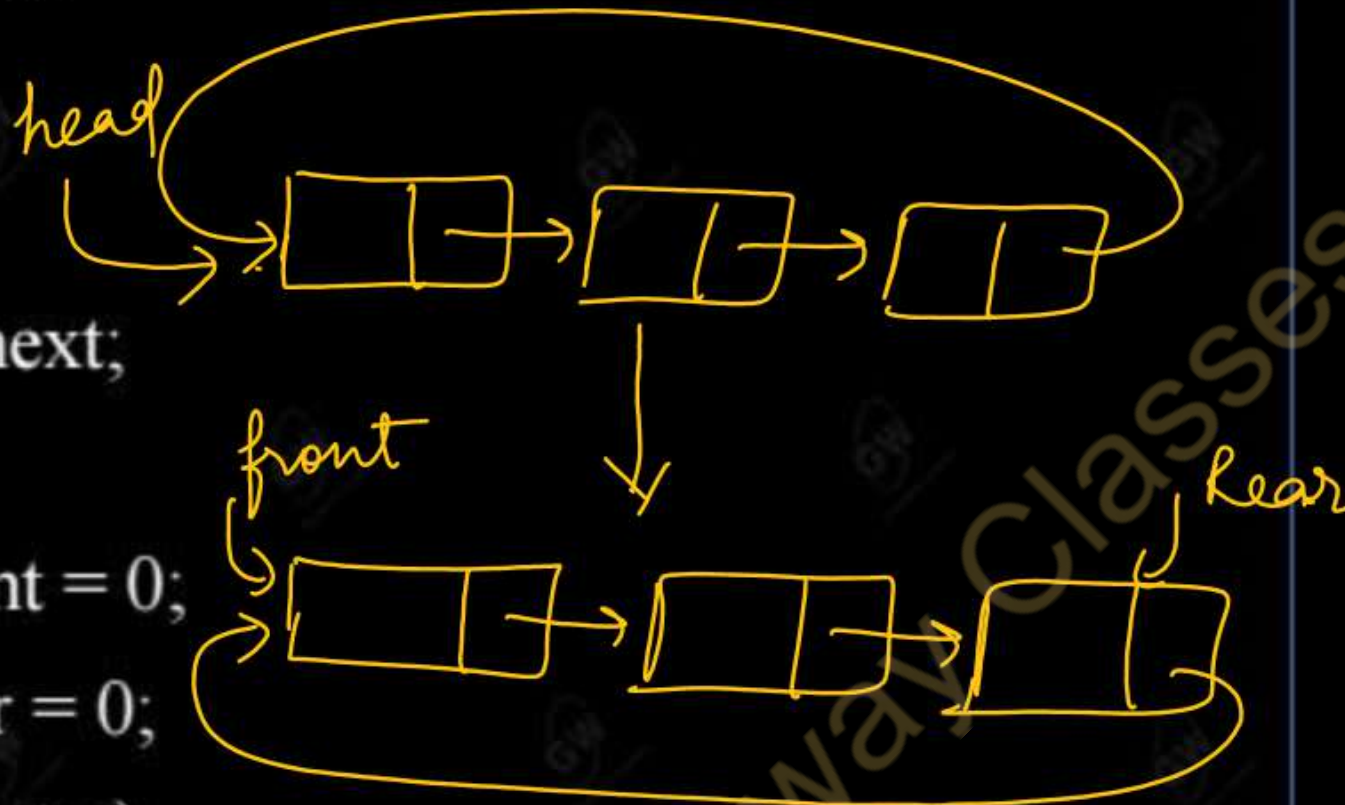
```
  struct node *newnode;
```

```
  newnode = (struct node*)malloc(sizeof(struct node));
```

```
  newnode -> data = x;
```

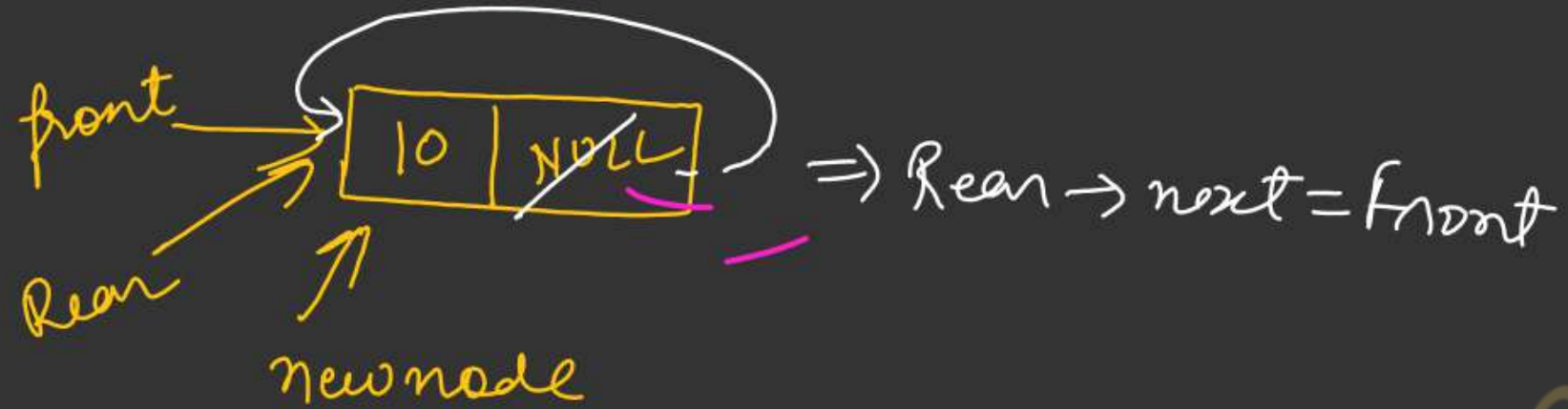
```
  newnode -> next = 0;
```

Circular LL

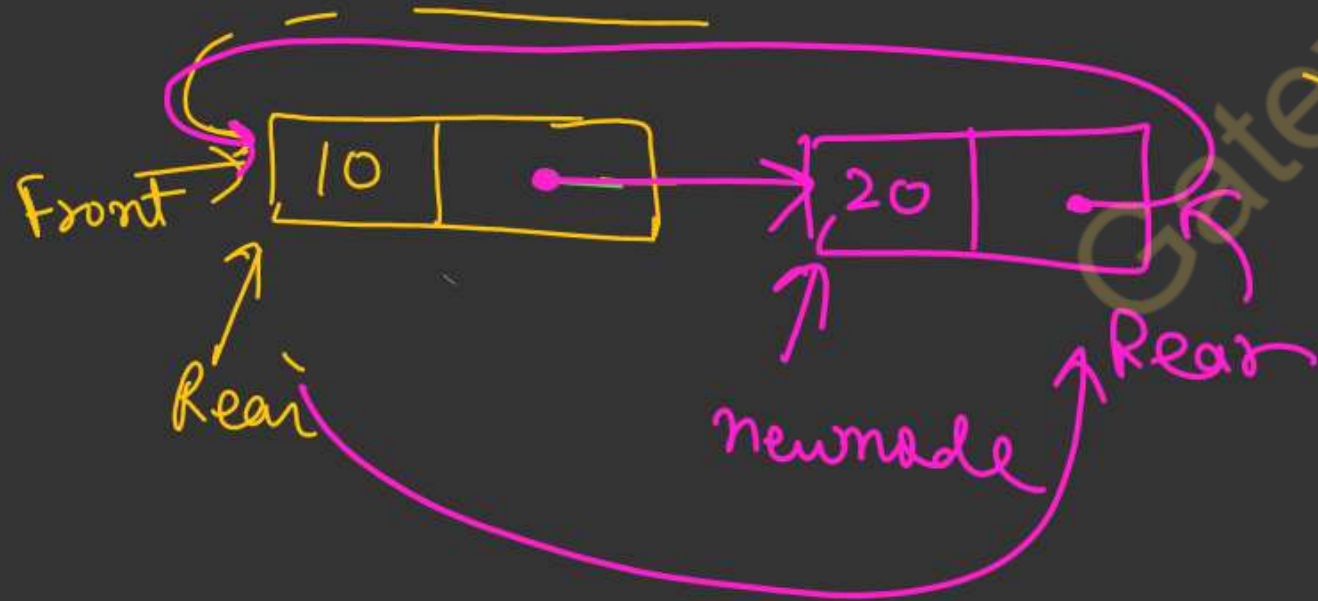


```
(I) if (front == 0 && rear == 0)
{
    front = rear = newnode;
    rear -> next = front;
}
else (II)
{
    rear -> next = newnode;
    rear = newnode;
    rear -> next = front;
}
}
```

Front $\rightarrow$ NULL
Rear $\rightarrow$ NULL



enqueue(20)



$\left\{ \begin{array}{l} \text{Rear} \rightarrow \text{next} = \text{newnode} \\ \text{Rear} = \text{newnode} \\ \text{Rear} \rightarrow \text{next} = \text{Front} \end{array} \right.$

Linked list Implementation of Circular Queue

```
void dequeue()
```

```
{ struct node *temp = front;
```

```
if (front == 0 && rear == 0) // Empty Queue
```

```
{ printf("underflow"); }
```

(I) else if (front == rear)

```
{ front = rear = 0;
```

```
free(temp);
```

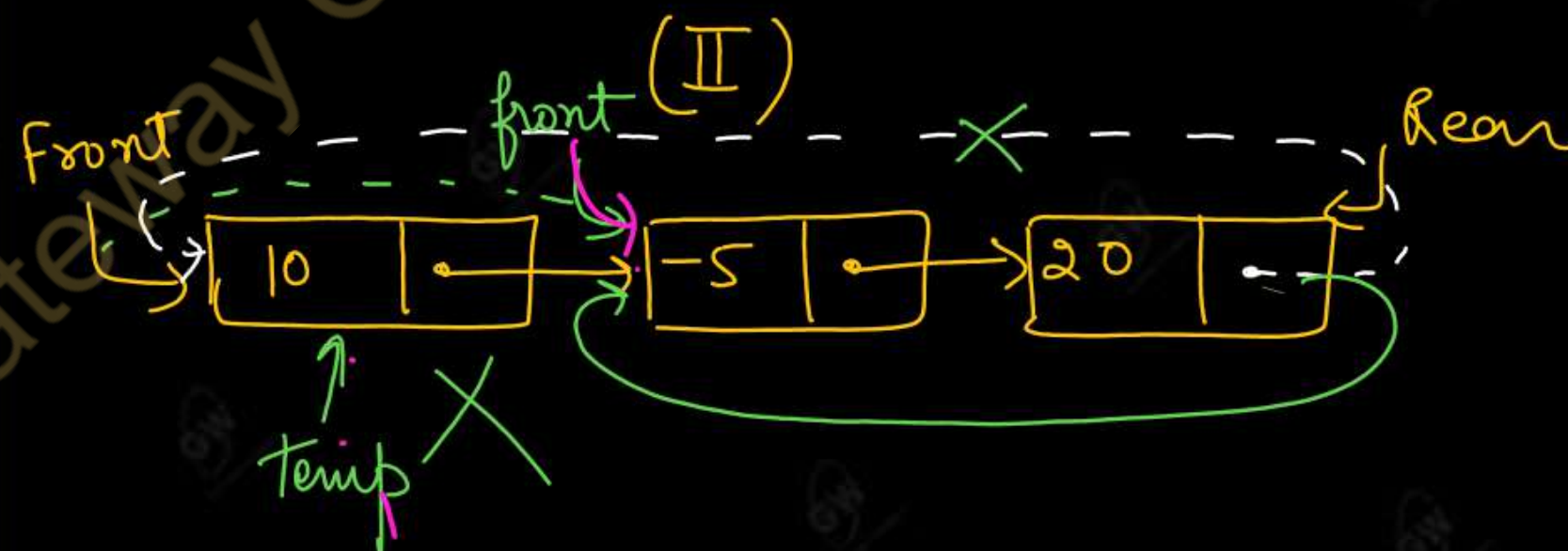
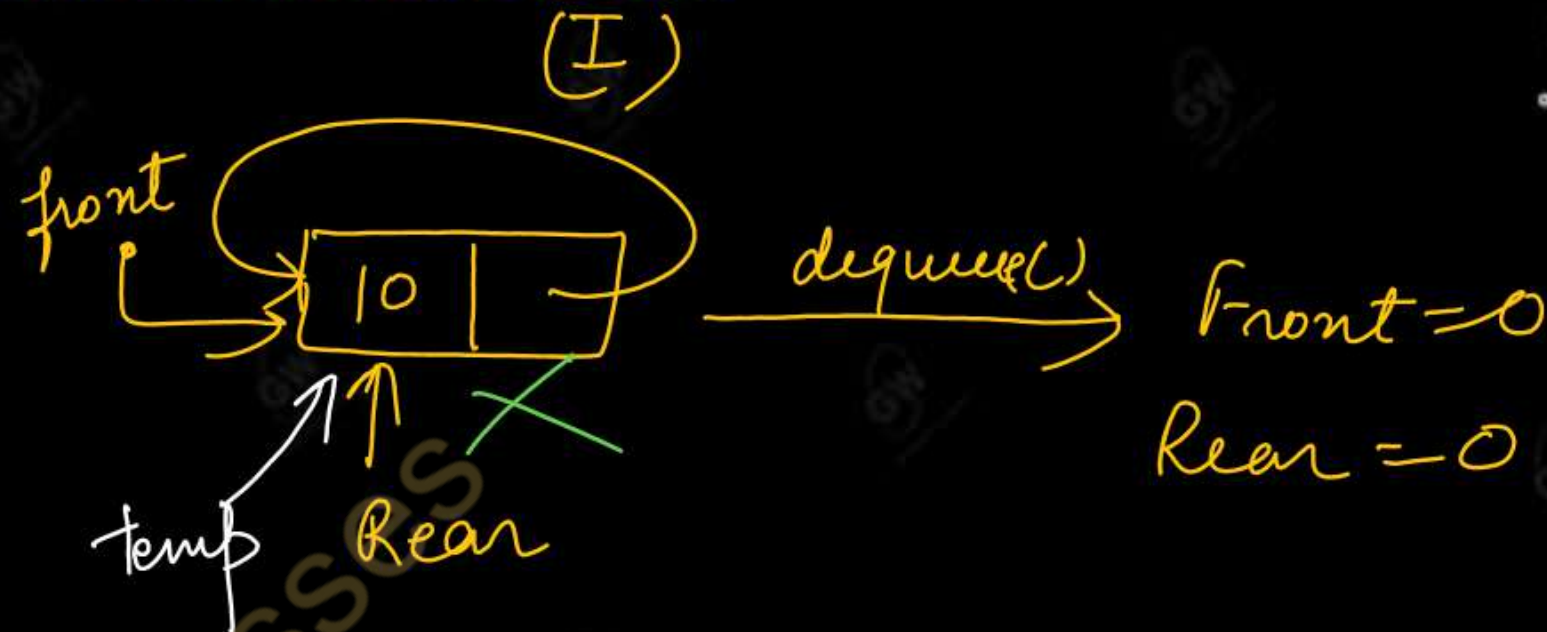
```
}
```

(II) else

```
{ front = front -> next;
```

```
rear -> next = front;
```

```
free(temp); }
```



Linked list Implementation of Circular Queue

```
void display()
```

```
{ struct node *temp;
```

```
temp = front;
```

```
if (front == 0 && rear == 0)
```

```
{ printf("underflow"); }
```

```
else
```

```
{ while (temp -> next != front) X
```

```
{ printf("%d ", temp -> data);
```

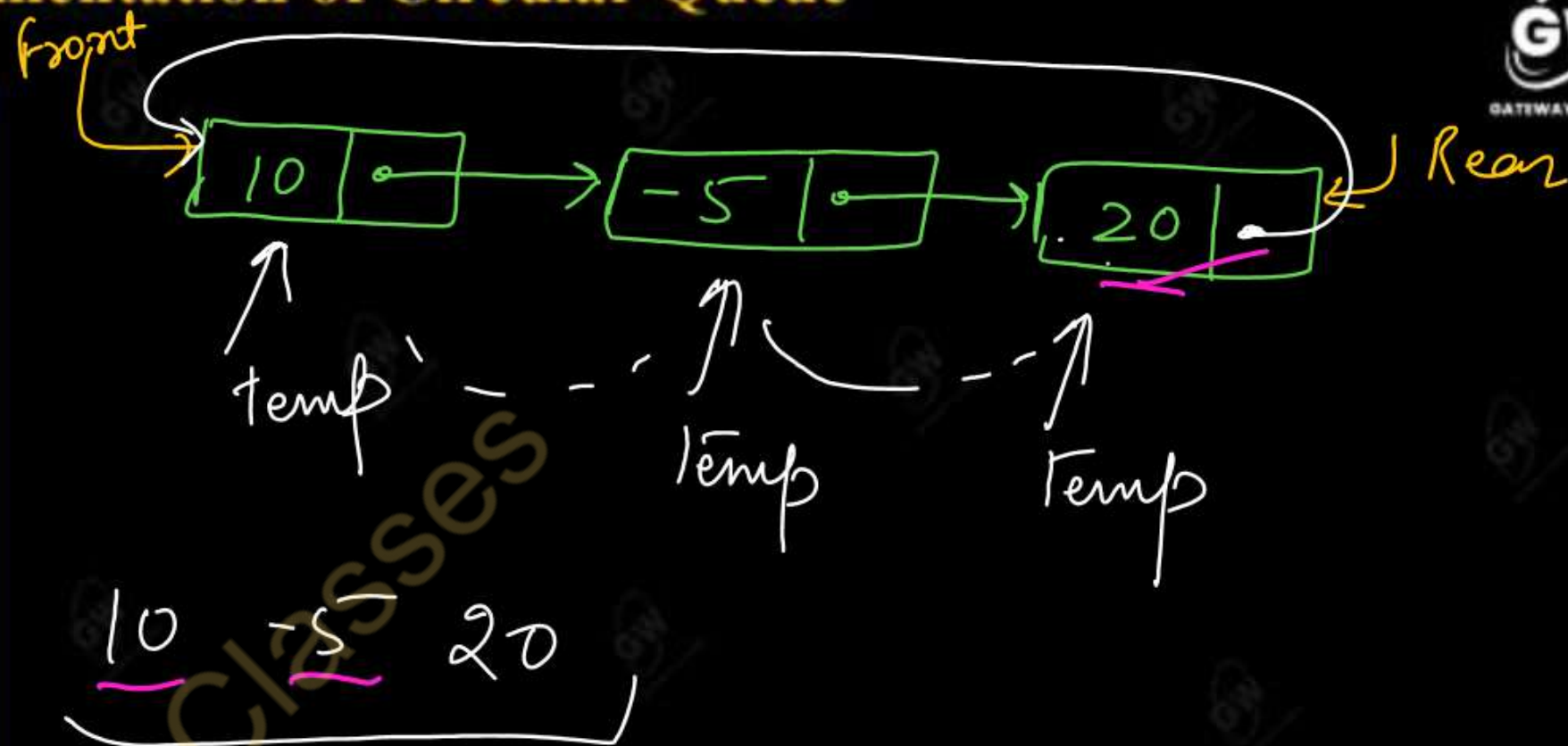
```
temp = temp -> next;
```

```
}
```

```
printf("%d ", temp -> data);
```

```
}
```

```
}
```



```
void peek()
{
    if (front == 0 && rear == 0)
    {
        printf("Queue is empty\n");
    }
    else
    {
        printf("Front element is: %d\n", front -> data);
    }
}
```

```
int main()
{
    int choice, x;
    while (1)
    {
        printf("\n--- Queue Menu (Linked List) ---\n");
        printf("1. Insert element\n");
        printf("2. Delete element\n");
        printf("3. Peek element\n");
        printf("4. Display queue\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice)
        {
            case 1: printf("Enter element to insert: ");
                    scanf("%d", &x);
```

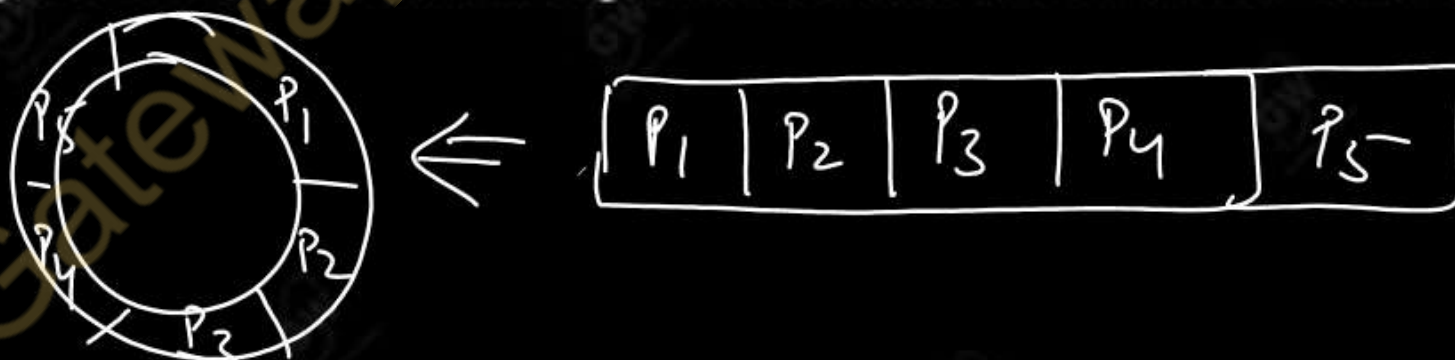
```
                    enqueue(x);
                    break;
            case 2: dequeue();
                    break;
            case 3: peek();
                    break;
            case 4: display();
                    break;
            case 5: return 0;
            default: printf("Invalid choice\n");
        }
    }
}
```

➤ Buffer in Computer Systems:

- Computer systems provide a holding area for maintaining communication between two processes, two programs, or even two systems.

➤ CPU Scheduling:

- In Round-Robin Scheduling Algorithm, a circular queue is utilized to maintain processes that are in a ready state.



➤ Traffic System:

- Circular queue is also utilized in traffic systems that are controlled by computers. Each traffic light turns ON in a circular incrementation pattern in a constant interval of time.

Data Structure

Unit-2 DPP-10

1. Write the condition for empty and full circular queue. **(AKTU2022-23)**
2. What is circular Queue? Write a C code to insert an element in circular queue? **(AKTU2022-23)**
3. Define priority queue. Given one application of priority queue. **(AKTU2019-20,2018-19)**
4. Write the syntax to check whether a given circular queue is full or empty? **(AKTU2019-20)**
5. Write a C program to implement the array representation of circular queue. **(AKTU2017-18)**



AKTU

B.Tech III-Sem



GATEWAY CLASSES

CS IT & CS Allied

DATA STRUCTURE

Unit-2 Lecture-11

Today's Target

- Double Ended Queue (Deque) ✓
- Circular Array Implementation of Deque
- Priority queue ✓
- AKTU PYQs
- DPP



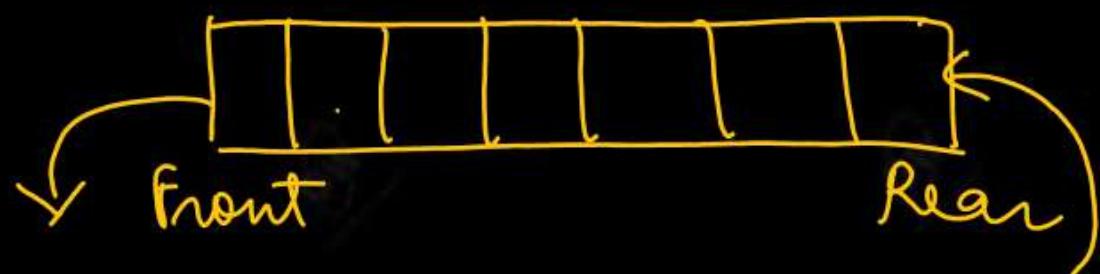
By Dr. Nidhi Parashar Ma'am

Ph.D (CSE)

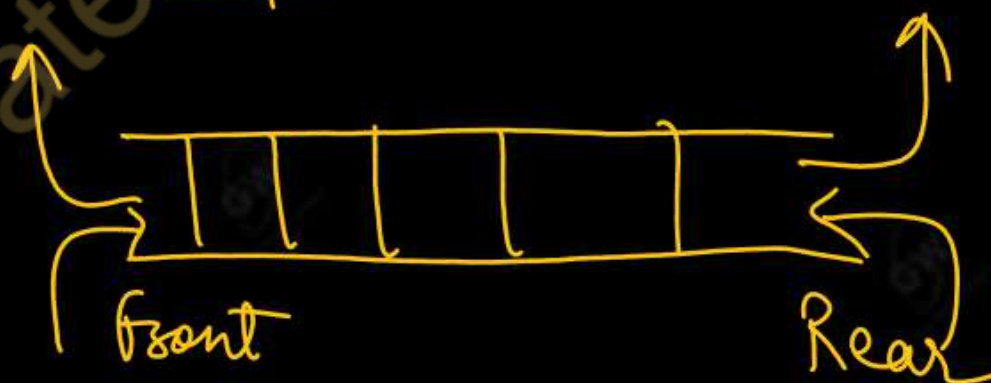
- M.Tech Gold Medalist
- Net Qualified

- Deque is a linear data structure and a generalized version of Queue that allows the insertion and deletion of elements from both ends (**front**, and **rear**).
- Deque can act as both Stack and Queue.
- It is useful in many problems where we need to have a subset of all operations also like insert/remove at front and insert/remove at the end.
- It is typically implemented either using a doubly linked list or circular array.

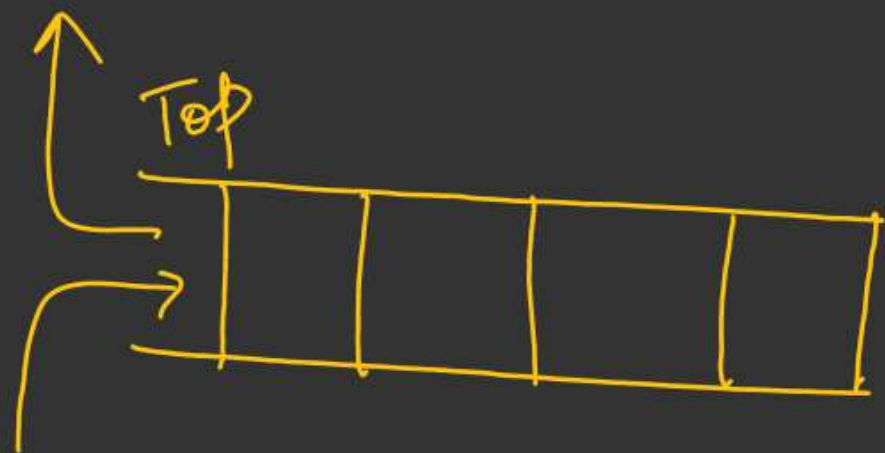
Linear Queue



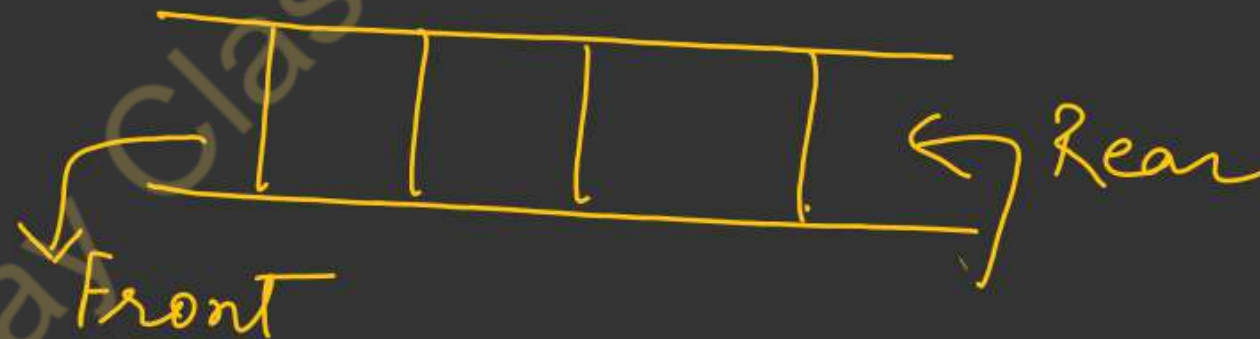
Deque



Deque as Stack



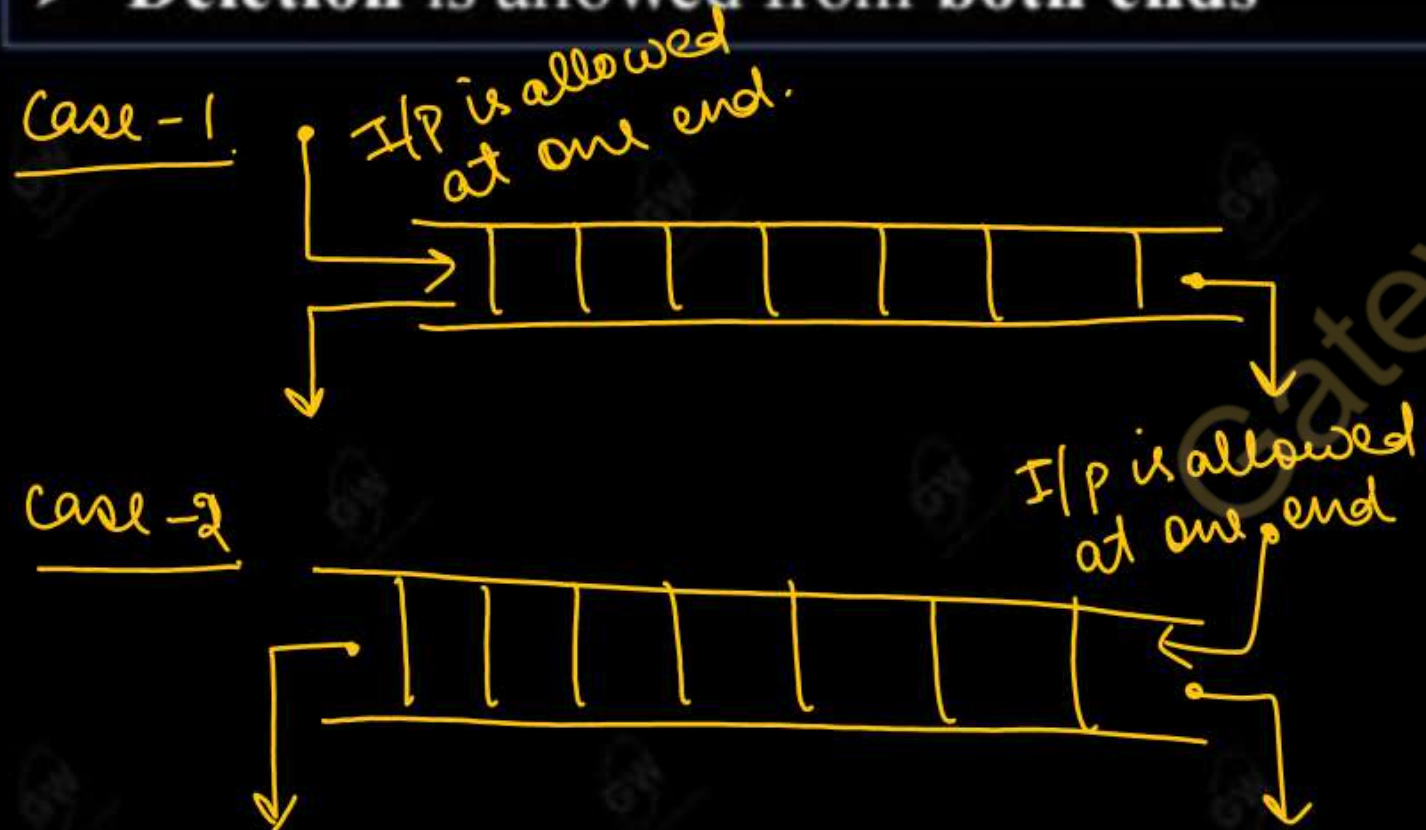
Deque as Queue



1. Input-Restricted Deque

This deque type restricts the insertion operation, but it is flexible when removing elements.

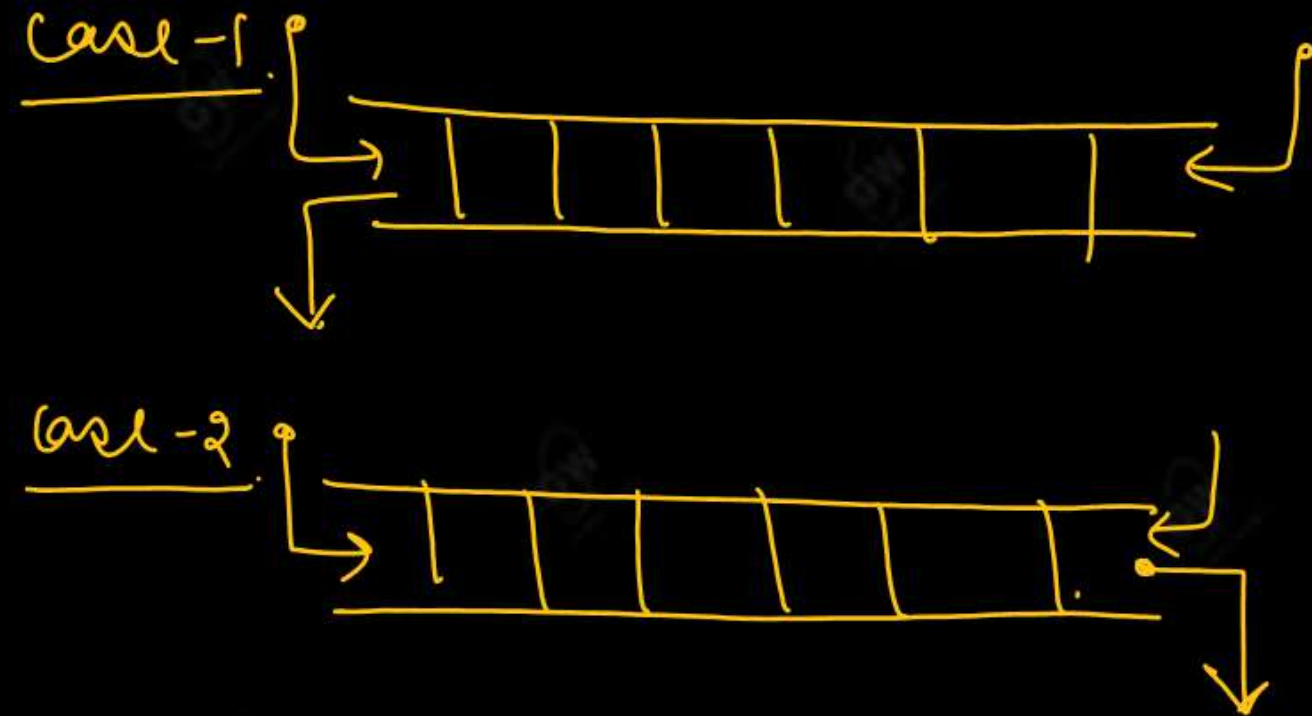
- **Insertion** is allowed only at **one end** (either the front or rear).
- **Deletion** is allowed from **both ends**



2. Output-Restricted Deque

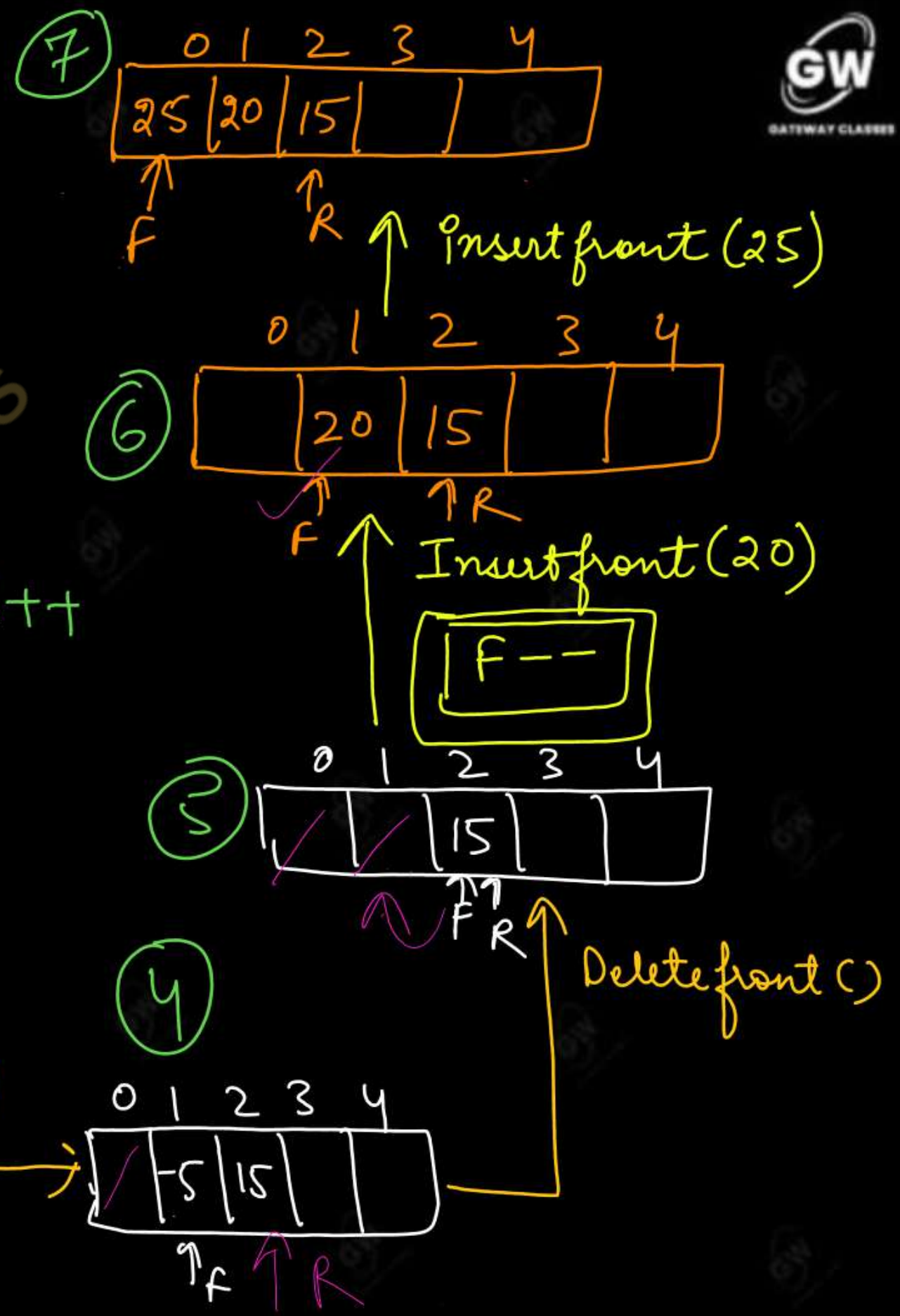
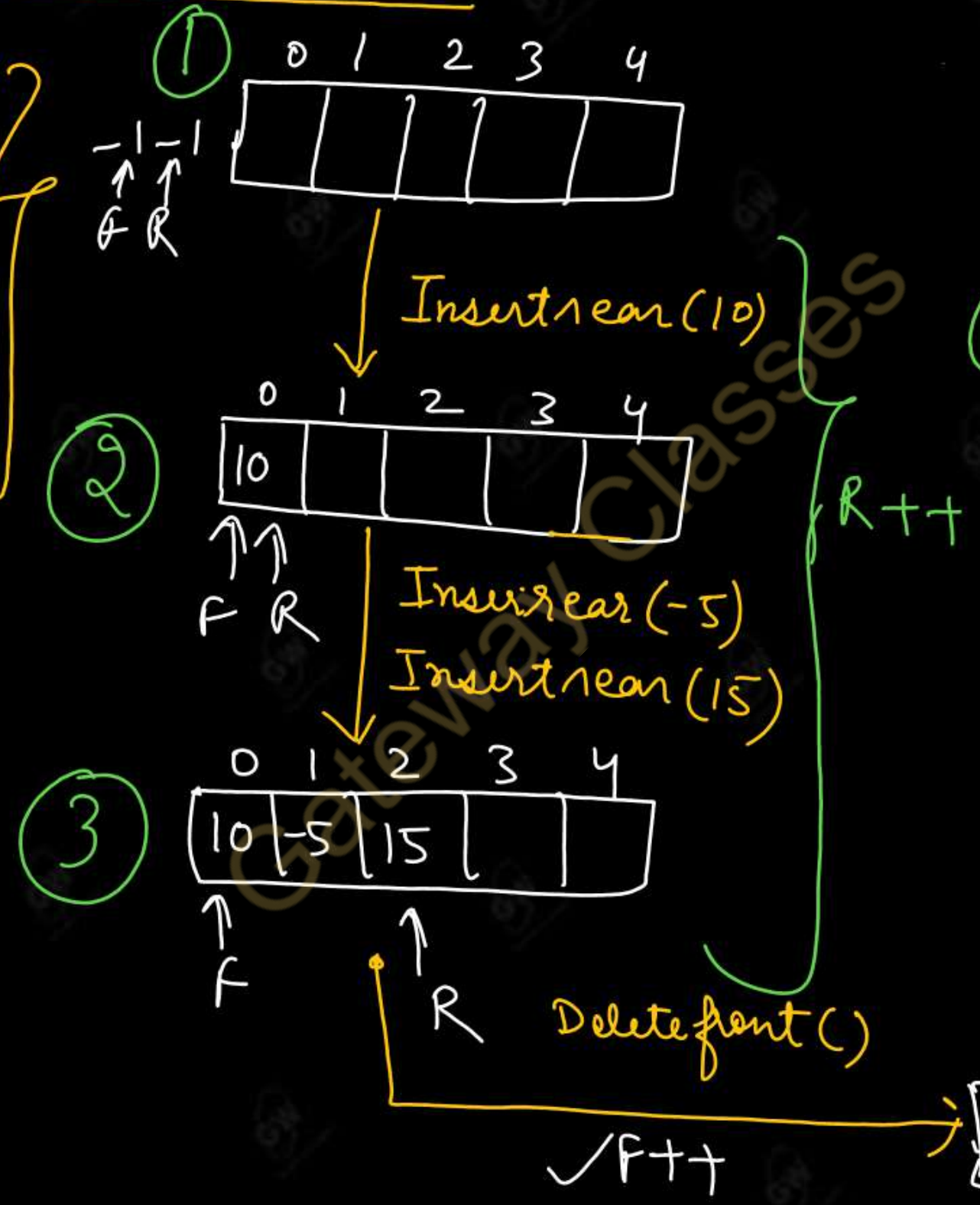
This deque type restricts the deletion operation, but it is flexible when adding elements.

- **Deletion** is allowed only at **one end** (either the front or rear).
- **Insertion** is allowed at **both ends**.

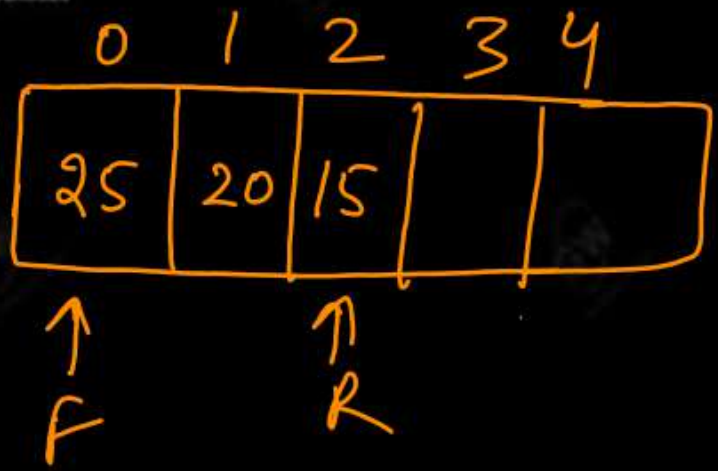


operations performed on Deque

- 1) insert front ()
- 2) insert rear ()
- 3) Delete front ()
- 4) Delete rear ()
- 5) Display ()
- 6) get front ()



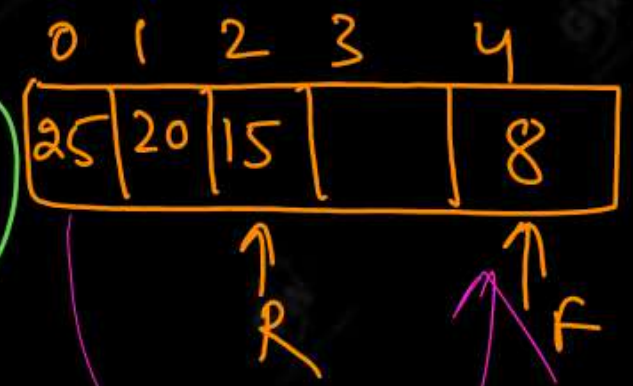
8



if $(F=0)$
then if you want
to insert at front
then
Set $F = n-1$.

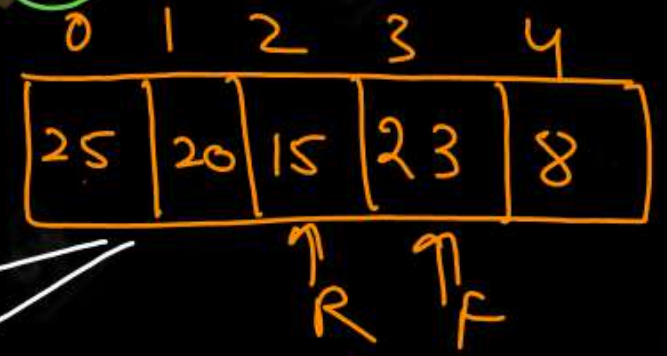
↓ insert front (8)

9



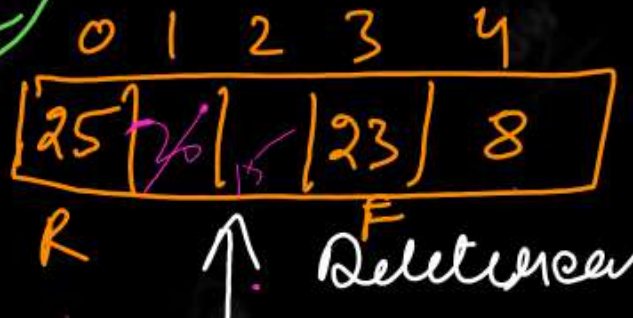
insert at front (23)

10



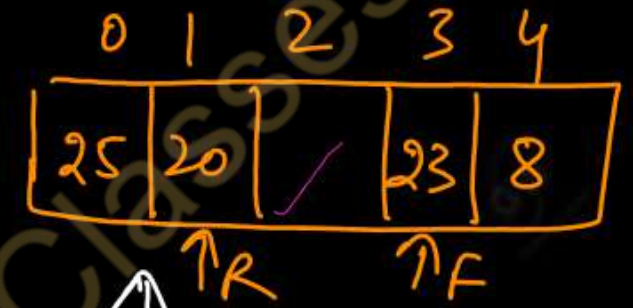
① Here Queue is full
($F=0 \& R=N-1$)
② $(F == R+1)$

12



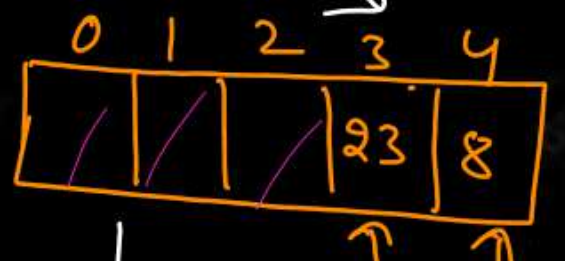
while deleting from Rear
if at present $R=0$
then
Set $R = n-1$.

11



delete rear()

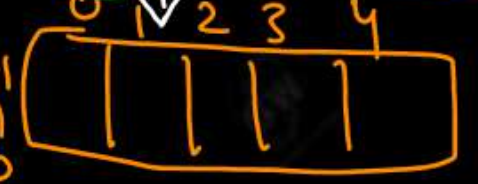
13



Delete from Rear()

Delete from Rear()

15



14

CircularArray Implementation of Deque

```
#define N 5
int deque[N];
int F = -1, R = -1;
```

void enqueueFront(int x)

```
{ if((F == 0 && R == N-1) || (F == (R + 1))) // Overflow condition
{ printf("Queue is full"); }
```

```
else if(F == -1 && R == -1) // Queue is empty
{ F = R = 0;
```

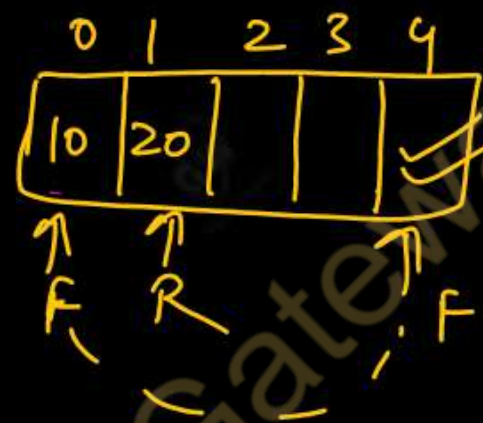
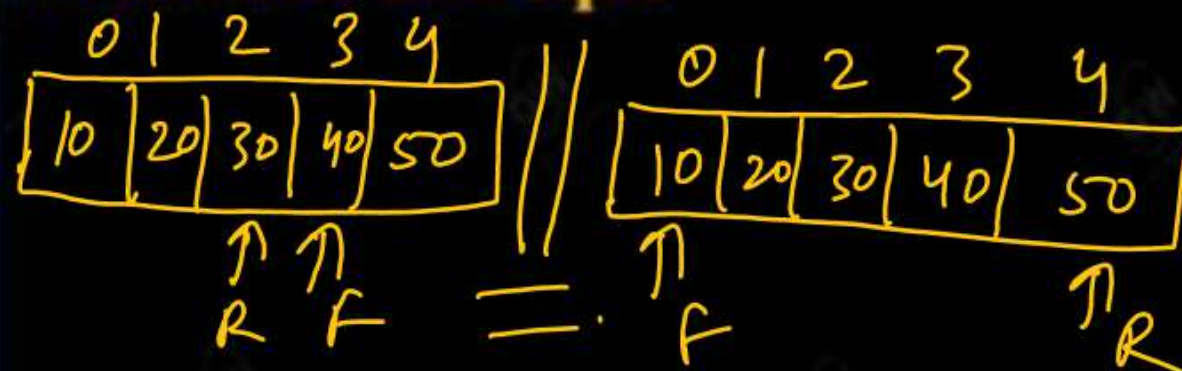
```
  deque[F] = x;
```

```
}
else if(F == 0)
```

```
{ F = N - 1;
  deque[F] = x;
}
```

```
else
```

```
{ F--;
  deque[F] = x;
}}
```



CircularArray Implementation of Deque

```
void enqueueRear(int x)
```

```
{
```

```
if((F == 0 && R == N-1) || (F == R+1))
```

```
{ printf("Queue is full"); }
```

```
else if(F == -1 && R == -1)
```

```
{ F = R = 0;
```

```
  deque[R] = x;
```

```
}
```

```
else if(R == N-1)
```

```
{ R = 0;
```

```
  deque[R] = x;
```

```
}
```

```
else
```

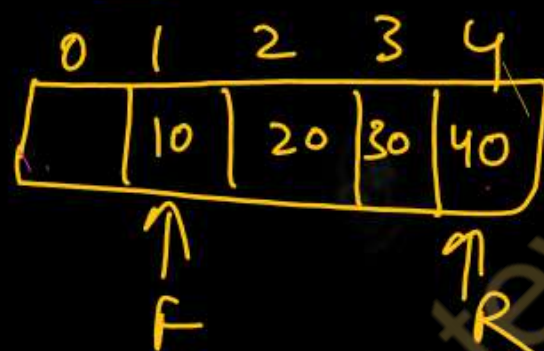
```
{ R++;
```

```
  deque[R] = x;
```

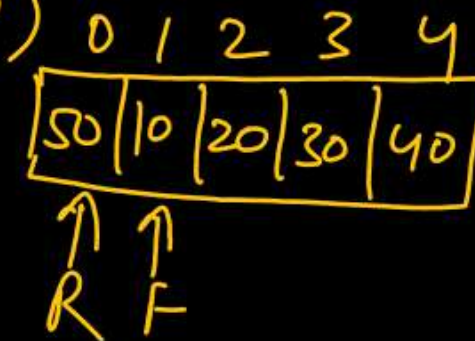
```
}
```

```
}
```

// Queue full condition



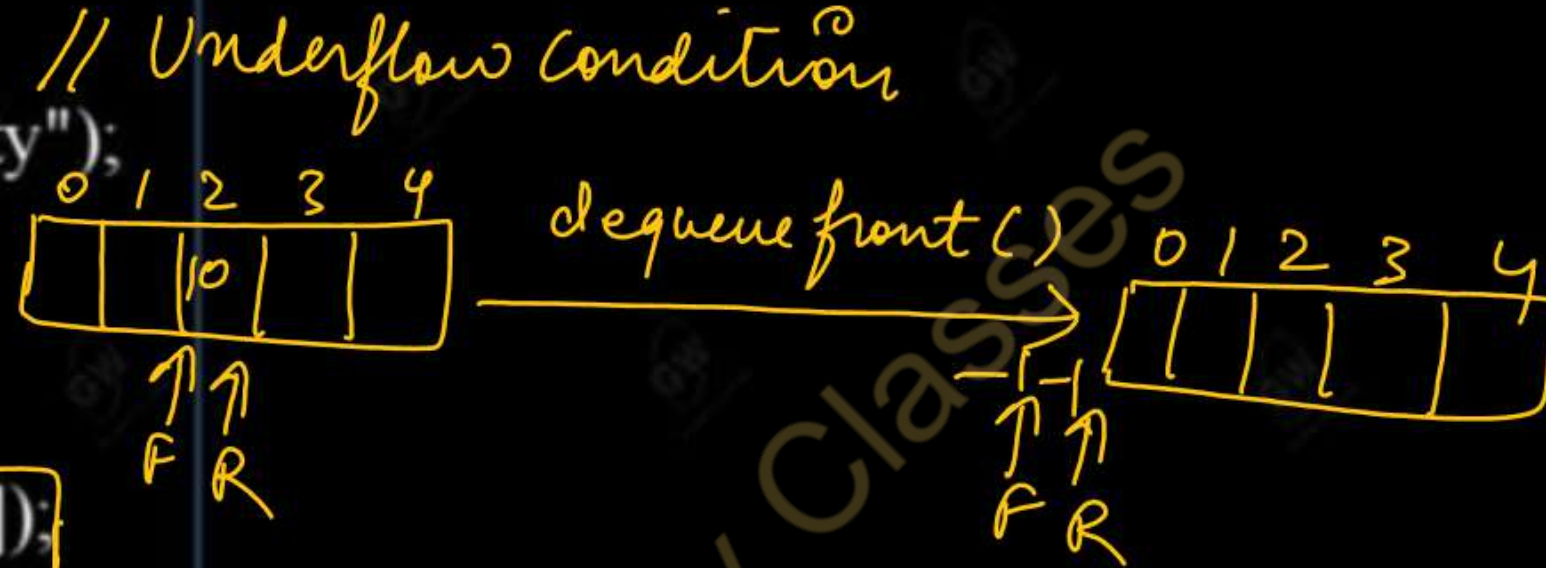
enqueueRear(50)



CircularArray Implementation of Deque

```
void dequeueFront()
```

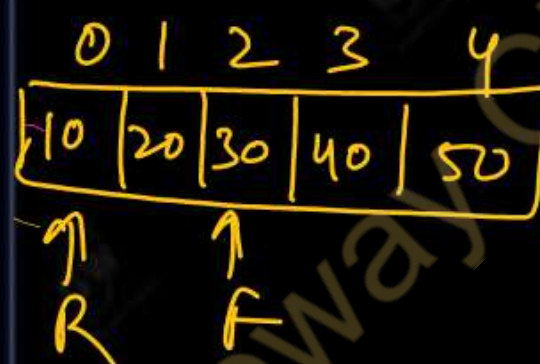
```
{
    if(F == -1 && R == -1) // Underflow condition
        printf("Queue is empty");
    else if(F == R)
    {
        F = R = -1;
    }
    else if(F == N-1)
    {
        printf("%d", deque[F]);
        F = 0;
    }
    else
    {
        printf("%d", deque[F]);
        F++;
    }
}
```



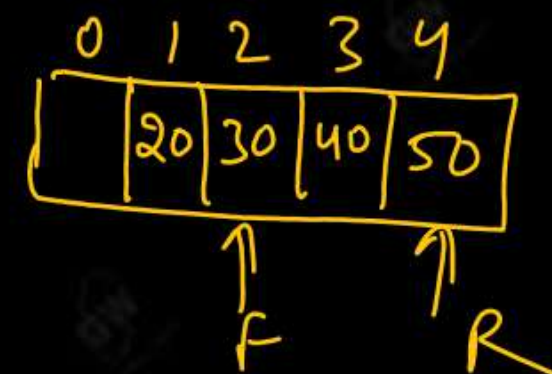
void dequeueRear()

```
{
  if(F == -1 && R == -1)
    printf("Underflow");
  else if(F == R)
  { F = R = -1; }
  else if(R == 0)
  { printf("%d", deque[R]);
    R = N - 1;
  }
  else
  { printf("%d", deque[R]);
    R--;
  }
}
```

// Queue Empty condition



deleteRear()

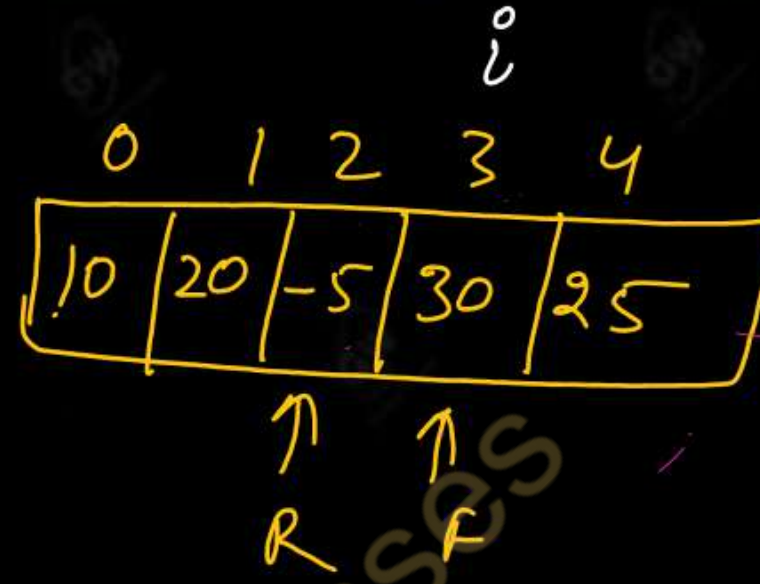


void display()

```
{
    int i = F;
    while(i != R)
    {
        printf("%d ", deque[i]);
        i = (i + 1) % N;
    }
    printf("%d", deque[R]);
}
```

void getFront()

```
{
    printf("%d", deque[F]);
}
```

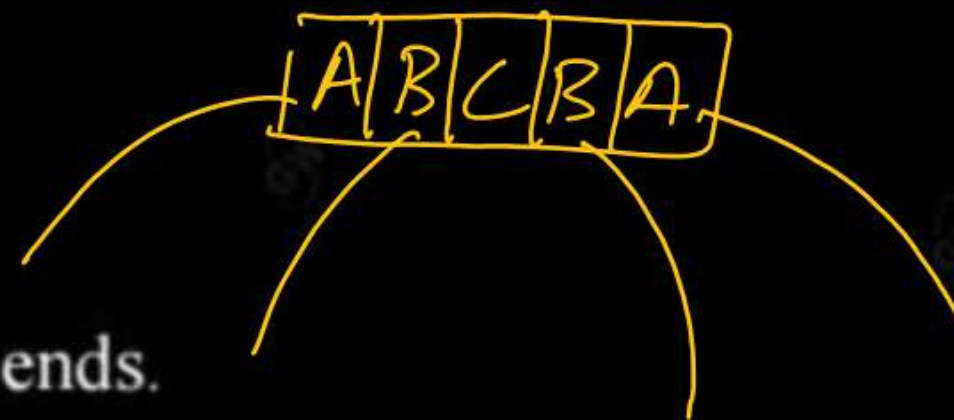


output -

30 25 10 20 -5

1. Implementing other data structures

- **Stack:** By restricting operations to one end.
- **Queue:** By restricting insertion at rear and deletion at front.
- **Priority Queue:** By inserting based on priority at appropriate ends.



2. Palindrome Checking

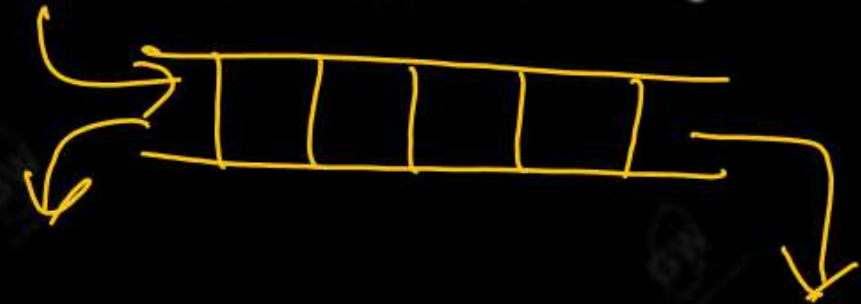
- Since we can add/remove elements from both ends, deques are useful for checking whether a string is a palindrome.

3. Undo/Redo Functionality

- Text editors, browsers, and software often use deques to maintain history of operations.
- **Undo** = pop from rear, **Redo** = push back.

4. Job Scheduling

- In OS, deques are used for scheduling processes (**work stealing** in multiprocessor scheduling).
- Each processor keeps a deque of tasks, where it can push/pop from one end, and idle processors can "steal" tasks from the other end.



5. Cache Implementation

- Deques are often used in **LRU (Least Recently Used)** cache implementations to quickly remove the least recently used item (front) and add new items (rear).

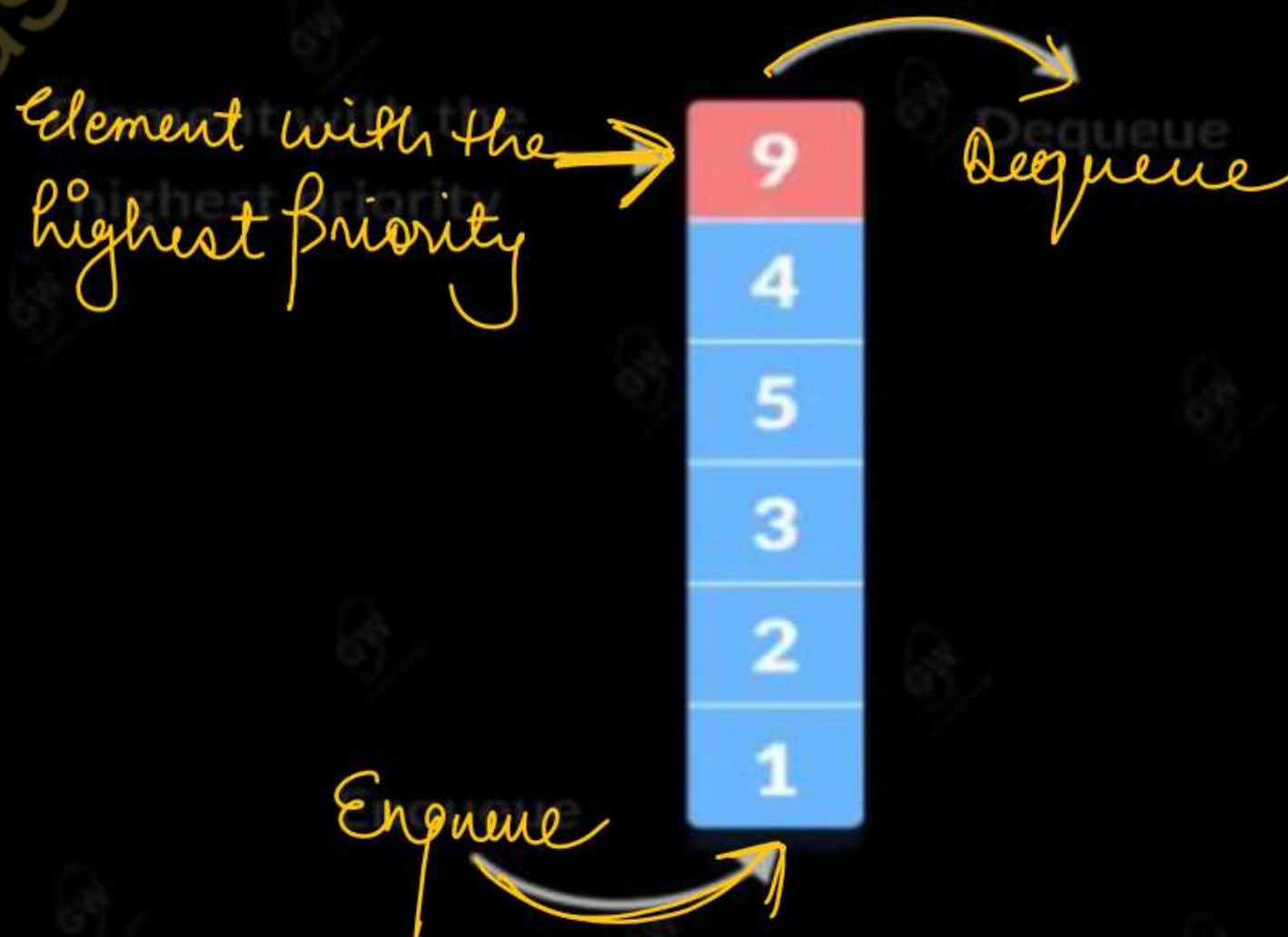
6. Real-Time Systems

- Used in scenarios where both FIFO (first-in-first-out) and LIFO (last-in-first-out) policies are required dynamically.
- Example: Task scheduling in embedded systems.

- A **special type of queue** in which each element is associated with a **priority value**.
- Elements are served on the basis of their priority. That is, higher priority elements are served first.
- However, if elements with the same priority occur, they are served according to their order in the queue.

Assigning Priority Value

- Generally, value of element itself is considered for assigning priority.
- The element with the highest value is considered the highest priority element.
- However, we can assume the element with lowest value as the highest priority element.



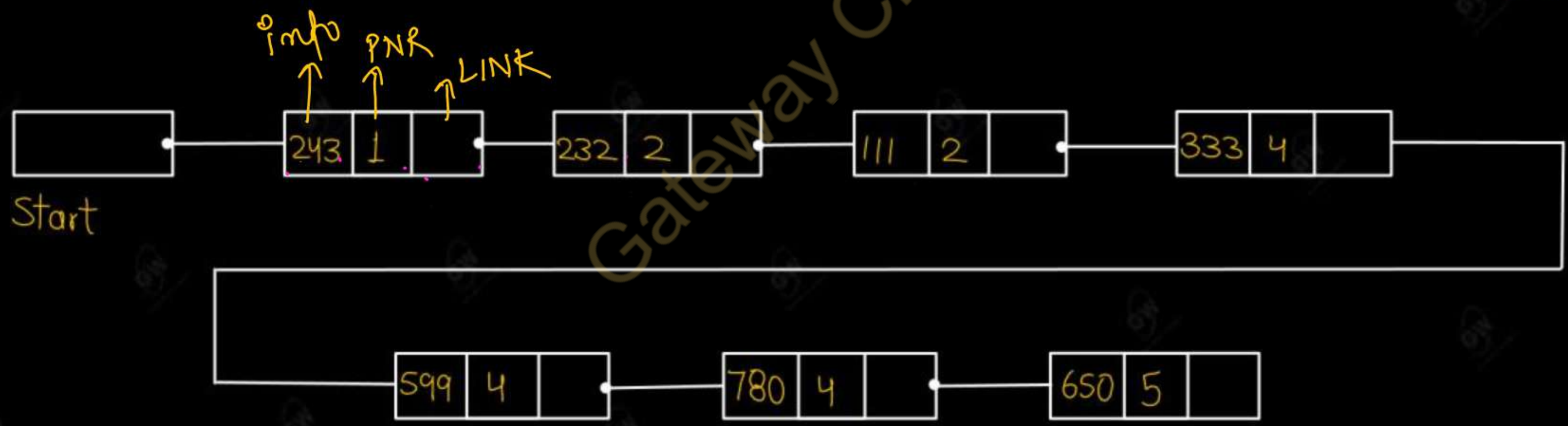
Characteristics of a Priority Queue in Data Structures

- Every element in the queue has some **priority assigned**.
- When we add an item, it is inserted in a position based on its priority.
- All the queue elements must be comparable.
- The higher priority element is dequeued before the one with low priority.
- More than one element has same priority, the element will be served as per the First Come First Serve Principle.
- Priority Queue is used in algorithms such as Dijkstra's algorithm, Prim's algorithm, and Huffman Coding.

Example

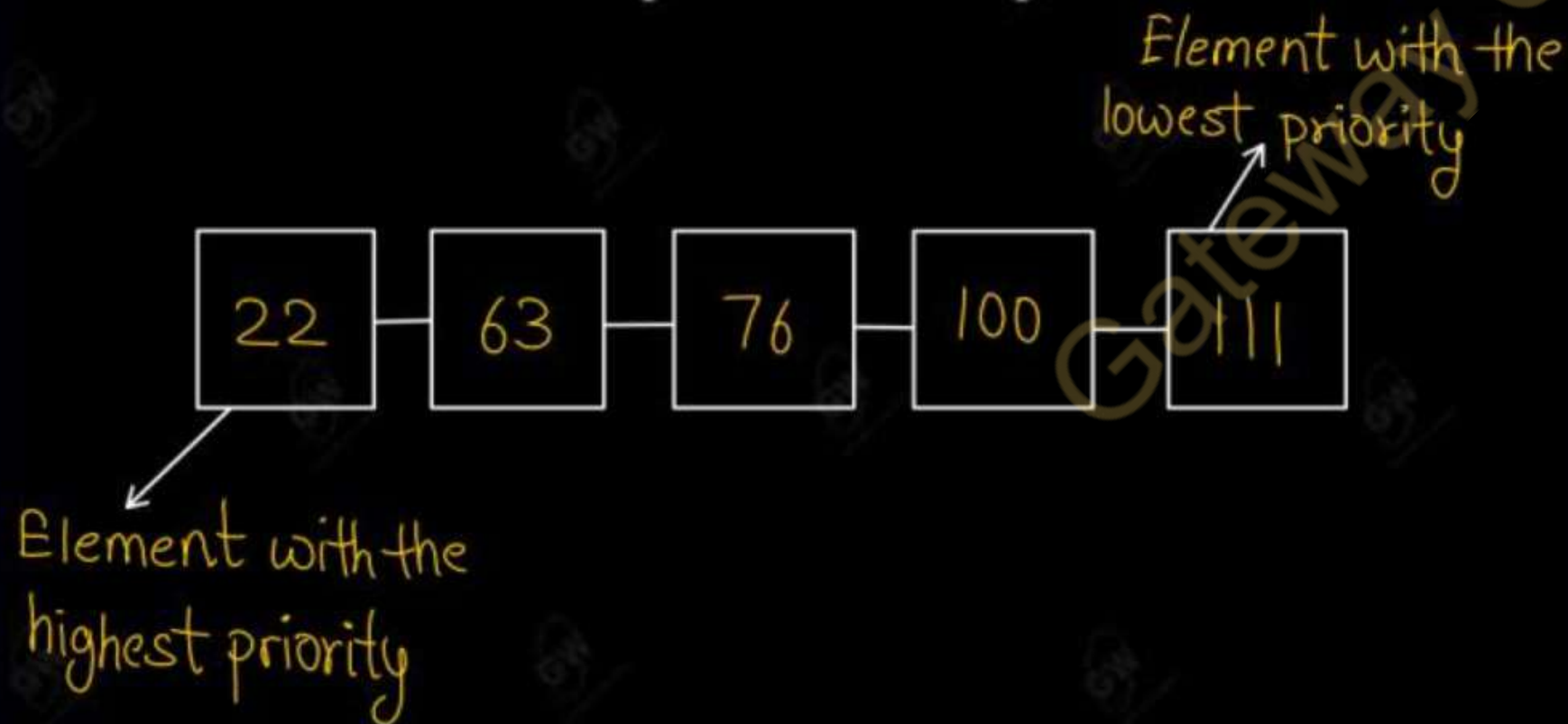
	INFO	PNR	LINK
0	232	2	4
1	333	4	6
2	599	4	2
3	243	1	0
4	111	2	5
5	650	5	1
6	780	4	

lowest Number
↓
means highest priority



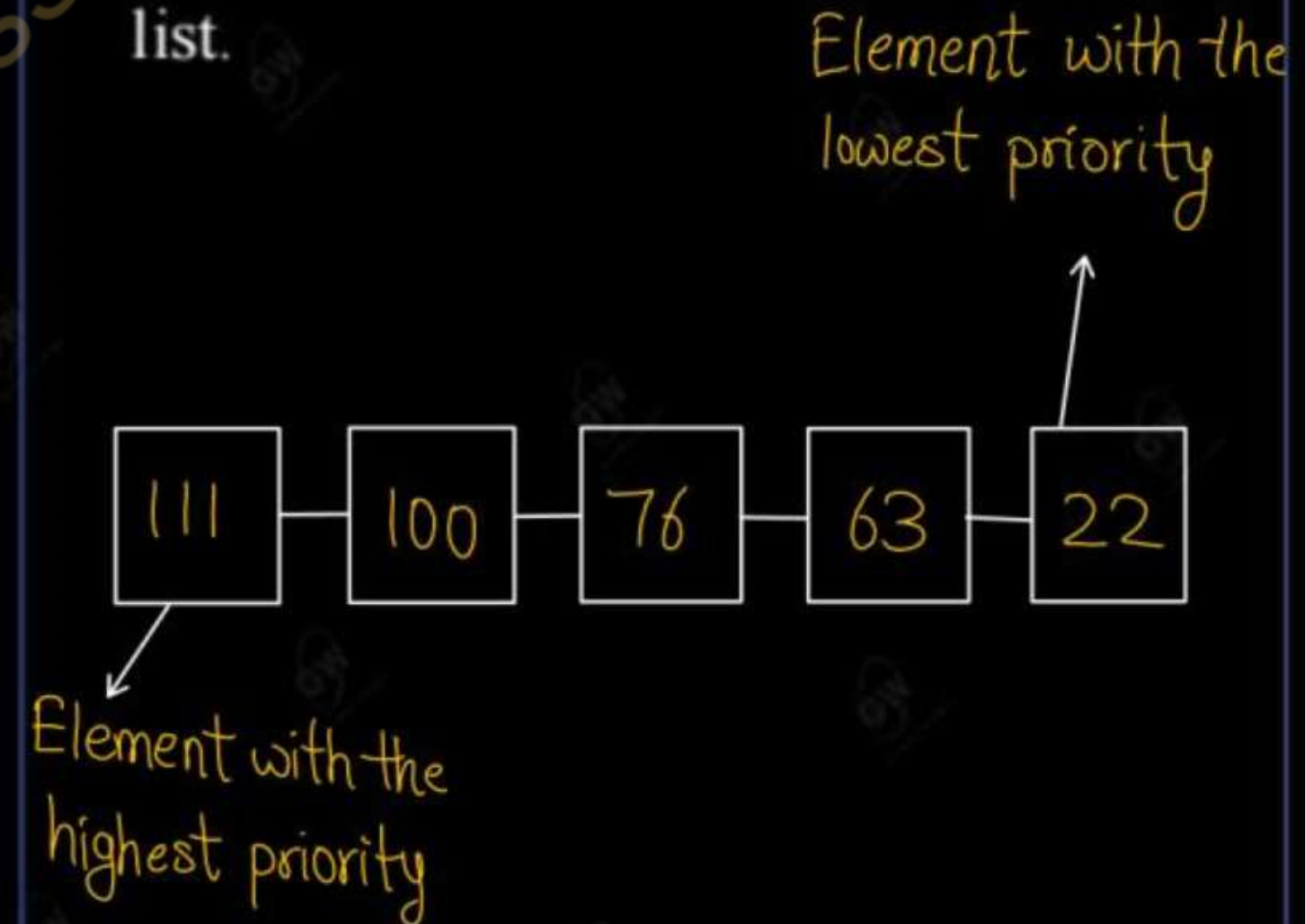
1. Ascending Order Priority Queue

- Element with lower priority value is given a higher priority in the priority list.
- when we dequeue from priority queue, 22 will be removed from the queue and dequeue returns 22.



2. Descending Order Priority Queue

- Element with a higher priority number is given as a higher priority in a priority list.



1. Hospital emergency room: A medical facility provides immediate treatment to patients with health conditions or injuries, often prioritizing cases based on urgency.

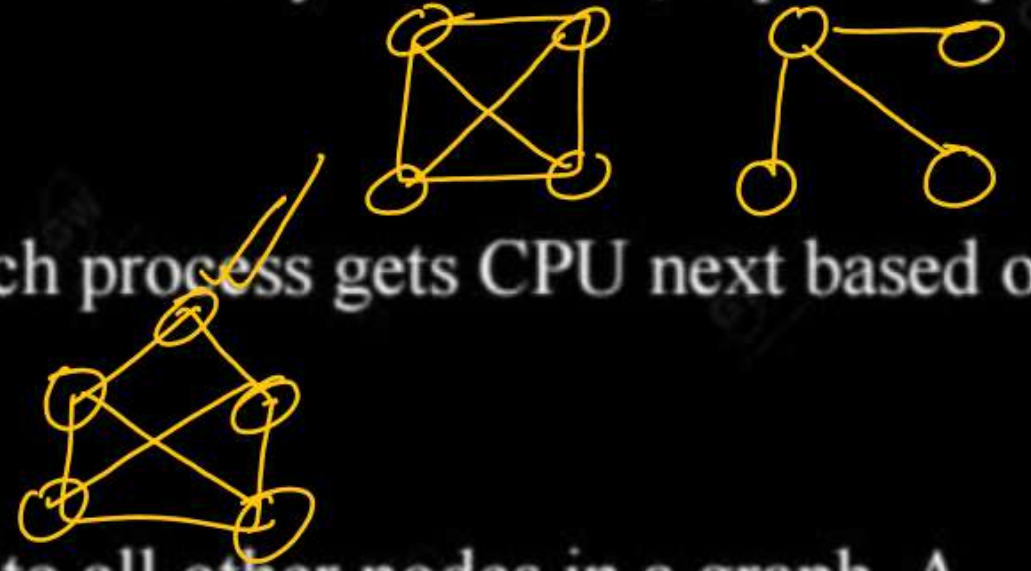
2. Task scheduling: Priority queues ensure that high-priority tasks such as system tasks, input/output operations, or critical applications are processed first.

3. Process management: The priority queue is used to decide which process gets CPU next based on the process priority.

4. Dijkstra's algorithm: To find shortest path from a source node to all other nodes in a graph. A priority queue is used to efficiently select the next node with the smallest tentative distance.

5. Prim's algorithm: To find minimum spanning tree (MST) in a weighted, undirected graph. A priority queue is used to select the next edge with the smallest weight to add to the MST.

6. Heap sort: A sorting algorithm that uses a binary heap (a type of priority queue) to sort elements.



Data Structure

Unit-2 DPP-11

1. Estimate A double ended Queue (deque) is a linear list in which additions may be made at either end. Obtain a data representation mapping a deque into one dimensional array. Write C function to add and delete elements from either end of deque. (AKTU2023-24)
2. Define priority queue. Given one application of priority queue. (AKTU2019-20,2018-19)
3. What is the significance of priority queue? (AKTU2017-18)



Gateway Classes



Empowering Learners, Transforming Futures

B.Tech Courses



GET IT ON
Google Play



Download on the
App Store

Helpline No- 7819 0058 53